

Pedro Henrique Scheidt Prazeres

Análise Comparativa de Modelos de Linguagem para Detecção de Phishing por E-Mail

Florianópolis

2026

Pedro Henrique Scheidt Prazeres

Análise Comparativa de Modelos de Linguagem para Detecção de Phishing por E-Mail

Trabalho de Conclusão de Curso apresentado ao Curso de Sistemas de Informação da Universidade Federal de Santa Catarina como requisito parcial para a conclusão da disciplina INE5660 — Trabalho de Conclusão de Curso II.

Universidade Federal de Santa Catarina
Centro Tecnológico
Departamento de Informática e Estatística
Curso de Sistemas de Informação

Orientador: Prof. Ricardo Felipe Custodio, Dr.

Florianópolis
2026

PRAZERES, PEDRO HENRIQUE SCHEIDT

Análise Comparativa de Modelos de Linguagem para Detecção de Phishing por E-Mail / Pedro Henrique Scheidt Prazeres ; orientador, Prof. Ricardo Felipe Custodio, Dr. – Florianópolis, 2026.

132 p.

Trabalho de Conclusão de Curso – Universidade Federal de Santa Catarina, Centro Tecnológico, Programa de Graduação em Sistemas de Informação, Florianópolis, 2026.

1. Segurança da Informação. 2. Modelos de Linguagem de Larga Escala. 3. Detecção de Phishing. 4. Privacidade de Dados. 5. Classificação de E-mails. I. Custodio, Ricardo Felipe. II. Universidade Federal de Santa Catarina. III. Título.

Pedro Henrique Scheidt Prazeres

Análise Comparativa de Modelos de Linguagem para Detecção de Phishing por E-Mail

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do título de Bacharel em Sistemas de Informação e aprovado em sua forma final pela Banca Examinadora.

Florianópolis, _____ de _____ de 2026.

Banca Examinadora:

Prof. Ricardo Felipe Custodio, Dr.
Orientador — UFSC

Prof(a).

UFSC

Prof(a).

Membro externo

Dedico esta obra a meus pais e minha irmã, que me acompanharam em todos os passos do caminho, me ajudando a levantar quando caí e me encorajando a continuar andando, quando seria mais fácil parar.

AGRADECIMENTOS

Agradeço a Deus, pela minha vida.

Agradeço também imensamente aos meus pais Andrea Maria Arantes Scheidt Prazeres e Ricardo Janning Prazeres por todo o esforço e sacrifícios que realizaram ao longo da vida, me oferecendo todas as oportunidades que eu precisava, serei eternamente grato por tudo que fizeram.

Junto deles, quero estender também minha gratidão à minha irmã Isabela e cunhado Artur. Os quatro me encorajaram ao longo de minha jornada, sempre me convencendo a enxergar meu próprio potencial, mesmo nos momentos mais difíceis. Expresso minha gratidão também aos meus sobrinhos, Francisco e Cecília, que me fazem sorrir independentemente do meu humor, simplesmente por estarem lá.

Agradeço aos meus padrinhos Ricardo e Rosane por estarem sempre presentes na minha vida.

Sou grato também aos meus amigos, em especial ao João Pedro Garcia, que me auxiliou na redação deste documento, mesmo nos dias que estava cansado do trabalho, e certamente preferiria descansar. Agradeço também aos meus amigos de longa data Thiago Garcia, Lucas Pedroso, Felipe Negrão, Felipe Durão, Felipe Roque Rossi, Luigi,

*"The most important step a man can take. It's not the first one, is it? It's the next one.
Always the next step, Dalinar."*

— Brandon Sanderson, *Oathbringer*

RESUMO

O *phishing* permanece entre os vetores de ataque mais explorados em incidentes de segurança da informação, e o uso ofensivo de modelos de linguagem de larga escala (*Large Language Models*, LLMs) tornou as mensagens fraudulentas mais persuasivas, ao eliminar os erros que antes denunciavam a fraude. A mesma tecnologia, porém, apresenta-se como contramedida, pois os LLMs interpretam a intenção e o contexto do texto e passaram a ser avaliados como classificadores de *e-mails* suspeitos. A adoção desses modelos impõe, contudo, uma escolha de implantação: modelos comerciais de grande porte, acessados via API, oferecem alta capacidade de detecção, mas exigem o envio do conteúdo dos *e-mails* a servidores de terceiros, o que expõe dados pessoais e sigilosos e levanta riscos de conformidade frente à LGPD e ao GDPR, enquanto modelos abertos e leves, executados localmente, mantêm os dados sob o controle da organização, ao custo de menor capacidade de processamento. Este trabalho realiza uma análise comparativa entre quatro modelos leves executados localmente (Gemma 3 QAT 4B, Qwen3-4B, Granite 4.0 Tiny e Llama 3.2 3B) e um modelo comercial de referência (Claude Haiku 4.5) na classificação binária de *e-mails* de *phishing*. Para isso, construiu-se um *dataset* balanceado com mensagens do *Nazario Phishing Corpus* e do *Enron Corpus*, submetidas aos cinco modelos sob um protocolo uniforme de inferência, com *prompt zero-shot* único, decodificação determinística e saída em JSON estruturado. O desempenho foi medido por métricas de classificação binária (precisão, revocação, *F1-Score*, acurácia, MCC e AUC), complementadas pela latência e pelo custo de inferência, e as diferenças entre o modelo de referência e cada modelo local foram verificadas pelo teste de McNemar e por *bootstrap* pareado. Os resultados indicam que o Claude Haiku 4.5 obteve o melhor desempenho e que as diferenças frente aos modelos locais são estatisticamente significativas. Entre os locais, o Llama 3.2 3B apresentou o melhor resultado, seguido de perto pelo Qwen3-4B, enquanto o Gemma 3 QAT 4B e o Granite 4.0 Tiny ficaram mais distantes da referência, em razão do alto volume de falsos positivos. Conclui-se que os modelos locais não igualam o modelo comercial, mas os melhores entre eles se aproximam o suficiente para constituir uma alternativa viável quando a privacidade dos dados é prioritária.

Palavras-chave: Segurança da Informação. Modelos de Linguagem de Larga Escala. Detecção de *Phishing*. Privacidade de Dados. Classificação de *E-mails*.

ABSTRACT

Phishing remains among the most exploited attack vectors in information security incidents, and the offensive use of Large Language Models (LLMs) has made fraudulent messages more persuasive by removing the errors that once exposed the fraud. The same technology, however, also serves as a countermeasure, since LLMs interpret the intention and context of text and have been evaluated as classifiers of suspicious e-mails. Adopting these models imposes a deployment choice: large commercial models, accessed through APIs, offer high detection capability but require sending e-mail contents to third-party servers, which exposes personal and confidential data and raises compliance risks under the LGPD and the GDPR, whereas open lightweight models, executed locally, keep data under the organization's control, at the cost of lower processing capability. This work presents a comparative analysis between four lightweight models executed locally (Gemma 3 QAT 4B, Qwen3-4B, Granite 4.0 Tiny, and Llama 3.2 3B) and a commercial reference model (Claude Haiku 4.5) in the binary classification of phishing e-mails. To this end, a balanced dataset was built with messages from the Nazario Phishing Corpus and the Enron Corpus, submitted to the five models under a uniform inference protocol, with a single zero-shot prompt, deterministic decoding, and structured JSON output. Performance was measured with binary classification metrics (precision, recall, F1-Score, accuracy, MCC, and AUC), complemented by latency and inference cost, and the differences between the reference model and each local model were assessed with McNemar's test and paired bootstrap. The results indicate that Claude Haiku 4.5 achieved the best performance and that its differences from the local models are statistically significant. Among the local models, Llama 3.2 3B achieved the best result, closely followed by Qwen3-4B, whereas Gemma 3 QAT 4B and Granite 4.0 Tiny remained more distant from the reference due to their high volume of false positives. It is concluded that local models do not match the commercial model, but the best among them come close enough to constitute a viable alternative when data privacy is a priority.

Keywords: Information Security. Large Language Models. Phishing Detection. Data Privacy. E-mail Classification.

LISTA DE ILUSTRAÇÕES

Figura 1 – Curva ROC.	40
Figura 2 – Diagrama PRISMA da seleção dos trabalhos correlatos.	45
Figura 3 – Visão geral da arquitetura da solução.	54
Figura 4 – Curva ROC do Claude Haiku 4.5.	80
Figura 5 – Matriz de confusão do Claude Haiku 4.5.	81
Figura 6 – Distribuição do <i>phishing_likelihood</i> por e-mail do Claude Haiku 4.5.	82
Figura 7 – Matriz de confusão do Gemma 3 QAT 4B.	83
Figura 8 – Distribuição do <i>phishing_likelihood</i> por e-mail do Gemma 3 QAT 4B.	84
Figura 9 – Curva ROC do Gemma 3 QAT 4B.	85
Figura 10 – Curva ROC do Qwen3-4B.	86
Figura 11 – Distribuição do <i>phishing_likelihood</i> por e-mail do Qwen3-4B.	87
Figura 12 – Matriz de confusão do Qwen3-4B.	87
Figura 13 – Matriz de confusão do Granite 4.0 Tiny.	88
Figura 14 – Distribuição do <i>phishing_likelihood</i> por e-mail do Granite 4.0 Tiny.	89
Figura 15 – Curva ROC do Granite 4.0 Tiny.	90
Figura 16 – Curva ROC do Llama 3.2 3B.	92
Figura 17 – Distribuição do <i>phishing_likelihood</i> por e-mail do Llama 3.2 3B.	93
Figura 18 – Matriz de confusão do Llama 3.2 3B.	94
Figura 19 – <i>F1-Score</i> da classe <i>phishing</i> por modelo nas três regras de decisão.	95
Figura 20 – Curvas ROC dos modelos comparados.	96
Figura 21 – Pares discordantes do teste de McNemar entre o Claude Haiku 4.5 e cada modelo local (dataset de 2000 elementos).	98
Figura 22 – <i>Bootstrap</i> pareado das diferenças (modelo local menos Claude Haiku 4.5) em <i>F1-Score</i> , MCC e AUC, com IC de 95%.	99

LISTA DE TABELAS

Tabela 1 – E-mails preservados por inteiro e truncados, por limite de caracteres.	65
Tabela 2 – E-mails truncados (acima de 5000 caracteres) por classe.	66
Tabela 3 – Métricas de classificação por modelo (com <i>limiar</i> ≥ 50 fixo).	78
Tabela 4 – E-mails com erro de formato irreparável no Gemma 3 QAT 4B.	85
Tabela 5 – E-mails com erro de formato irreparável no Granite 4.0 Tiny.	91
Tabela 6 – Distribuição dos valores de <i>phishing_likelihood</i> emitidos pelo Llama 3.2 3B.	93
Tabela 7 – E-mails com erro de formato irreparável no Llama 3.2 3B.	94
Tabela 8 – Métricas de classificação por modelo no ponto de corte ótimo (<i>F1-Score</i> máximo).	97
Tabela 9 – Validação do ponto de corte ótimo por partição 70/30: <i>F1-Score</i> (%) com o limiar estimado apenas na partição de estimativa.	97
Tabela 10 – Métricas operacionais por modelo.	100

LISTA DE QUADROS

1	Matriz de confusão.	36
2	Tabela de contingência para o teste de McNemar.	41
3	Exemplo de e-mail rotulado como <i>phishing</i> na base, mas que constitui <i>spam</i>	63
4	Exemplos de e-mails descartados pelo filtro de qualidade semântica. .	67
5	Configuração do ambiente experimental.	71
6	Modelos de linguagem avaliados no experimento.	72
7	<i>Prompt</i> -base <i>zero-shot</i> utilizado na inferência.	74
8	Campos de um registro e exemplo de entrada do <i>dataset</i>	75
9	Triagem dos estudos recuperados na base Scopus.	116
10	<i>Digests</i> SHA256 dos artefatos dos modelos locais.	132

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
AUC	Area Under the Curve
BEC	Business Email Compromise
CIA	Confidentiality, Integrity, Availability
CPU	Central Processing Unit
FNR	False Negative Rate
FPR	False Positive Rate
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
IA	Inteligência Artificial
IDS	Intrusion Detection System
JSON	JavaScript Object Notation
LGPD	Lei Geral de Proteção de Dados Pessoais
LLM	Large Language Model
MCC	Matthews Correlation Coefficient
MoE	Mixture of Experts
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
QAT	Quantization-Aware Training
RAM	Random Access Memory
ROC	Receiver Operating Characteristic
SIEM	Security Information and Event Management
SMS	Short Message Service
SOAR	Security Orchestration, Automation and Response

TCC	Trabalho de Conclusão de Curso
TPR	True Positive Rate
UFSC	Universidade Federal de Santa Catarina
URL	Uniform Resource Locator
VRAM	Video Random Access Memory

SUMÁRIO

1	INTRODUÇÃO	17
1.1	Contextualização	17
1.2	Problema	18
1.3	Justificativa	18
1.4	Objetivos	19
1.4.1	Objetivo Geral	19
1.4.2	Objetivos Específicos	20
1.5	Metodologia de Pesquisa	20
1.6	Escopo e Limitações	21
1.7	Organização do Documento	22
2	FUNDAMENTAÇÃO TEÓRICA	24
2.1	Phishing	24
2.2	Modelos de Linguagem de Larga Escala	29
2.2.1	Técnicas de <i>Prompting</i>	30
2.2.2	Parâmetros e requisitos de <i>hardware</i>	31
2.3	Problemas Resultantes de Utilização de LLMs	32
2.3.1	Alucinações	32
2.3.2	Ataques centrados em IAs	33
2.3.3	Contaminação dos dados de teste	33
2.3.4	Privacidade e o risco do uso de APIs externas	34
2.3.5	Implicações para a Escolha entre Modelos Locais e Externos	35
2.4	Métricas de Avaliação de Classificadores	35
2.4.1	O <i>trade-off</i> entre falsos positivos e falsos negativos no contexto de segurança	38
2.4.2	Curva ROC e AUC	39
2.4.3	Comparação estatística entre classificadores	40
2.4.4	Latência e o percentil 95	42
3	REVISÃO DO ESTADO DA ARTE	43
3.1	Protocolo de Revisão Sistemática	43
3.2	Estratégias de Busca e Filtros de Seleção	43
3.3	Análise Crítica dos Trabalhos Correlatos	46
3.3.1	<i>Benchmarking and Evaluating Large Language Models in Phishing Detection for Small and Midsize Enterprises: A Comprehensive Analysis</i>	46

3.3.2	<i>Can AI Keep You Safe? A Study of Large Language Models for Phishing Detection</i>	47
3.3.3	<i>Devising and Detecting Phishing Emails Using Large Language Models</i>	48
3.4	Análise Comparativa dos Trabalhos	50
4	CONSTRUÇÃO DA ABORDAGEM	52
4.1	Caracterização da Pesquisa	52
4.2	Arquitetura da Solução	53
4.3	Tecnologias Empregadas	55
4.3.1	Linguagem de Programação	55
4.3.2	Containerização e Execução Local	55
4.3.3	Biblioteca de Acesso aos Modelos	56
4.4	Funcionamento da Aplicação	56
5	METODOLOGIA EXPERIMENTAL	60
5.1	Variáveis do Experimento	60
5.2	Etapas Metodológicas	62
5.2.1	Escolha e Preparo do <i>Dataset</i>	62
5.2.1.1	Fonte e Composição	62
5.2.1.2	Inclusão e Pré-processamento	63
5.2.1.3	Amostragem e Balanceamento	67
5.2.2	Seleção e Configuração dos Modelos de LLMs	68
5.2.2.1	Critérios de Seleção	68
5.2.2.2	Modelos Seleccionados	68
5.2.2.3	Restrição de <i>Hardware</i>	70
5.2.2.4	Ambiente de Execução	70
5.2.2.5	Configuração de Inferência	72
5.2.3	Protocolo de Prompting	72
5.2.4	Execução dos Experimentos e Análise	75
5.2.4.1	Protocolo de Execução	75
5.2.4.2	Plano de Análise	76
6	ANÁLISE DOS RESULTADOS	78
6.1	Resultados de Classificação	78
6.2	Análise por Modelo	79
6.2.1	Claude Haiku 4.5	79
6.2.2	Gemma 3 QAT 4B	82
6.2.3	Qwen3-4B	86
6.2.4	Granite 4.0 Tiny	88
6.2.5	Llama 3.2 3B	91
6.3	Análise Comparativa entre os Modelos	95

6.3.1	Comparação Direta dos Modelos Contra o Claude Haiku 4.5	96
6.4	Métricas Operacionais	99
6.5	Discussão	101
7	CONSIDERAÇÕES FINAIS	104
7.1	Retomada dos Objetivos	104
7.2	Síntese dos Principais Resultados	105
7.3	Contribuições	106
7.4	Limitações	106
7.5	Sugestões para Trabalhos Futuros	106
	REFERÊNCIAS	108

APÊNDICES 115

APÊNDICE A – PROTOCOLO DE REVISÃO SISTEMÁTICA 116

APÊNDICE B – CONFIGURAÇÕES DO AMBIENTE EXPERIMENTAL132

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

O *phishing* permanece entre os vetores de ataque mais explorados em incidentes de segurança da informação (BAKI; VERMA, 2021). Trata-se de uma forma de engenharia social na qual o atacante se passa por uma entidade confiável e induz a vítima a revelar credenciais, autorizar transferências ou executar programas maliciosos (SAKA et al., 2024). O e-mail constitui o principal canal dessas campanhas, pois alcança um grande número de alvos a baixo custo e explora a confiança que o usuário atribui à própria caixa de entrada (BAKI; VERMA, 2021; JAIN et al., 2025). O volume de ataques cresce ano após ano, e estimativas recentes apontam bilhões de mensagens fraudulentas em circulação, com o *phishing* entre as ameaças mais reportadas no período (GRIFFITHS, 2025). O impacto financeiro acompanha essa escala, pois o *phishing* figura de forma recorrente entre os vetores iniciais mais frequentes e mais caros de violações de dados, com custo médio por incidente na ordem de milhões de dólares (IBM Security; Ponemon Institute, 2025). Adicionalmente, o usuário permanece suscetível mesmo quando treinado, já que a decisão de responder a um e-mail depende de fatores contextuais e cognitivos difíceis de neutralizar (JAYATILAKA; ARACHCHILAGE; BABAR, 2024).

Um fator recente agravou esse cenário, que é o uso ofensivo de modelos de linguagem de larga escala (*Large Language Models*, LLMs) pelos próprios atacantes (SIVANESWARAN et al., 2026). A disponibilização massiva desses modelos reduziu as barreiras técnicas e linguísticas para produzir mensagens persuasivas, personalizadas e gramaticalmente corretas (SIVANESWARAN et al., 2026). Dessa forma, os sinais clássicos que denunciavam uma fraude, como erros de ortografia, construções truncadas e formatações suspeitas, deixam de ser indicadores confiáveis (EZE; SHAMIR, 2024). Estudos demonstram que e-mails de *phishing* gerados por LLMs alcançam taxas de sucesso comparáveis às de mensagens redigidas por especialistas humanos, a uma fração do tempo e do custo (HEIDING et al., 2024). Ademais, a literatura recente já trata a geração assistida por IA como tendência consolidada, capaz de escalar campanhas direcionadas (*spear phishing*) que antes exigiam trabalho manual intensivo (SIVANESWARAN et al., 2026).

Todavia, a mesma tecnologia que potencializa os ataques também se apresenta como contramedida. Já que os LLMs interpretam intenção, contexto e pragmática do texto, passaram a ser avaliados como classificadores de e-mails suspeitos, com resultados promissores em relação a filtros baseados em regras (CHATAUT; GYAWALI; USMAN, 2024; ZHANG et al., 2025). A adoção de um LLM como classificador, po-

rém, impõe uma escolha de implantação que está no centro deste trabalho. De um lado situam-se os modelos comerciais de grande porte, acessíveis via API e de alta capacidade, porém com custo recorrente por uso e uma implicação sensível, pois submeter o conteúdo do e-mail a um servidor de terceiros expõe dados que podem ser pessoais ou sigilosos (SHANMUGARASA et al., 2025). De outro situam-se os modelos abertos e leves, com poucos bilhões de parâmetros, executáveis em hardware acessível e sob infraestrutura controlada, que mantêm os dados dentro do perímetro da organização (NASUTION et al., 2025), mas que não dispõem da mesma capacidade de processamento dos grandes modelos comerciais. Essa dualidade entre capacidade e soberania dos dados ainda carece de comparação empírica sistemática sob um protocolo uniforme, lacuna que este trabalho propõe abordar.

1.2 PROBLEMA

A literatura sobre o uso de LLMs na detecção de *phishing* cresce, e os resultados iniciais são promissores (CHATAUT; GYAWALI; USMAN, 2024; ZHANG et al., 2025; HEIDING et al., 2024). Todavia, esses estudos comparam principalmente modelos comerciais de grande porte, sem considerar que o envio de dados sensíveis a servidores externos levanta problemas de privacidade, conforme discutido na Seção 3.3. A literatura carece de uma análise comparativa entre modelos comerciais de grande porte, acessados por API externa, e modelos locais, em especial os modelos leves que executam em *hardware* acessível para esta tarefa. Sem essa comparação, a decisão sobre qual modelo implantar permanece sem base concreta, pois envolve um *trade-off* de várias dimensões, a saber, capacidade de detecção, custo de inferência, latência, requisitos de *hardware* e, no centro deste trabalho, a exposição de dados sensíveis a terceiros. Equipes de segurança que precisam escolher entre privacidade e capacidade não dispõem de dados consolidados para fundamentar a escolha. Diante disso, formula-se a pergunta que orienta esta pesquisa: modelos de linguagem leves, executados localmente, são capazes de oferecer desempenho competitivo frente a um modelo comercial de referência na classificação binária de e-mails de *phishing*?

1.3 JUSTIFICATIVA

A primeira justificativa é de segurança. Conforme exposto na contextualização, os modelos de linguagem conferem aos atacantes a capacidade de produzir mensagens persuasivas e isentas dos erros que antes denunciavam a fraude (SIVANESWARAN et al., 2026; EZE; SHAMIR, 2024). Filtros heurísticos e baseados em regras, que dependem desses sinais superficiais, perdem eficácia diante de e-mails redigidos por LLMs (HEIDING et al., 2024). A defesa precisa, portanto, atuar na mesma camada

que o ataque passou a explorar, a do significado (semântica) do texto. Nesse ponto, os LLMs se apresentam como classificadores defensivos viáveis, pois interpretam a intenção, o contexto e a pragmática da mensagem, e não apenas a sua forma (CHATAUT; GYAWALI; USMAN, 2024; ZHANG et al., 2025).

A segunda justificativa, eixo central deste trabalho, é a privacidade. Submeter o conteúdo integral de um e-mail a uma API de terceiros constitui, por si só, uma exposição de dados, mesmo quando a mensagem analisada é legítima. E-mails podem conter dados pessoais, credenciais, identificadores corporativos e informações sigilosas, e seu envio para fora do perímetro da organização transfere o controle sobre esse material a um provedor externo. A literatura indica que o risco não é hipotético, pois modelos de linguagem podem memorizar e reproduzir trechos dos dados a que foram expostos (CARLINI et al., 2021; CARLINI et al., 2022), reter conteúdo em ciclos de treinamento e inferir atributos sensíveis a partir de fragmentos aparentemente inócuos (SHANMUGARASA et al., 2025; DUAN et al., 2024; ROCHER; HENDRICKX; MONTJOYE, 2019). No plano regulatório, tanto a LGPD no Brasil quanto o GDPR na Europa impõem restrições ao tratamento de dados pessoais por agentes externos e à transferência internacional de dados, o que torna a exposição a APIs comerciais um vetor de risco legal e de conformidade (BRASIL, 2018; European Parliament; Council of the European Union, 2016). Em contrapartida, modelos executados localmente em infraestrutura controlada mantêm os dados dentro do perímetro da organização e eliminam esse vetor de exposição (NASUTION et al., 2025), o que viabiliza uma abordagem mais segura e privada.

A terceira justificativa é prática e econômica. Modelos com poucos bilhões de parâmetros executam em GPUs de consumo, ou mesmo em CPUs, sem custo recorrente por token (NGUYEN et al., 2025), o que viabiliza a adoção por organizações de pequeno e médio porte, para as quais o orçamento de uso intensivo de APIs comerciais pode ser proibitivo. Caso a comparação demonstre que esses modelos alcançam desempenho próximo ao de um modelo comercial de referência, configura-se uma alternativa concreta de implantação que combina baixo custo e soberania dos dados. Caso revele uma diferença significativa de eficácia, o resultado é igualmente relevante, pois permite às equipes de segurança avaliarem o *trade-off* entre desempenho e privacidade com base em evidência empírica concreta.

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Comparar modelos de linguagem leves executados localmente (Gemma 3 QAT, Qwen3, Granite 4.0 Tiny e Llama 3.2) com um modelo comercial de grande porte (Claude) na detecção de *e-mails* de *phishing*, avaliando a eficácia relativa de cada

modelo com métricas estatísticas de classificação binária e considerando, de forma complementar, o custo de inferência, a latência e as implicações de privacidade.

1.4.2 Objetivos Específicos

- a) Revisar sistematicamente a literatura sobre modelos de linguagem de larga escala na detecção de *phishing*;
- b) Identificar e analisar o principal vetor de ataque abordado (*phishing*) e suas características detectáveis;
- c) Construir um *dataset* balanceado de *e-mails* de *phishing* e *e-mails* reais;
- d) Selecionar e justificar os modelos de IA mais adequados para a análise comparativa, com embasamento da literatura, contemplando ao menos um modelo comercial e ao menos dois modelos locais;
- e) Desenvolver uma *pipeline* de testes funcional e escalável para realizar a inferência dos modelos selecionados sobre o *dataset* construído, garantindo um protocolo uniforme de avaliação;
 - a) Executar os testes de inferência, coletando sistematicamente as saídas dos modelos e a latência para cada amostra;
- f) Avaliar os testes com métricas estatísticas de classificação binária, como precisão, *recall* e *F1-Score*, além de taxas de falsos positivos e falsos negativos, latência e custo de *tokens* de inferência;
- g) Comparar os resultados entre os modelos, discutindo as implicações de desempenho, custo e privacidade para a escolha de um classificador de *phishing* baseado em LLMs locais.

1.5 METODOLOGIA DE PESQUISA

Esta é uma pesquisa aplicada, de abordagem quantitativa e procedimento experimental (PRODANOV; FREITAS, 2013). Quanto aos objetivos, é descritiva e comparativa, pois caracteriza o desempenho de cada modelo e os contrasta entre si. A unidade de análise é o par (modelo, *e-mail*), e a variável dependente é a classificação binária produzida (*phishing* ou legítimo), confrontada com o rótulo de referência (*ground truth*) do *dataset*. A variável independente, manipulada de forma controlada, é o modelo de linguagem, em cinco níveis. Os demais fatores, do *prompt* aos parâmetros de decodificação, são mantidos fixos, de modo que a única diferença entre as condições testadas

seja o modelo avaliado. A caracterização completa está no Capítulo 4 e o desenho das variáveis, no Capítulo 5.

O trabalho parte de uma revisão sistemática da literatura sobre LLMs na detecção de *phishing* e sobre a avaliação de classificadores binários (Capítulo 3), que fundamenta as escolhas metodológicas. Sobre essa base, o experimento avança em quatro etapas, detalhadas no Capítulo 5. A primeira constrói o *dataset* a partir de fontes públicas reconhecidas, com o *phishing* extraído do *Nazario Phishing Corpus* (NAZARIO, 2005) e os *e-mails* legítimos do *Enron Corpus* (KLIMT; YANG, 2004), sob balanceamento entre as classes, filtro de qualidade e remoção de duplicatas. A segunda seleciona e configura os modelos avaliados: quatro abertos, executados localmente em ambiente único e controlado, e um comercial, acessado via *API* sob as mesmas condições. A terceira padroniza o protocolo de inferência, com um *prompt* único em estratégia *zero-shot*, parâmetros de decodificação fixos (temperatura 0 e top-p 1) e saída em JSON estruturado, o que viabiliza uma comparação justa entre os modelos (CHATAUT; GYAWALI; USMAN, 2024; ZHANG et al., 2025; HEIDING et al., 2024). A quarta submete os *e-mails* aos modelos e registra as métricas: as de classificação (precisão, revocação, F1-Score, acurácia, matriz de confusão, taxas de falsos positivos e falsos negativos, coeficiente de correlação de *Matthews* e AUC) e as operacionais (latência, contagem de *tokens* e custo por inferência). Para sustentar as afirmações comparativas, a diferença de desempenho entre o modelo de referência e cada modelo local é submetida ao teste de McNemar (DIETTERICH, 1998), que separa diferença real de variação amostral, e a um *bootstrap* pareado (EFRON; TIBSHIRANI, 1993), que estima intervalos de confiança para a magnitude dessas diferenças.

1.6 ESCOPO E LIMITAÇÕES

Este trabalho limita-se à classificação binária de *e-mails*, que separa cada mensagem entre *phishing* e legítima. As mensagens estão em inglês, idioma que reflete a composição do *dataset* adotado (Capítulo 5). A análise considera apenas o conteúdo textual: corpo, assunto, cabeçalhos textuais e remetente. Ficam fora do recorte:

- a) a análise de anexos maliciosos;
- b) a verificação ativa de URLs em serviços de reputação ou *sandboxes*;
- c) o *phishing* em outras modalidades, como SMS (*smishing*), voz (*vishing*) e QR-code (*quishing*);
- d) a integração dos modelos em plataformas de produção, como SIEM, SOAR e *gateways* de e-mail;

- e) o *fine-tuning* dos modelos avaliados, todos empregados em sua forma *off-the-shelf*, sem ajuste para a tarefa;
- f) a resiliência a *prompts* adversariais e a ataques voltados a LLMs, como *prompt injection*;
- g) a análise de explicabilidade das decisões dos modelos.

Quanto às limitações, a generalização dos resultados está condicionada à composição e à atualidade do *dataset*, pois campanhas de *phishing* posteriores à coleta podem trazer características fora da distribuição dos dados de teste. Há ainda um descompasso temporal entre as fontes das duas classes: os *e-mails* legítimos do *Enron Corpus* antecedem em cerca de duas décadas as campanhas de *phishing* do *Nazario*, de modo que pistas de época e de estilo podem auxiliar a separação das classes independentemente dos indicadores de fraude (NAZARIO, 2005; KLIMT; YANG, 2004; SOOD, 2024). Como esse viés incide igualmente sobre os cinco modelos, a comparação relativa entre eles permanece válida, embora as métricas absolutas devam ser lidas como potencialmente otimistas. Soma-se a isso o risco de contaminação discutido na Seção 2.3.3: parte dos *e-mails* pode ter integrado o treinamento dos modelos, principalmente do Claude. Esse efeito, porém, atua contra a hipótese do estudo, pois favorece o próprio modelo de referência e torna a comparação mais conservadora para os modelos locais. O próprio Claude é uma caixa-preta, cuja versão exata e comportamento podem mudar entre execuções e atualizações do provedor, o que limita a reprodutibilidade estrita; seu custo de inferência, por sua vez, é estimado com base nos preços vigentes na data dos experimentos. Por fim, os resultados são atrelados aos modelos utilizados, de forma que modelos diferentes podem apresentar resultados distintos, e a comparação aqui realizada não é necessariamente extrapolável para outros modelos comerciais ou locais.

No plano operacional, os modelos locais são avaliados em uma única configuração de *hardware*, e sua latência pode variar bastante em outros ambientes. Os resultados também refletem o protocolo *zero-shot* adotado: a escolha do *prompt* influencia o desempenho, e os números obtidos não representam necessariamente o teto de capacidade de cada modelo. Por fim, mesmo um bom resultado não credencia esses modelos como a melhor escolha para produção, onde ficariam expostos a ataques adversariais cuja resiliência não é analisada aqui.

1.7 ORGANIZAÇÃO DO DOCUMENTO

O restante deste trabalho está organizado em seis capítulos. O Capítulo 2 apresenta a fundamentação teórica, com os conceitos de segurança da informação, *phishing*, métricas de classificação binária e modelos de linguagem de larga escala, incluindo

seus riscos de privacidade. O Capítulo 3 traz a revisão do estado da arte, descrevendo o protocolo de busca e os trabalhos correlatos selecionados sobre detecção de *phishing* com modelos de linguagem. O Capítulo 4 descreve a construção da abordagem, abrangendo a caracterização da pesquisa, a arquitetura da solução, as tecnologias empregadas e o funcionamento da aplicação de avaliação. O Capítulo 5 detalha a metodologia experimental, com as variáveis do experimento e as etapas metodológicas que vão da preparação do *dataset* à coleta das métricas.

O Capítulo 6 apresenta e discute os resultados, partindo das métricas de classificação das cinco condições, passando à análise individual de cada modelo, com sua curva ROC e seu padrão de erro característico, e ao confronto comparativo apoiado no teste de McNemar e no *bootstrap* pareado, até as métricas operacionais de latência e custo e a discussão dos achados à luz da questão de pesquisa.

Por fim, o Capítulo 7 retoma a questão de pesquisa à luz dos resultados, sintetiza os principais achados e registra as contribuições, as limitações e os trabalhos futuros do estudo.

2 FUNDAMENTAÇÃO TEÓRICA

A segurança da informação protege sistemas computacionais, redes e dados contra ameaças cibernéticas e tornou-se indispensável à medida que praticamente todos os setores da sociedade passaram a depender de tecnologias digitais (ADMASS; MUNAYE; DIRO, 2024; BERTINO, 2016). Historicamente, a área organiza-se em torno de três propriedades, a confidencialidade, a integridade e a disponibilidade (CIA, do inglês *Confidentiality, Integrity, Availability*) (ADMASS; MUNAYE; DIRO, 2024; BERTINO, 2016). A esse conjunto somou-se a privacidade como requisito crítico (BERTINO, 2016). Embora seja com frequência confundida com a confidencialidade, a privacidade possui dimensões próprias, jurídicas e individuais, como o cumprimento da Lei Geral de Proteção de Dados (LGPD) (BRASIL, 2018) e o respeito às preferências dos titulares dos dados (BERTINO, 2016). Garantir essas propriedades tornou-se mais difícil com a sofisticação dos ataques e a ampliação das superfícies de ataque, impulsionada pela coleta massiva e pelo compartilhamento generalizado de dados (BERTINO, 2016).

Ainda assim, o objetivo da segurança nunca foi a proteção absoluta, reconhecidamente inviável, e sim o estabelecimento de salvaguardas com boa relação custo-benefício, robustas o suficiente para que o *hardware* e o *software* não constituam os pontos mais frágeis do sistema (DENNING; DENNING, 1979). A criptografia moderna ilustra esse princípio, pois, embora um ataque de força bruta, no qual o atacante testa sucessivas combinações até acertar a chave, seja teoricamente possível (SHARAFALDIN; Habibi Lashkari; GHORBANI, 2018), o tempo necessário para percorrer todas as combinações de uma chave bem dimensionada o torna inviável na prática (SCHNEIER, 1996). Portanto, a fragilidade raramente reside na tecnologia em si, mas no fator humano que a opera.

2.1 PHISHING

Phishing constitui uma das modalidades mais comuns e custosas de ataques de engenharia social no contexto da segurança da informação e caracteriza-se como uma técnica fraudulenta na qual um atacante personifica uma entidade confiável para induzir a vítima a revelar informações sensíveis, transferir recursos financeiros, executar arquivos maliciosos ou conceder acessos privilegiados a sistemas (BAKI; VERMA, 2021; EZE; SHAMIR, 2024). Embora o termo *phishing* tenha sido cunhado apenas em 1996 e o primeiro ataque registrado date de 1995 (BAKI; VERMA, 2021), o conceito subjacente é consideravelmente mais antigo. Eze e Shamir (2024) destacam que ataques análogos, baseados em enganação por cartas, foram documentados ainda na primeira metade do século XIX, como a denominada "*Letter from Jerusalem*", na qual o atacante prometia

à vítima uma parcela de uma grande quantia em dinheiro mediante o pagamento prévio de uma taxa pequena, supostamente destinada a subornar carcereiros (atualmente conhecido no Brasil como "golpe do príncipe nigeriano", ou "fraude 419"(GREE, 2026)). Esse padrão de explorar a confiança e a expectativa de recompensa da vítima persiste até os dias atuais, ainda que adaptado às plataformas digitais contemporâneas e amplificado em escala pelas tecnologias modernas de comunicação.

A persistência e a eficácia do *phishing* como vetor de ataque decorrem de uma característica que o distingue da maioria dos ataques cibernéticos convencionais: ele não necessita penetrar defesas técnicas como *firewalls*, sistemas de detecção de intrusão ou mecanismos criptográficos. O *phishing* mira diretamente o componente humano dos sistemas computacionais, amplamente reconhecido como o elo mais fraco na segurança de dados (BAKI; VERMA, 2021). Por mais robustas que sejam as proteções técnicas implementadas em uma organização, como controles de acesso, criptografia e segmentação de rede, basta que um único usuário seja persuadido a fornecer voluntariamente suas credenciais ou a executar um anexo malicioso para que toda a estrutura defensiva seja contornada. Esse fato torna o *phishing* atrativo para atacantes, já que se trata de um ataque de baixo custo operacional, alto retorno potencial e tecnicamente trivial em sua execução básica, ainda que possa ser arbitrariamente sofisticado em suas variantes mais elaboradas, como o *spear phishing*, modalidade direcionada na qual o atacante personaliza a mensagem para uma vítima específica após coleta prévia de informações sobre o alvo (BAKI; VERMA, 2021).

Os ataques de *phishing* podem ser classificados segundo o canal de entrega utilizado, o que dá origem a uma taxonomia que se expandiu à medida que novas plataformas de comunicação digital se popularizaram. Entre as principais modalidades, destacam-se:

- *Phishing por e-mail*: modalidade tradicional e mais prevalente, na qual a mensagem fraudulenta é entregue pelo correio eletrônico (BAKI; VERMA, 2021; JAIN et al., 2025);
- *Spear phishing*: variante direcionada na qual o atacante personaliza a mensagem para uma vítima específica ou grupo restrito, frequentemente após reconhecimento prévio do alvo em redes sociais profissionais (BAKI; VERMA, 2021; JAIN et al., 2025);
- *Whaling*: subcategoria do *spear phishing* dirigida a executivos de alto escalão ou figuras de destaque em uma organização, com o objetivo de maximizar o impacto da invasão bem-sucedida (JAIN et al., 2025);
- *Smishing*: variante baseada em mensagens SMS, que explora o fato de dispositivos móveis frequentemente truncarem URLs e dificultarem a inspeção visual de remetentes (JAIN et al., 2025);

- *Vishing*: variante baseada em chamadas de voz, na qual o atacante se passa por funcionário de instituições financeiras ou suporte técnico (JAIN et al., 2025);
- *Quishing*: variante mais recente, fundamentada na exploração de códigos *QR* fraudulentos que redirecionam a vítima a páginas maliciosas ao serem lidos por dispositivos móveis (JAIN et al., 2025).

Apesar dessa diversidade, o *phishing* por *e-mail* mantém-se como o canal mais relevante no cenário atual de ameaças, tanto em volume absoluto quanto em impacto financeiro. Segundo o relatório *Cost of a Data Breach 2025* da IBM Security (IBM Security; Ponemon Institute, 2025), estima-se que aproximadamente 3,4 bilhões de *e-mails* de *phishing* sejam enviados diariamente ao redor do mundo (GRIFFITHS, 2025) e que cada violação de dados associada a esse vetor custe, em média, cerca de 4,8 milhões de dólares à organização afetada. Esses números justificam o foco contínuo da literatura científica e da indústria de segurança no estudo e na mitigação dessa modalidade específica, à qual este trabalho se dedica. A preferência dos atacantes pelo *e-mail* decorre da combinação de vários fatores. Observa-se, em primeiro lugar, sua escalabilidade, pois o custo marginal de envio de uma mensagem adicional é praticamente nulo, o que permite que campanhas atinjam milhões de potenciais vítimas com investimento mínimo. Em segundo lugar, destaca-se sua presença no ambiente corporativo e pessoal, já que o *e-mail* permanece como ferramenta central de comunicação organizacional, integra-se cotidianamente aos fluxos de trabalho dos usuários e, portanto, é aberto e processado com baixo nível de atenção crítica. Em terceiro lugar, a relativa facilidade de manipulação dos cabeçalhos da mensagem, como nome exibido do remetente, endereço aparente e assunto, possibilita a personificação visual de entidades legítimas sem exigir conhecimento técnico avançado do atacante. Por fim, mesmo quando as mensagens contêm *hiperlinks* que apontam para domínios suspeitos, a representação textual do *link* pode ser arbitrariamente diferente de seu destino real, o que explora uma assimetria entre o que o usuário visualiza e o que efetivamente é acessado ao clicar (EZE; SHAMIR, 2024).

Quanto à sua estrutura interna, *e-mails* de *phishing* tendem a apresentar elementos recorrentes que, em conjunto, configuram o padrão canônico desse tipo de ataque. Eze e Shamir (2024), ao analisarem manualmente uma amostra representativa de *e-mails* fraudulentos, identificaram que essas mensagens articulam tipicamente quatro componentes complementares:

- Identidade falsificada do remetente: a mensagem aparenta originar-se de uma fonte confiável, como instituições financeiras, provedores de serviços digitais, departamentos internos da organização da vítima ou contatos pessoais previamente conhecidos;

- Senso de urgência: o texto comunica a necessidade de ação imediata, frequentemente associada a um prazo curto e específico (por exemplo, 24 ou 48 horas), com o objetivo de desencorajar a verificação cuidadosa do conteúdo pela vítima;
- Chamada explícita à ação: a mensagem solicita que o destinatário execute uma ação concreta e mensurável, como clicar em um *hiperlink*, baixar um anexo, responder com informações pessoais ou efetuar uma transferência financeira;
- Descrição de consequências negativas: caso a ação solicitada não seja executada no prazo estipulado, são apresentadas penalidades como suspensão de conta, perda de acesso a serviços, multas financeiras ou comprometimento de segurança.

Em estudo empírico sobre como usuários decidem responder a *e-mails*, Jayatilaka, Arachchilage e Babar (2024) observaram que os participantes associam mensagens curtas a tentativas de fraude e tratam *e-mails* mais longos como indício de legitimidade, na suposição de que um atacante não se daria ao trabalho de redigir um texto extenso. Convém, porém, distinguir o que conta como "longo" nesse contexto. A percepção de extensão registrada pelos participantes refere-se ao texto redigido para um leitor humano, cujo limite prático é a paciência de quem lê: mesmo um *e-mail* considerado longo por essa medida raramente ultrapassa alguns parágrafos. Esse patamar é muito inferior ao de mensagens verdadeiramente extensas, como notificações automatizadas, relatórios ou encadeamentos de respostas, que podem alcançar dezenas de milhares de caracteres. Como a mensagem de *phishing* é escrita para induzir uma ação rápida, ela privilegia textos objetivos e diretos, de modo que, ainda que ocasionalmente alongada para simular legitimidade, permanece dentro de uma faixa moderada de extensão e não atinge a ordem de grandeza dos *e-mails* genuinamente longos.

A categorização temática dos pretextos utilizados também revela padrões consistentes. Eze e Shamir (2024), ao analisarem qualitativamente uma amostra aleatória de 100 *e-mails* de *phishing*, observaram que aproximadamente 48% das mensagens solicitavam atualização ou verificação urgente de contas, 36% notificavam supostas atividades suspeitas detectadas em contas do usuário e parcelas menores tratavam de suspensão de contas, verificação de transações financeiras, atividades em contas estudantis ou convites para eventos. Essa concentração temática indica que, embora superficialmente diversos, os *e-mails* de *phishing* exploram um conjunto relativamente restrito de gatilhos psicológicos, predominantemente associados ao medo de perdas, como de acesso a serviços, de recursos financeiros ou de segurança pessoal, e à urgência percebida pela vítima.

A predominância desses gatilhos afetivos foi confirmada empiricamente em larga escala por Onwuegbuche et al. (2025), que anotaram mais de cinco mil *e-mails* de *phishing* dirigidos a universidades e identificaram seis estados afetivos recorrente-

mente explorados pelos atacantes: medo, urgência, ganância, curiosidade, confiança e compaixão. Os autores constataram que medo e ganância, praticamente ausentes nas mensagens legítimas analisadas, distinguem com clareza os dois tipos de comunicação, e que os *e-mails* de *phishing* elevam o nível de excitação emocional (*arousal*) da mensagem, sobretudo no corpo, combinando pares como medo-urgência e urgência-confiança para induzir respostas impulsivas, ao passo que comunicações legítimas mantêm tom predominantemente neutro. O estudo reforça, dessa forma, que a eficácia do *phishing* apoia-se menos em falhas técnicas e mais na manipulação das respostas emocionais da vítima.

Historicamente, a literatura sobre detecção automatizada de *phishing* baseou-se em heurísticas e características superficiais das mensagens, como presença de erros gramaticais e ortográficos, formatação inconsistente, uso de domínios visualmente próximos a marcas legítimas (técnicas como *typosquatting* e *homograph attacks*), correspondência com listas negras de URLs conhecidamente maliciosos e análise de cabeçalhos de autenticação (NASUTION et al., 2025). Tais abordagens, embora razoavelmente eficazes contra campanhas convencionais, partem do pressuposto implícito de que *e-mails* de *phishing* apresentam sinais textuais e estruturais detectáveis que os diferenciam de mensagens legítimas, pressuposto que vem sendo progressivamente erodido pela emergência de uma nova geração de ataques (EZE; SHAMIR, 2024; HEIDING et al., 2024).

A disponibilização ampla de Modelos de Linguagem de Larga Escala (*LLMs*) a partir do início da década inaugurou uma inflexão significativa no panorama do *phishing* por *e-mail*. Eze e Shamir (2024) argumentam que, enquanto campanhas tradicionais se caracterizavam pelo envio massivo de mensagens textualmente idênticas ou quase idênticas a um grande número de destinatários, característica que facilitava sua detecção por filtros baseados em similaridade entre mensagens recebidas em curto intervalo, modelos generativos permitem a produção automatizada de mensagens únicas para cada destinatário, todas gramaticalmente corretas, contextualmente plausíveis e estilisticamente convincentes. Esse fenômeno compromete simultaneamente dois pilares clássicos da detecção automatizada de *phishing*: a detecção por duplicação, já que cada mensagem é distinta, e a detecção por anomalias linguísticas, já que os *LLMs* produzem texto formalmente impecável e eliminam os erros que historicamente sinalizavam fraudes ao usuário atento.

Ademais, Eze e Shamir (2024) demonstram empiricamente, com análise descritiva sobre um *corpus* de *e-mails* de *phishing* gerados por IA, que tais mensagens apresentam características estilísticas mensuravelmente distintas daquelas redigidas manualmente, incluindo maior diversidade de palavras, frequência elevada de verbos e pronomes, expressão de sentimentos mais positivos e uso de palavras com maior comprimento médio. Embora essas diferenças permitam, em princípio, a discriminação entre *e-mails* de *phishing* gerados por humanos e por IA quando submetidos a

classificadores devidamente treinados, elas também evidenciam que a fronteira tradicional entre mensagens "obviamente suspeitas" e mensagens "convincentemente legítimas" tornou-se substancialmente mais tênue.

Nesse cenário, em que modelos generativos produzem e-mails de phishing cada vez mais persuasivos e individualizados, a análise semântica e contextual desses ataques passa a requerer ferramentas defensivas capazes de operar na mesma camada de raciocínio utilizada na geração ofensiva, camada que envolve compreensão de intenção comunicativa, contexto e coerência discursiva, e não apenas inspeção de características lexicais ou estruturais superficiais. Os LLMs configuram, nesse sentido, candidatos naturais a classificadores defensivos, dada sua capacidade de processar e interpretar texto natural com sofisticação comparável à demonstrada na geração ofensiva (NASUTION et al., 2025; EZE; SHAMIR, 2024). A seleção de um modelo para essa finalidade, contudo, não se reduz à sua capacidade de detecção, envolvendo também a privacidade dos dados analisados e o custo operacional de sua utilização em escala.

2.2 MODELOS DE LINGUAGEM DE LARGA ESCALA

Os Modelos de Linguagem de Larga Escala são sistemas de inteligência artificial capazes de interpretar e gerar texto em linguagem natural. Internamente, organizam-se como redes neurais profundas, modelos computacionais que aprendem a executar uma tarefa a partir de exemplos, ajustando valores internos chamados parâmetros, em vez de seguir regras escritas manualmente (WU; FENG, 2018). Muitos LLMs atuais apoiam-se na arquitetura *Transformer*, cujo principal recurso é o mecanismo de atenção: em lugar de processar o texto estritamente palavra após palavra, o modelo avalia ao mesmo tempo como cada parte da sequência se relaciona com as demais (VASWANI et al., 2017). Esse mecanismo permite captar dependências contextuais longas, isto é, relações entre termos distantes no texto, como um pronome e o nome a que ele se refere muitas palavras antes (VASWANI et al., 2017). O que distingue os LLMs modernos é a escala: modelos comerciais de grande porte reúnem de centenas de bilhões a, possivelmente, trilhões de parâmetros (GULYAMOV et al., 2026; SOUSA, 2025).

No nível mais básico, os LLMs resolvem a tarefa probabilística de modelagem de linguagem, que consiste em estimar a probabilidade conjunta de sequências de *tokens*, as unidades mínimas de texto em que palavras e fragmentos de palavras são divididos (XIAO; ZHU, 2025). Durante a inferência, o modelo processa o texto de entrada como uma sequência de *tokens* e gera a saída *token a token*, computando, a cada passo, uma distribuição de probabilidade sobre o vocabulário (GULYAMOV et al., 2026; VASWANI et al., 2017). Esse processo, aparentemente simples, de predição do próximo *token*,

quando repetido sobre grandes bases de treinamento (com páginas da web, livros, enciclopédias e repositórios de código), faz o modelo adquirir conhecimento amplo sobre linguagem natural, semântica, fatos do mundo e padrões de raciocínio (XIAO; ZHU, 2025; VASWANI et al., 2017). Dessa forma, capacidades como coerência textual, raciocínio e geração de respostas adequadas ao contexto surgem da identificação e da reprodução estatística de padrões aprendidos, e não de regras programadas de modo explícito (GULYAMOV et al., 2026; VASWANI et al., 2017).

Os LLMs transformaram diversas atividades graças à sua capacidade generalista, que dispensa um treino específico para cada tarefa e os torna adaptáveis a uma ampla variedade de usos (XIAO; ZHU, 2025). Algumas de suas aplicações práticas são, conforme Sivanewaran et al. (2026):

- Redação de diversos tipos de textos;
- Geração, *debugging* e *deploy* de código;
- *Chatbots* inteligentes que ultrapassam *workflows* definidos (se X então Y);
- Análise de grandes volumes de dados para fins de segurança, respostas e *insights* (SIVANESWARAN et al., 2026).

2.2.1 Técnicas de *Prompting*

Modelos de linguagem não executam tarefa alguma sem antes receber o *input* de um usuário (WHITE et al., 2023). Esse *input*, conhecido como *prompt*, define os objetivos que o LLM deve cumprir e pode conter ainda outras ordens e informações (WHITE et al., 2023). A elaboração de *prompts* especializados é chamada "engenharia de *prompt*", que organiza as melhores formas de comunicar a intenção ao LLM conforme o objetivo pretendido (WHITE et al., 2023). Entre as estratégias de *prompting*, destacam-se:

- ***Prompting Zero-shot***: o modelo recebe apenas a descrição da tarefa, sem exemplos prévios, e recorre ao conhecimento adquirido no pré-treinamento para gerar previsões para a nova tarefa (SOUSA, 2025).
- ***Prompting Few-shot***: ao *prompt* são acrescentados poucos exemplos de pares entrada-saída, que induzem o modelo a compreender o padrão da tarefa antes de gerar a resposta. Essa técnica obteve ganhos expressivos em tarefas complexas, com destaque para modelos amplos como o *GPT-3* ou *Claude*, embora exija atenção à seleção dos exemplos para reduzir vieses (SOUSA, 2025).
- ***Chain-of-Thought***: o modelo é instruído a explicitar etapas intermediárias de raciocínio antes da resposta final, o que favorece processos lógicos coerentes e encadeados (WEI et al., 2022). A técnica orienta-se a problemas de matemática

e de senso comum, com aumento de acurácia em testes de *benchmark* (WEI et al., 2022).

- **Expert Assignment / Persona Prompting:** o modelo é instruído a agir como uma função ou papel específico (SOUSA, 2025). Exemplo: "Você é um engenheiro de segurança de dados responsável por detecção de phishing [...]". Ainda que seja uma técnica de *prompting* bastante utilizada, Hu, Rostami e Thomason (2026) em experimentos recentes ainda não publicados indicaram que o *Expert Assignment* faz o *LLM* soar mais inteligente, porém piora o raciocínio real do modelo.

2.2.2 Parâmetros e requisitos de *hardware*

O número de parâmetros de um *LLM* corresponde à quantidade de pesos ajustáveis presentes em sua rede neural, valores numéricos otimizados durante o treinamento que codificam o conhecimento adquirido a partir dos dados (ZHAO et al., 2023). Na prática, cada parâmetro contribui de forma marginal para as representações internas do modelo, e a escala total influencia diretamente suas capacidades, como raciocínio, conhecimento factual e generalização para novas tarefas (WHITE et al., 2023). A evolução recente dos *LLMs* caracteriza-se por um crescimento próximo do exponencial no número de parâmetros: o *GPT-2* (2019) possuía cerca de 1,5 bilhão de parâmetros (OPENAI, 2019), ao passo que modelos contemporâneos como o *GPT-4*, *Claude* e o *Qwen3-235B-A22B* ultrapassam a casa das centenas de bilhões e podem atingir trilhões de parâmetros (GULYAMOV et al., 2026; SOUSA, 2025). Esse crescimento impõe requisitos elevados de *hardware*, em especial de memória de vídeo (*VRAM*) e de capacidade computacional de *GPUs*, o que torna o treinamento e a inferência dos maiores modelos uma tarefa custosa demais para usuários comuns (SHENG et al., 2023). Em resposta, técnicas como quantização (redução da precisão numérica dos pesos), destilação e arquiteturas *Mixture-of-Experts* (*MoE*), que ativam apenas uma fração dos parâmetros por inferência, popularizaram-se como estratégias para viabilizar modelos eficientes em *hardware* acessível (WAN et al., 2023).

Os modelos selecionados para este trabalho refletem essa diversidade de estratégias: *Claude (Anthropic)* é um modelo proprietário cuja contagem exata de parâmetros não é divulgada publicamente, prática comum entre fornecedores comerciais, e serve de *baseline* de comparação com os modelos locais (Anthropic, 2025). Não é um modelo local, pois está acessível somente por interface web ou por requisições de *API* (Anthropic, 2025); *Granite 4 Tiny* (IBM) possui 7 bilhões de parâmetros totais, dos quais apenas 1 bilhão é ativado por inferência, o que oferece o potencial de um modelo maior com consumo menor de recursos computacionais (SOULE; BERGMANN, 2025); *Gemma 3 QAT 4B* (Google) é um modelo denso de 4 bilhões de parâmetros otimizado por *Quantization-Aware Training* a 4 bits, o que reduz o consumo de *VRAM* de cerca de 8 GB (em precisão *BF16*) para aproximadamente 2,6 GB (YVINEC; CULLITON, 2025);

Qwen3-4B (Alibaba), modelo denso de 4 bilhões de parâmetros, ilustra a tendência de modelos compactos com desempenho competitivo frente a modelos bem maiores, já que a *Alibaba* o reporta como capaz de rivalizar com o *Qwen2.5-72B-Instruct*, um modelo com 18 vezes mais parâmetros, em diversos *benchmarks* (YANG et al., 2025; Qwen Team; Alibaba Cloud, 2025); *Llama 3.2 3B (Meta)* é um modelo denso de 3 bilhões de parâmetros, a menor variante de texto da família *Llama 3.2*, projetada pela *Meta* para execução em dispositivos de borda e com recursos limitados (GRATTAFIORI et al., 2024; Meta AI, 2024). Neste trabalho é executado quantizado a 4 bits (*Q4_0*), o que reduz ainda mais seu consumo de *VRAM*.

2.3 PROBLEMAS RESULTANTES DE UTILIZAÇÃO DE LLMS

Apesar dos avanços notáveis em geração e compreensão de linguagem natural, a adoção de LLMS em aplicações reais implica um conjunto de problemas que decorrem tanto da natureza de "caixa preta" desses sistemas quanto das configurações impostas por desenvolvedores e fornecedores de dados (SUN et al., 2024; BOMMASANI et al., 2021). Tais problemas adquirem maior relevância em contextos sensíveis à segurança, como a análise de *e-mails*, em que falhas de confiabilidade ou vazamento de informações podem comprometer a integridade da pessoa ou da organização que emprega a tecnologia (CARLINI et al., 2021; WEIDINGER et al., 2022). Entre as limitações discutidas na literatura e pertinentes a este estudo, destacam-se as alucinações (JI et al., 2023), os ataques de *prompt injection* (GRESHAKE et al., 2023), a contaminação dos dados de teste (GOLCHIN; SURDEANU, 2024) e, principalmente, os riscos à privacidade associados ao uso de APIs externas (YAO et al., 2024).

2.3.1 Alucinações

As alucinações referem-se à geração de conteúdo factualmente incorreto, fabricado ou inconsistente com o contexto fornecido ao modelo (JI et al., 2023). Esse fenômeno está diretamente relacionado à forma como os LLMS são treinados, já que parte significativa dos dados de pré-treinamento é coletada via web scraping e pode conter erros, informações imprecisas e conteúdo fabricado (JI et al., 2023; XIAO; ZHU, 2025). Como consequência, os modelos podem produzir respostas plausíveis na forma, porém incorretas no conteúdo, o que representa um risco crítico em domínios sensíveis (JI et al., 2023; SOUSA, 2025). Aplicado à detecção de *phishing*, esse comportamento implica classificações errôneas, tanto na forma de falsos positivos quanto de falsos negativos, o que justifica a necessidade de avaliação empírica rigorosa com métricas estatísticas de classificação (JI et al., 2023). Em consonância, Onwuegbuche et al.

(2025) registraram alucinações de autocontradição ao empregarem *LLMs* na análise de *e-mails* de *phishing*, com um mesmo modelo devolvendo classificações distintas para uma única mensagem conforme pequenas variações na formulação do *prompt*, o que reforça a importância de fixar a instrução e de validar as respostas obtidas.

2.3.2 Ataques centrados em IAs

Outro problema relevante envolve os ataques direcionados às próprias inteligências artificiais, como o *prompt injection* e o *jailbreak*, que exploram a forma como os *LLMs* interpretam instruções em linguagem natural para contornar mecanismos de segurança e alinhamento (GRESHAKE et al., 2023). Esses ataques empregam *prompts* maliciosos para induzir o modelo a gerar conteúdo restrito ou a revelar informações memorizadas durante o treinamento (SHANMUGARASA et al., 2025). Na análise de *e-mails*, e de qualquer *input* externo de forma geral, essa vulnerabilidade torna-se especialmente sensível: um e-mail malicioso pode conter, no próprio corpo, instruções projetadas para enganar o classificador, levando o modelo a ignorar indícios de fraude ou a emitir um veredito favorável ao atacante (GRESHAKE et al., 2023). Configura-se, portanto, um vetor de ataque que opera na mesma camada utilizada para a classificação (GRESHAKE et al., 2023; SHANMUGARASA et al., 2025).

2.3.3 Contaminação dos dados de teste

Um risco específico à avaliação de *LLMs* decorre da própria forma como esses modelos são pré-treinados, a partir de grandes volumes de dados coletados da web pública (GOLCHIN; SURDEANU, 2024; XIAO; ZHU, 2025). Muitos dos conjuntos usados para medir desempenho, incluindo os de detecção de *phishing*, também são públicos e estão hospedados em repositórios abertos (GOLCHIN; SURDEANU, 2024). Existe, portanto, a possibilidade de que parte do conjunto de teste já tenha sido vista pelo modelo durante o treinamento (GOLCHIN; SURDEANU, 2024). Quando isso ocorre, o modelo pode apenas reproduzir respostas memorizadas, em vez de generalizar a partir das características do *e-mail* (MAGAR; SCHWARTZ, 2022), o que infla artificialmente as métricas e compromete a validade da avaliação, como argumentam Sainz et al. (2023).

A gravidade desse efeito está ligada ao tamanho do modelo. A literatura indica que a capacidade de memorização cresce com o número de parâmetros, e modelos maiores tendem a reter e reproduzir trechos exatos dos dados de treinamento com mais frequência (CARLINI et al., 2022). Para os modelos leves avaliados neste trabalho, com poucos bilhões de parâmetros, esse risco é menor, embora ainda não negligenciável, pois sua capacidade de memorização é mais limitada. No caso do modelo comercial de

grande porte, adotado como *baseline*, a hipótese não pode ser descartada e poderia favorecê-lo por um motivo alheio à real capacidade de generalização.

2.3.4 Privacidade e o risco do uso de APIs externas

A questão mais crítica no contexto deste trabalho, contudo, refere-se aos riscos de privacidade introduzidos pelo uso de LLMs disponibilizados via APIs externas. Nesse sentido, Shanmugarasa et al. (2025) sistematiza esses riscos em quatro categorias principais:

- vulnerabilidades nos dados de treinamento;
- riscos associados aos *prompts* fornecidos pelos usuários;
- vazamentos nas saídas geradas pelos modelos;
- ameaças decorrentes da integração com agentes e ferramentas externas.

As três últimas categorias afetam diretamente a fase de uso do modelo e, portanto, são especialmente pertinentes a sistemas que processam dados sensíveis em tempo de inferência (GRESHAKE et al., 2023; SOUSA, 2025). Na análise de *e-mails* para detecção de *phishing*, cada mensagem submetida a um modelo externo é transmitida a um servidor de terceiros (YAO et al., 2024). Os *prompts* constituem, assim, uma fonte recorrente de vazamento de informações pessoais, pois tendem a conter dados como nomes, endereços, credenciais, identificadores corporativos e demais informações que, no contexto de um *e-mail*, são intrínsecas ao próprio conteúdo a ser analisado (SOUSA, 2025). Mesmo quando há cifragem de transporte e armazenamento, o provedor do serviço mantém acesso ao conteúdo em claro durante o processamento, o que compromete a confidencialidade dos dados (SHANMUGARASA et al., 2025; YAO et al., 2024) e pode violar legislações de privacidade digital como a *LGPD* (BRASIL, 2018) e a *GDPR* (European Parliament; Council of the European Union, 2016).

Ademais, políticas de retenção de muitos provedores permitem que os *prompts* enviados sejam incorporados a futuros ciclos de treinamento, o que amplia o risco de memorização e posterior reprodução de informações sensíveis em saídas geradas para outros usuários (CARLINI et al., 2021; CARLINI et al., 2022; DUAN et al., 2024). Há, ainda, um segundo vetor de risco igualmente relevante: a capacidade de inferência dos LLMs (SHANMUGARASA et al., 2025). Modelos contemporâneos conseguem inferir, com alta precisão, atributos pessoais a partir de dados aparentemente inofensivos, o que automatiza em larga escala um processo antes restrito a analistas humanos (SHANMUGARASA et al., 2025). Por consequência, ainda que o usuário aplique mecanismos de anonimização ou redação prévia, o conteúdo residual de um *e-mail* pode bastar para que o modelo reconstrua informações sensíveis sobre o remetente, o destinatário ou a

organização envolvida (SOUSA, 2025). Soma-se a esse cenário o fato de que, segundo Sousa (2025), as técnicas de mitigação aplicáveis a serviços de LLMs externos ainda são embrionárias e limitadas, justamente pela ausência de controle sobre o ambiente de execução do modelo.

2.3.5 Implicações para a Escolha entre Modelos Locais e Externos

O conjunto de problemas descritos evidencia que a escolha entre um LLM hospedado em ambiente local e um LLM acessado via API externa não constitui apenas uma decisão de natureza técnica ou econômica, mas envolve, fundamentalmente, considerações sobre privacidade e soberania dos dados. Modelos locais, ao executarem inteiramente em infraestrutura controlada pela organização, eliminam a necessidade de transmitir o conteúdo dos *e-mails* analisados a terceiros, o que mitiga os principais vetores de vazamento descritos por Shanmugarasa et al. (2025) e Sousa (2025). Tal característica alinha-se a regulamentações como a *LGPD* (BRASIL, 2018), no contexto brasileiro, e o *GDPR* (European Parliament; Council of the European Union, 2016), no contexto europeu, que impõem restrições rigorosas ao tratamento de dados pessoais por agentes externos.

2.4 MÉTRICAS DE AVALIAÇÃO DE CLASSIFICADORES

A detecção de *phishing* por *e-mail* configura, em sua forma mais simples, um problema de classificação binária: a partir de um *e-mail* de entrada, o sistema atribui um entre dois rótulos possíveis, *phishing* ou legítimo. A acurácia isolada não é suficiente para avaliar classificadores desse tipo, pois os erros têm custos assimétricos, como ocorre na segurança da informação; por isso, recorre-se a um conjunto de métricas que distinguem os tipos de acerto e de erro cometidos pelo modelo (POWERS, 2020; AXELSSON, 2000). Ainda que a decisão final seja binária, os modelos avaliados neste trabalho também expõem uma pontuação contínua de suspeita, o que permite examinar o desempenho conforme o limiar de decisão varia. Esta seção aborda, assim, tanto as métricas derivadas da decisão discreta quanto os instrumentos próprios da pontuação contínua, como a curva ROC e a AUC.

A análise sistemática do desempenho de um classificador binário parte da matriz de confusão (Quadro 1), estrutura que cruza as predições do modelo com os rótulos verdadeiros (*ground truth*) do conjunto de teste e organiza os resultados em quatro categorias fundamentais (POWERS, 2020):

- **Verdadeiros positivos** (*TP*, *true positives*): *e-mails* de *phishing* corretamente classificados como *phishing*;

- **Verdadeiros negativos** (*TN, true negatives*): *e-mails* legítimos corretamente classificados como legítimos;
- **Falsos positivos** (*FP, false positives*): *e-mails* legítimos incorretamente sinalizados como *phishing* pelo modelo;
- **Falsos negativos** (*FN, false negatives*): *e-mails* de *phishing* que o modelo deixou passar, classificando-os como legítimos.

		Valor verdadeiro	
		Positivos	Negativos
Previsto	Positivos	TP	FP
	Negativos	FN	TN

Fonte: Elaborado pelo autor.

Quadro 1: Matriz de confusão.

Dessas quatro contagens derivam-se as principais métricas empregadas na avaliação de classificadores binários, cada uma respondendo a uma pergunta distinta sobre o comportamento do modelo (POWERS, 2020). A acurácia (*accuracy*) é a de interpretação mais imediata e corresponde à proporção total de classificações corretas sobre o conjunto avaliado, conforme a Equação 2.1:

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.1)$$

Apesar de sua simplicidade, a acurácia pode transmitir uma impressão enganosa do desempenho real quando as classes do problema são desbalanceadas. Powers (2020) observa que, em distribuições assimétricas, um classificador trivial que sempre prevê a classe majoritária atinge acurácia muito elevada sem gerar qualquer informação útil. Se apenas 1% dos *e-mails* recebidos forem tentativas de *phishing*, por exemplo, um modelo que nunca sinaliza mensagem alguma como suspeita alcançaria 99% de acurácia, número aparentemente excelente, porém inútil na prática, pois nenhum ataque real seria detectado. Embora o *dataset* construído neste trabalho seja deliberadamente balanceado entre as duas classes, o que atenua esse problema no plano experimental, a acurácia isolada nada informa sobre a natureza dos erros cometidos, o que justifica o uso de métricas complementares.

A precisão (*precision*) responde à pergunta: "dos *e-mails* que o modelo marcou como *phishing*, quantos eram de fato tentativas de *phishing*". Sua definição formal consta na Equação 2.2 (POWERS, 2020):

$$\text{Precisão} = \frac{TP}{TP + FP}. \quad (2.2)$$

Precisão elevada indica que, quando o modelo levanta uma suspeita, ela tende a ser fundamentada, o que a torna particularmente sensível à taxa de falsos positivos (POWERS, 2020).

A revocação (*recall*), também denominada sensibilidade (*sensitivity*) ou taxa de verdadeiros positivos (*TPR, true positive rate*), responde à pergunta: "dos *e-mails* de *phishing* existentes no conjunto avaliado, quantos o modelo conseguiu identificar?". Sua formulação consta na Equação 2.3 (POWERS, 2020):

$$\text{Revocação} = \frac{TP}{TP + FN}. \quad (2.3)$$

Revocação elevada significa que o modelo deixa passar poucos ataques reais, o que a torna sensível à taxa de falsos negativos (POWERS, 2020).

Além dessas duas medidas centradas na classe positiva, convém reportar de forma explícita as taxas de erro. A taxa de falsos positivos (*FPR, false positive rate*) é a proporção de *e-mails* legítimos sinalizados de modo incorreto, conforme a Equação 2.4 (POWERS, 2020):

$$\text{Taxa de Falsos Positivos (FPR)} = \frac{FP}{FP + TN}. \quad (2.4)$$

A taxa de falsos negativos (*FNR, false negative rate*) é a proporção de *e-mails* de *phishing* que escaparam ao modelo e corresponde ao complemento da revocação, conforme a Equação 2.5 (POWERS, 2020):

$$\text{Taxa de Falsos Negativos (FNR)} = \frac{FN}{FN + TP} = 1 - \text{Revocação}. \quad (2.5)$$

Como precisão e revocação tendem a variar em sentidos opostos (modelos mais agressivos na sinalização de *phishing* elevam a revocação ao custo da precisão, e vice-versa (POWERS, 2020)), consolidou-se a prática de sintetizar essas duas métricas em uma única quantidade. O *F1-Score* é a média harmônica entre precisão e revocação, conforme a Equação 2.6 (POWERS, 2020):

$$F1\text{-Score} = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}}. \quad (2.6)$$

Utiliza-se a média harmônica, em detrimento da aritmética pois ela penaliza fortemente os desequilíbrios entre as duas componentes. Um modelo com precisão de 100% e revocação de 1% obteria *F1-Score* próximo a 2%, o que reflete sua inutilidade na prática. O *F1-Score* só assume valores elevados quando ambas as métricas o são ao mesmo tempo, propriedade que o torna adequado como métrica-síntese em problemas como o aqui investigado.

O *F1-Score*, porém, ignora os verdadeiros negativos: seu cálculo não usa a contagem *TN*. O coeficiente de correlação de *Matthews* (*MCC*, *Matthews correlation coefficient*) corrige essa lacuna ao incorporar as quatro células da matriz de confusão em uma única medida (CHICCO; JURMAN, 2020). Ele expressa a correlação entre os rótulos previstos e os verdadeiros e varia entre -1 e $+1$: o valor $+1$ indica classificação perfeita, 0 corresponde a desempenho equivalente ao acaso e -1 a desacordo total entre predição e realidade. Sua definição consta na Equação 2.7:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (2.7)$$

Por considerar acertos e erros nas duas classes, o *MCC* só produz valor alto quando o modelo apresenta bom desempenho tanto nos *e-mails* de *phishing* quanto nos legítimos, o que o torna mais informativo que o *F1-Score* sob desbalanceamento de classes (CHICCO; JURMAN, 2020). Por essa razão, é reportado neste trabalho como medida-síntese complementar.

2.4.1 O *trade-off* entre falsos positivos e falsos negativos no contexto de segurança

A escolha do ponto operacional de um classificador, ou seja, o grau em que se prioriza a precisão em detrimento da revocação ou o contrário, não constitui uma decisão puramente técnica e depende do contexto em que o sistema será implantado. Na segurança da informação, os dois tipos de erro acarretam custos distintos e, em geral, assimétricos.

Um falso negativo, ou seja, deixar passar um *e-mail* de *phishing*, representa uma falha defensiva direta: a mensagem maliciosa chega à caixa de entrada do usuário, e a contenção do ataque passa a depender unicamente da percepção humana, justamente o elo reconhecidamente mais frágil da cadeia de segurança (BAKI; VERMA, 2021). As consequências de um único falso negativo bem-sucedido podem ser graves: comprometimento de credenciais, perda financeira, instalação de *malware* ou exfiltração de dados sigilosos (BAKI; VERMA, 2021; GRIFFITHS, 2025). Já um falso positivo, ou seja, marcar um *e-mail* legítimo como *phishing*, gera custos de produtividade e operação: mensagens relevantes podem ser ignoradas, atrasadas ou bloqueadas e, com o tempo, a recorrência de alarmes falsos corrói a confiança do usuário no sistema, fenômeno conhecido na literatura como fadiga de alertas (TARIQ et al., 2025).

Em análise clássica sobre sistemas de detecção de intrusão (*IDS*), Axelsson (2000) argumenta que o fator limitante da efetividade prática de sistemas defensivos costuma ser a taxa de falsos alarmes, e não a taxa de detecção em si. Esse fenômeno tem raiz na chamada falácia da taxa de base (*base-rate fallacy*): como, em operação normal, o volume de eventos benignos supera o de eventos maliciosos em várias ordens de

grandeza, até taxas baixíssimas de falsos positivos geram, em termos absolutos, mais alarmes falsos do que verdadeiros, o que sobrecarrega o operador humano e o leva a desconsiderar progressivamente os alertas emitidos (BAR-HILLEL, 1980). Ainda que a análise original de Axelsson (2000) tenha sido desenvolvida para sistemas IDS baseados em registros de auditoria, sua lógica se transfere com naturalidade à detecção de *phishing* por *e-mail*: o volume de mensagens legítimas que trafega em qualquer organização excede em muito o número de tentativas diárias de *phishing*, e um classificador que reporta volumes excessivos de falsos positivos passa rapidamente a ser ignorado, o que compromete sua utilidade independentemente de sua capacidade nominal de detecção (TARIQ et al., 2025).

Por essas razões, este trabalho adota o *F1-Score* como métrica principal, pois equilibra precisão e revocação em uma única quantidade resistente a desbalanceamentos, mas reporta também precisão, revocação, *FPR* e *FNR* de forma separada, de modo que organizações com prioridades operacionais distintas possam interpretar os resultados conforme seus próprios *trade-offs*.

2.4.2 Curva ROC e AUC

Quando um classificador produz uma pontuação contínua (por exemplo, uma probabilidade estimada de pertinência à classe positiva), em vez de um rótulo binário direto, o limiar de decisão a partir do qual a pontuação é convertida em rótulo passa a ser um parâmetro ajustável (FAWCETT, 2006). Variar esse limiar desloca o classificador entre diferentes pontos operacionais, trocando revocação por precisão e vice-versa (FAWCETT, 2006). A curva *ROC* (*Receiver Operating Characteristic*) é a representação gráfica desse *trade-off*, que dispõe a *True Positive Rate* (*TPR*) no eixo vertical contra a *False Positive Rate* (*FPR*) no eixo horizontal à medida que o limiar varia (POWERS, 2020; FAWCETT, 2006).

Um classificador puramente aleatório produz uma curva que coincide com a diagonal entre os pontos (0, 0) e (1, 1) (linha vermelha na Figura 1), situação em que qualquer ganho em revocação é acompanhado de um aumento equivalente na taxa de falsos positivos (FAWCETT, 2006). Um classificador perfeito, por sua vez, alcança o vértice superior esquerdo do gráfico ($FPR = 0$, $TPR = 1$), no qual todos os positivos são corretamente identificados sem que nenhum negativo seja erroneamente sinalizado (linha verde na Figura 1) (FAWCETT, 2006). A área sob a curva ROC (*AUC*, *area under the curve*) sintetiza, em um único valor entre 0 e 1, a capacidade discriminativa do classificador de forma independente de qualquer limiar específico: uma *AUC* igual a 1 indica separação perfeita entre as classes, enquanto uma *AUC* próxima a 0,5 indica desempenho equivalente ao acaso (FAWCETT, 2006).

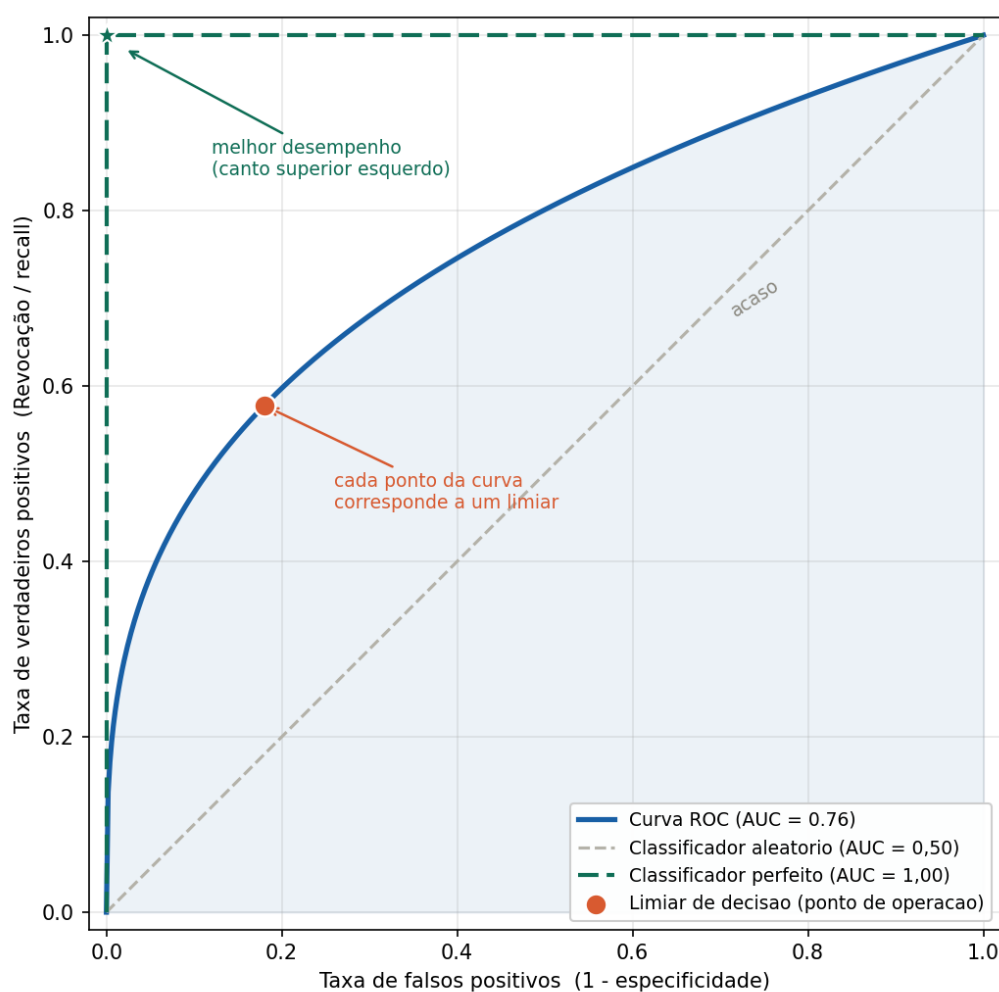


Figura 1 – Curva ROC.

Fonte: Elaborado pelo autor.

2.4.3 Comparação estatística entre classificadores

As métricas descritas até aqui resumem o desempenho de cada classificador isoladamente. Quando o objetivo é decidir se um modelo é de fato melhor que outro, porém, uma diferença numérica nas métricas não basta, pois ela pode ter surgido por acaso, em função da amostra de teste utilizada. O teste de *McNemar* permite avaliar se essa diferença é estatisticamente significativa (DIETTERICH, 1998). Aplicado a dois classificadores avaliados sobre o mesmo conjunto de teste, o teste considera apenas os casos em que eles discordam: as mensagens que o primeiro acertou e o segundo errou, e vice-versa (DIETTERICH, 1998). A hipótese nula é que ambos têm a mesma taxa de erro, situação em que esses dois tipos de discordância deveriam ocorrer em proporções semelhantes. Um desequilíbrio acentuado entre eles, medido por uma estatística com distribuição qui-quadrado, leva à rejeição da hipótese nula e indica que a diferença de desempenho dificilmente é fruto do acaso (DIETTERICH, 1998).

Para formalizar o teste, classificam-se as predições dos dois modelos sobre o mesmo conjunto em quatro contagens, segundo o acerto ou erro de cada um, como mostra o Quadro 2. Interessam apenas as duas células de discordância: b , o número de mensagens que o primeiro modelo acertou e o segundo errou, e c , o número daquelas que o primeiro errou e o segundo acertou. As células de concordância, a e d , em que ambos acertam ou ambos erram, não entram no cálculo, pois não distinguem os modelos.

		Modelo 2	
		Acertou	Errou
Modelo 1	Acertou	a	b
	Errou	c	d

Fonte: Elaborado pelo autor.

Quadro 2: Tabela de contingência para o teste de McNemar.

Sob a hipótese nula, espera-se $b = c$, e a estatística do teste mede o quanto a discordância observada se afasta desse equilíbrio, conforme a Equação 2.8:

$$\chi^2 = \frac{(b - c)^2}{b + c}. \quad (2.8)$$

Como b e c são contagens discretas, mas a estatística é aproximada por uma distribuição qui-quadrado contínua com um grau de liberdade, aplica-se a correção de continuidade de Yates, que torna o teste mais conservador e reduz o risco de rejeitar a hipótese nula indevidamente, principalmente quando o total de discordâncias é pequeno (DIETTERICH, 1998). A forma corrigida é apresentada na Equação 2.9:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}. \quad (2.9)$$

O valor de χ^2 é então convertido em um valor- p ; quando este fica abaixo do nível de significância adotado, rejeita-se a hipótese nula e conclui-se que a diferença de desempenho entre os modelos dificilmente é fruto do acaso. Por trabalhar com predições pareadas sobre os mesmos dados, o teste é apropriado para a comparação direta entre modelos (DIETTERICH, 1998).

O teste de *McNemar* responde se dois modelos diferem, mas o faz apenas sobre a decisão discreta e em termos da taxa de erro. Ele não informa o tamanho da diferença nem se aplica diretamente a métricas como o *F1-Score*, o coeficiente de correlação de *Matthews* (*MCC*) ou a *AUC*, que combinam as contagens de forma não linear e não possuem fórmula fechada simples para a variância da diferença entre dois modelos.

Para esses casos, o *bootstrap* oferece uma alternativa empírica (EFRON; TIBSHIRANI, 1993).

O *bootstrap* estima a distribuição amostral de uma estatística por reamostragem dos próprios dados (EFRON; TIBSHIRANI, 1993; DROR et al., 2018). A partir do conjunto de teste original, com n *e-mails*, sorteiam-se, com reposição, B novos conjuntos de mesmo tamanho n , chamados réplicas (tipicamente B na casa dos milhares) (EFRON; TIBSHIRANI, 1993; DROR et al., 2018). Em cada réplica, recalcula-se a métrica de interesse para os dois modelos e a diferença Δ entre eles ($\Delta F1$, ΔMCC , ΔAUC) (EFRON; TIBSHIRANI, 1993; DROR et al., 2018). Os B valores de Δ assim obtidos aproximam a distribuição da diferença, e o intervalo de confiança de 95% corresponde aos percentis de 2,5% e 97,5% dessa distribuição (EFRON; TIBSHIRANI, 1993; DROR et al., 2018). Cabe observar que, em cada réplica, os mesmos *e-mails* reamostrados alimentam os dois modelos (*bootstrap* pareado), o que preserva a correlação entre eles e produz um intervalo correto e mais estreito (EFRON; TIBSHIRANI, 1993; DROR et al., 2018). Se o intervalo não contém o zero, a diferença é significativa no nível adotado; sua largura e posição ainda revelam a magnitude e a direção da vantagem, informação que um teste de hipótese isolado não fornece (EFRON; TIBSHIRANI, 1993; DROR et al., 2018).

2.4.4 Latência e o percentil 95

Além da qualidade da classificação, a viabilidade prática de um classificador depende de seu custo de execução, sobretudo da latência, definida como o tempo decorrido entre a submissão de um *e-mail* e a devolução da resposta pelo modelo. A média da latência resume o comportamento típico, mas é sensível a valores extremos e pouco informa sobre as respostas mais lentas, justamente as que comprometem a experiência de uso em um fluxo real de triagem. Por isso, reporta-se também o percentil 95 ($p95$), valor abaixo do qual se situam 95% das medições, de modo que apenas 5% das inferências demoram mais que ele. O $p95$ funciona, assim, como indicador da cauda superior da distribuição, ou seja, do pior caso esperado em condições normais de operação. A leitura conjunta da média e do $p95$ distingue modelos de latência estável, em que os dois valores são próximos, de modelos de desempenho irregular, em que o $p95$ se distancia da média e revela picos ocasionais de lentidão.

3 REVISÃO DO ESTADO DA ARTE

O Capítulo 2 apresentou os fundamentos sobre segurança da informação, *phishing*, modelos de linguagem de larga escala e métricas de avaliação de classificadores binários. Este capítulo dedica-se ao mapeamento da produção científica recente que reúne esses temas, com o intuito de situar a pesquisa frente à literatura existente e de identificar as convergências metodológicas, as omissões e as lacunas que justificam a investigação proposta.

O capítulo organiza-se em quatro seções. A Seção 3.1 descreve o protocolo de revisão sistemática adotado. A Seção 3.2 detalha as estratégias de busca e os filtros de seleção aplicados. A Seção 3.3 analisa criticamente cada trabalho correlato selecionado e expõe suas lacunas. Por fim, a Seção 3.4 sintetiza as limitações comuns aos trabalhos e posiciona a proposta deste *TCC* frente a elas.

3.1 PROTOCOLO DE REVISÃO SISTEMÁTICA

Para este trabalho adotou-se a revisão sistemática leve, que apresenta as principais abordagens da literatura junto ao processo estruturado de seleção documental que as identificou (KITCHENHAM, 2004). A revisão compreendeu cinco etapas sequenciais: definição das palavras-chave, execução das buscas, filtragem preliminar, leitura dinâmica e síntese dos resultados. O recorte temporal das buscas estendeu-se de outubro de 2025 a maio de 2026.

A base de dados utilizada na recuperação dos artigos foi a *Scopus* (Elsevier, 2026), amplamente reconhecida pela cobertura de produções científicas em computação, uma vez que reúne indexadores consolidados como *IEEE*, *Association for Computing Machinery* e *Elsevier*, entre outros. O *Google Scholar*, embora não tenha servido como base de busca primária, foi empregado como fonte auxiliar para aferir o *H5-index* (MESTER, 2016) dos periódicos e conferências em que os artigos recuperados haviam sido publicados, conforme detalhado na Seção 3.2.

3.2 ESTRATÉGIAS DE BUSCA E FILTROS DE SELEÇÃO

A seleção dos trabalhos correlatos seguiu uma busca em etapas sucessivas de refinamento. Inicialmente, na plataforma *Scopus*, foram filtrados os trabalhos cujos títulos versassem simultaneamente sobre modelos de linguagem de larga escala e *phishing* por *e-mail*, por meio da seguinte *string* de consulta:

```
( "large language model"OR "LLM"OR "language model") AND ( "phishing  
detection"OR "email phishing"OR "phishing")
```

Restringir a consulta ao campo *título*, sem buscar em palavras-chave e *abstract*, foi uma decisão deliberada: priorizaram-se os trabalhos cuja contribuição central estivesse explicitamente na intersecção entre detecção de *phishing* e *LLMs*, e não os que apenas tangenciavam o tema. Essa primeira consulta retornou 91 artigos, posteriormente submetidos aos filtros descritos a seguir.

O segundo filtro baseou-se na métrica *H5* dos veículos de publicação, abordagem inspirada em (SILVANO; MARCELINO, 2020). O *H5-index* corresponde ao maior número h tal que h artigos publicados pelo veículo nos últimos cinco anos tenham recebido ao menos h citações cada um (MESTER, 2016). Trata-se de uma métrica reconhecida pela comunidade científica por sua simplicidade de cálculo e por considerar simultaneamente o volume e o impacto das publicações, pois um periódico não alcança um *H5-index* elevado sem antes acumular um conjunto consistente de publicações com múltiplas citações (MESTER, 2016). Adotou-se como limiar mínimo um *H5-index* igual ou superior a vinte, conforme números obtidos no *Google Scholar*, ou seja, aceitaram-se apenas trabalhos de veículos com ao menos vinte trabalhos com pelo menos vinte citações nos últimos cinco anos. Esse critério visou assegurar que os documentos analisados tivessem passado por um processo seletivo criterioso e revisado por pares. A aplicação do filtro eliminou trinta e oito artigos, restando cinquenta e três.

A terceira etapa de filtragem consistiu na avaliação dos títulos e das palavras-chave dos artigos remanescentes, com o objetivo de remover trabalhos cujo escopo, embora situado na intersecção entre *LLMs* e *phishing*, divergisse da proposta deste *TCC*. Descartaram-se, especificamente, os trabalhos centrados exclusiva ou majoritariamente na *geração* de *e-mails* de *phishing* por *LLMs*, no *fine-tuning* de modelos próprios ou em outras técnicas não abordadas neste documento, preservando-se apenas os voltados à *detecção* de tentativas de engenharia social com modelos de linguagem em larga escala. Como instrumento de decisão, formulou-se a seguinte pergunta: "Este trabalho trata da utilização de *LLMs* na detecção de *phishing* sem a necessidade de *fine-tuning*, treinamento próprio ou outras técnicas que não serão utilizadas neste artigo?". Os trinta e sete trabalhos cuja resposta foi negativa foram descartados, restando dezesseis artigos.

Por fim, realizou-se a leitura dinâmica dos artigos remanescentes. Como critério adicional, removeram-se os trabalhos que não empregassem diretamente o *prompting* sobre *LLMs* já treinadas e disponíveis para a avaliação empírica de métricas de classificação ou de técnicas de *prompting*. Nessa etapa, cada trabalho foi avaliado segundo a pergunta: "Este artigo avaliou empiricamente modelos de linguagem na tarefa de detecção de *phishing*, comparando mais de um modelo entre si ou múltiplas

técnicas de *prompting* em um ou mais modelos?". Os trabalhos com resposta negativa foram descartados, e os demais, aceitos. Esse último filtro eliminou treze artigos e resultou em três trabalhos correlatos, analisados em detalhe na Seção 3.3. A Figura PRISMA a seguir (2) resume os filtros aplicados:

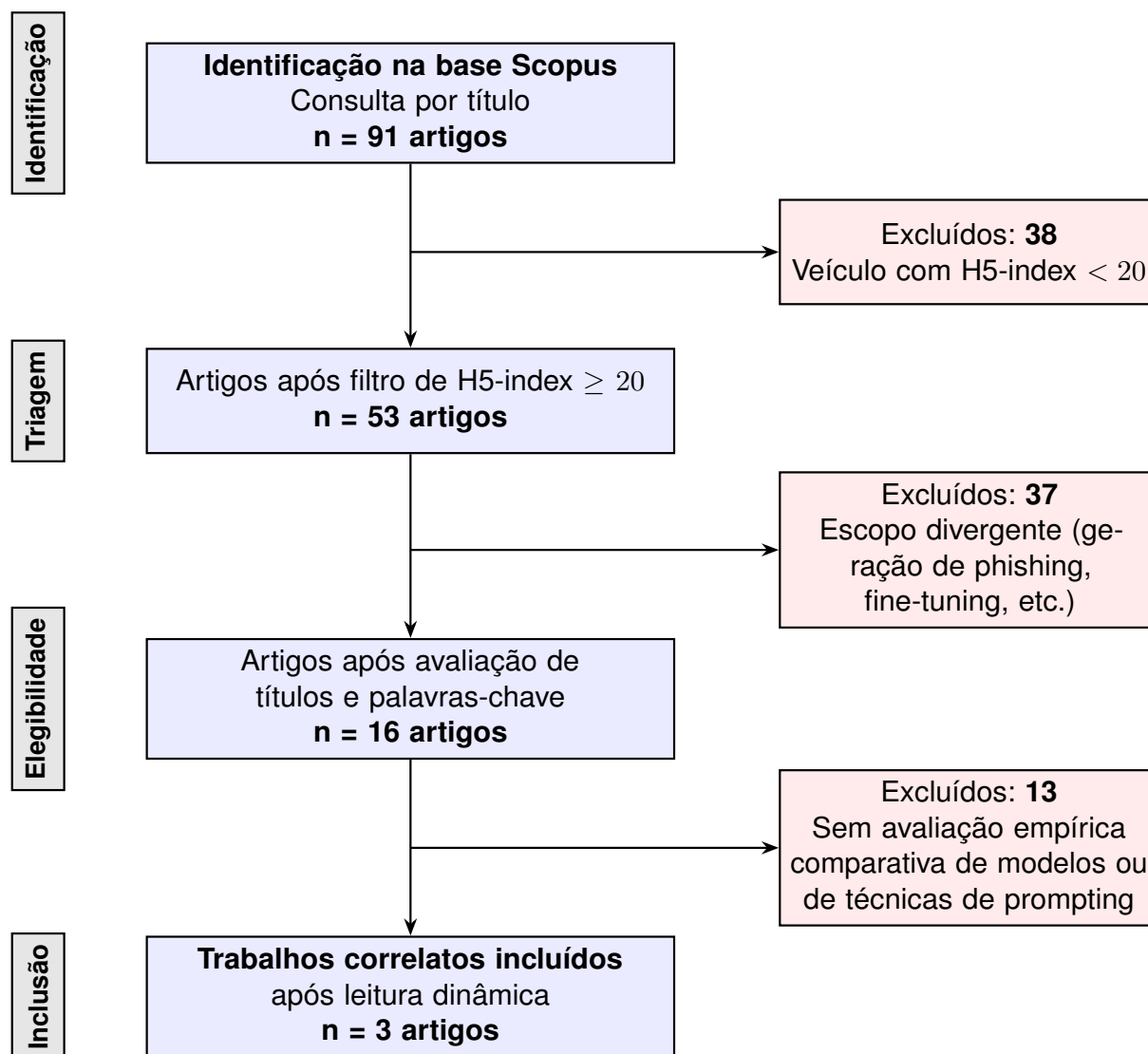


Figura 2 – Diagrama PRISMA da seleção dos trabalhos correlatos.

Fonte: elaborado pelo autor.

A relação completa dos documentos avaliados em cada etapa de triagem, com as respectivas decisões e motivos de exclusão, encontra-se no Apêndice A (Quadro 9).

Durante o processo de leitura, observou-se que a literatura atual associando os temas "phishing" e "modelos de linguagem em larga escala" pode ser organizada em três grandes grupos:

1. trabalhos que utilizam *LLMs* para a criação de *datasets* de *phishing*;
2. trabalhos que utilizam *LLMs* para a identificação de *phishing*, subdivididos em:

- a) artigos que empregam técnicas de *prompting* sobre *LLMs* já treinadas para identificação de *phishing*; categoria na qual este trabalho se insere;
 - b) artigos baseados em modelos pré-treinados associados a *fine-tuning* para identificação de *phishing*;
3. sistemas que combinam *LLMs* com outras técnicas para a detecção de *phishing*.

3.3 ANÁLISE CRÍTICA DOS TRABALHOS CORRELATOS

Esta seção examina individualmente os três trabalhos selecionados ao final do processo de filtragem descrito na Seção 3.2. Para cada um, identificam-se o problema abordado, a metodologia experimental adotada, incluindo modelos avaliados, conjunto de dados empregado e técnicas de *prompting* utilizadas, além dos principais resultados reportados e das limitações relevantes. A comparação direta entre cada trabalho e a proposta deste TCC é desenvolvida na Seção 3.4.

3.3.1 *Benchmarking and Evaluating Large Language Models in Phishing Detection for Small and Midsize Enterprises: A Comprehensive Analysis*

Em (ZHANG et al., 2025), os autores realizam um *benchmark* comparativo entre doze modelos de linguagem de larga escala na tarefa de detecção de *phishing* por *e-mail*, com foco específico nas necessidades de pequenas e médias empresas. O conjunto avaliado contempla quatro modelos proprietários (*GPT-4o*, *Claude 3.5 Sonnet*, *Gemini 1.5 Pro* e *GPT-3.5*) e oito modelos *open-source*, entre os quais *Llama-3*, *Gemma-2*, *Qwen2* e *Phi-3*, todos utilizados em sua versão base, sem *fine-tuning* ou ajuste de hiperparâmetros. O protocolo experimental aplica um único *template* de instrução *zero-shot* e avalia o desempenho com métricas de classificação binária, incluindo acurácia, precisão, revocação, *F1-score*, média geométrica, *BDR* e coeficiente de correlação de *Matthews*. O conjunto de dados é composto por 40 *e-mails* em inglês, balanceados em quatro categorias de dez mensagens cada (legítimos humanos, legítimos gerados por IA, *phishing* humanos e *phishing* gerados por IA), construído a partir dos *datasets* públicos "*Phishing Email Data by Type*" e "*Phishing Pot*", e suplementado por mensagens geradas pelo próprio *Llama-3-8b-Instruct*, uma escolha que os autores reconhecem como possível fonte de viés nos resultados das categorias geradas por IA.

O principal achado do estudo é o desempenho superior do *Llama-3-8b-Instruct*, que obteve *F1-score* de 0,9756, superando todos os demais modelos avaliados, inclusive seu equivalente de maior porte na mesma família (*Llama-3-70b-Instruct*, com *F1* = 0,9000) e modelos proprietários como *GPT-4o* (*F1* = 0,7083). Esse resultado sustenta a tese central dos autores, segundo a qual a viabilidade de detecção de *phishing* por *LLMs*

leves desafia a heurística de que modelos maiores e proprietários necessariamente apresentam desempenho superior em tarefas específicas de segurança. O estudo reconhece, entretanto, limitações importantes, entre as quais a restrição a modelos base, a inconsistência das respostas dos *LLMs* frente ao mesmo *input* em execuções distintas, a baixa eficácia contra ataques sofisticados como *spear phishing* e *Business Email Compromise*, e a possibilidade de atacantes refinarem iterativamente mensagens via *LLMs* generativos até burlarem classificadores baseados em *LLM*.

Quanto à construção do *dataset*, os autores criticam explicitamente *datasets* amplamente utilizados na literatura¹, alegando que se encontram desatualizados e contêm mensagens que não refletem o cenário contemporâneo de ataques, classificando-os como "inferiores". Embora indiquem a origem e a distribuição quantitativa dos *e-mails* utilizados, não disponibilizam os identificadores específicos das amostras selecionadas nem o conjunto final consolidado. A omissão compromete a reprodutibilidade, pilar do método científico, pois, sem as amostras exatas, nenhum pesquisador consegue repetir o experimento para confirmar ou refutar os resultados relatados. O problema agrava-se porque parte do *dataset* foi sintetizada pelos próprios autores com *LLM*, e essas mensagens tampouco foram publicadas, o que torna o conjunto avaliado, na prática, irreproduzível. A síntese do conjunto, todavia, afasta o risco de contaminação dos dados de teste (Seção 2.3.3), ainda que os autores não mencionem essa vantagem. Por fim, os autores não reconhecem a falha de privacidade inerente ao envio de *e-mails* pessoais a servidores externos, que leem o conteúdo e o utilizam para o treino de novos modelos.

3.3.2 *Can AI Keep You Safe? A Study of Large Language Models for Phishing Detection*

O artigo Chataut, Gyawali e Usman (2024) investiga a eficácia de *LLMs* na detecção de *e-mails* de *phishing*, comparando o desempenho de três modelos: *GPT-3.5*, *GPT-4* e uma versão customizada denominada *CyberGPT*² especialmente ajustada para essa tarefa por meio de instruções específicas sobre indicadores de *phishing* (remetentes suspeitos, links duvidosos, linguagem de urgência e solicitação de informações sensíveis). Com um conjunto de dados de 828 *e-mails* (504 legítimos e 324 de *phishing*) e *prompts* estruturados que retornam respostas em formato *JSON* com classificação, justificativa e nível de confiança, os autores constatarem que o *GPT-3.5* apresenta desempenho limitado (acurácia de 80,68% e revocação de apenas 52,47%), ao passo que o *GPT-4* (97,22%) e o *CyberGPT* (97,46%) alcançam resultados expressivamente

¹ nominalmente *Enron*, *SpamAssassin*, *TREC*, *Ling*, *CEAS*, *Nazario* e *Nigerian Fraud*

² a customização se deu utilizando a função de "GPTs customizados" da *OpenAI*, um recurso que permite utilizar diretrizes mais detalhadas. Os autores referem-se a este processo no artigo como "*fine-tuning*", contudo trata-se apenas de um tipo de instrução "especial", que modifica o comportamento do modelo (*OpenAI*, 2023).

superiores, com alta precisão, revocação acima de 93% e maior confiança nas predições. O estudo conclui que *LLMs* avançados, principalmente quando customizados, representam uma ferramenta promissora para fortalecer a cibersegurança e proteger usuários contra fraudes *online*.

Todavia, os autores avaliaram somente modelos grandes proprietários, sem considerar que o envio de *e-mails* privados a um servidor externo via *API* expõe dados pessoais e pode infringir legislações de privacidade, o que compromete tanto o experimento quanto a conclusão de que *LLMs* como o *GPT* seriam aplicáveis, na prática, à detecção de *e-mails*. Adicionalmente, tal qual em (ZHANG et al., 2025), os autores informam o nome do *dataset* utilizado, mas não os identificadores das amostras avaliadas. Essa omissão fere a reprodutibilidade, princípio básico do método científico, pois impede que outros pesquisadores repitam o experimento sobre o mesmo conjunto e verifiquem os resultados relatados. Por fim, a qualidade estrutural do *dataset* (JOURNAL, 2023) merece atenção: apesar da pontuação de usabilidade de 9,41 na plataforma *Kaggle*, os registros não separam o assunto do corpo do *e-mail*, concatenando ambos em uma única *string*, e, em parte das amostras, o assunto sequer está presente. Essa ausência de estrutura impede que o modelo receba os campos de forma identificada, o que pode prejudicar a análise de indicadores de *phishing* presentes especificamente no assunto.

Há ainda uma limitação que os próprios autores não levantam: a contaminação dos dados de teste, discutida na Seção 2.3.3. O *dataset* empregado é público e aberto, de modo que suas amostras podem ter integrado o treino dos modelos proprietários avaliados, todos de grande porte e, portanto, mais propensos à memorização. Como a *OpenAI* não divulga os dados de treinamento do *GPT-3.5* e do *GPT-4*, a hipótese não pode ser confirmada nem afastada, o que enfraquece a interpretação dos resultados.

Essa crítica adquire maior relevância pois os autores não propõem apenas um experimento, mas apresentam como proposta central o emprego prático desses modelos em segurança corporativa e pessoal ³. Uma adoção nesses moldes processaria as mensagens reais dos usuários, o que converte o risco de privacidade em um problema operacional, agravado pelos fatores discutidos na Seção 2.3.4.

3.3.3 *Devising and Detecting Phishing Emails Using Large Language Models*

O trabalho de Heiding et al. (2024) distingue-se dos anteriores por não se restringir à detecção, tratando de duas frentes distintas. Na primeira, geram-se *e-mails* de *phishing* e compara-se a eficácia de mensagens criadas pelo *GPT-4*, por um modelo humano de engenharia social (a *V-Triad*) e pela combinação das duas, medindo-se quantos participantes de um experimento com pessoas reais clicaram nos *links*. Na segunda, à

³ "..., the continuous development and deployment of such models hold promise for safeguarding individuals and organizations from the perils of online fraud..."

qual este trabalho se atém, avalia-se a capacidade de *LLMs* de detectar essas mesmas mensagens. Para a detecção, submeteram-se quatro modelos comerciais acessados via nuvem (*GPT-4*, *Claude-1*, *Bard* e *ChatLLaMA*⁴) a um conjunto de vinte *e-mails*, que reunia mensagens fraudulentas produzidas pelos próprios autores, exemplos retirados de repositórios públicos, *e-mails* legítimos de *marketing* e mensagens extraídas das caixas pessoais dos pesquisadores. Cada *e-mail* foi apresentado como texto puro, sem imagens nem logotipos, e o modelo era reiniciado a cada nova análise. Para cada mensagem, formularam-se quatro perguntas ao modelo, sobre a intenção do *e-mail*, a probabilidade de ser suspeito, a recomendação de resposta ao usuário e a chance de ter sido escrito por uma IA.

O resultado mais relevante para esta pesquisa diz respeito à sensibilidade do modelo à forma da pergunta. Questionado apenas sobre a intenção da mensagem, o *Claude*, melhor modelo do estudo, identificou corretamente como maliciosos 75% dos *e-mails* de *phishing* convencionais usados como controle, mas somente 25% dos gerados pelo *GPT-4* e 25% dos produzidos pela abordagem híbrida. Questionado de forma direta sobre o quão suspeita era a mensagem, a detecção elevou-se para 100% nas categorias criadas pelo *GPT-4*, pela *V-Triad* e pela combinação das duas. Instruir o modelo a procurar sinais de fraude alterou o resultado de forma expressiva, sem modificar o *e-mail* analisado. O *Claude* foi também o modelo mais estável e o único a oferecer recomendações consistentemente úteis, como consultar o site oficial da empresa antes de responder. Em contrapartida, registra-se uma variância significativa no *Bard*, que, quando não reiniciado entre perguntas, elevava a probabilidade de suspeita a cada nova consulta, até rotular como fraudulentos *e-mails* legítimos. Em alguns casos os modelos igualaram ou superaram humanos na identificação da intenção maliciosa, ainda que em geral ficassem um pouco abaixo.

As limitações da etapa de detecção, várias delas admitidas pelos próprios autores, restringem o alcance desses números. A amostra de vinte *e-mails* por modelo é pequena, e o próprio texto reconhece que os resultados devem ser lidos como indício, exigindo um conjunto maior para conclusões firmes. Como a detecção constitui a parte secundária de um artigo voltado majoritariamente à geração de *phishing*, o experimento tem caráter mais exploratório que conclusivo. Os autores observam ainda uma variância interna, pois um mesmo modelo, diante do mesmo *e-mail*, por vezes devolve classificações distintas, o que, segundo eles, indica faltar estabilidade para empregar *LLMs* de forma confiável nessa tarefa.

Cabe registrar ainda outros pontos. Os quatro modelos avaliados são todos comerciais e acessados remotamente por serviços de nuvem, e suas versões já foram sucedidas por gerações mais recentes. Como boa parte das mensagens foi montada para o próprio experimento, os autores escapam do risco de contaminação dos dados

⁴ modelo baseado no *LLaMA-2*

de teste (Seção 2.3.3), embora não comentem esse ponto. Há também a questão da privacidade, pois os autores submetem *e-mails* de suas próprias caixas a serviços externos e chegam a sugerir o uso prático desses modelos em filtros de *spam*, sem problematizar a exposição de mensagens privadas a terceiros, conforme discutido na Seção 2.3.4.

3.4 ANÁLISE COMPARATIVA DOS TRABALHOS

A análise individual conduzida na Seção 3.3 evidencia limitações recorrentes nos três trabalhos correlatos. Esta seção reúne os pontos comuns a eles e, a partir dessa convergência, delimita como a presente proposta se distingue da literatura examinada.

A primeira limitação está na seleção dos modelos. Predominam *LLMs* comerciais de grande porte ou modelos abertos com dezenas de bilhões de parâmetros, ao passo que os modelos leves, de poucos bilhões e viáveis para execução local em *hardware* acessível, recebem pouca atenção ou estão ausentes. Mais relevante que a escassez desses modelos é a falta da pergunta comparativa que orienta este trabalho, pois a maior parte dos correlatos avalia se determinado modelo é adequado à tarefa, e não se um modelo leve de execução local consegue rivalizar com um grande modelo comercial. Constitui exceção parcial o estudo de Zhang et al. (2025), que confronta portes distintos e registra um modelo aberto menor superando proprietários maiores, ainda que sob um único *template zero-shot* e com modelos hoje defasados.

A segunda limitação reside na opacidade dos dados, já que nenhum dos trabalhos disponibiliza uma base ampla e rastreável, seja por omitir os identificadores das amostras utilizadas (ZHANG et al., 2025; CHATAUT; GYAWALI; USMAN, 2024), seja por se apoiar em conjunto pequeno demais para servir de referência (HEIDING et al., 2024). A terceira diz respeito à técnica de *prompting*, com forte predomínio de *zero-shot* e *persona prompting* e pouca exploração de alternativas como *few-shot* ou *chain-of-thought*, ainda que estudos recentes indiquem que o *persona prompting* apenas torna a resposta do modelo mais convincente, sem elevar seu acerto efetivo (HU; ROSTAMI; THOMASON, 2026).

A quarta limitação, e eixo central deste trabalho, é o tratamento marginal conferido à privacidade. O envio de *e-mails* a servidores externos expõe dados pessoais e pode violar a *LGPD* e o *GDPR* (Seção 2.3.4); ainda assim, os trabalhos ou ignoram o ponto, ou recomendam o uso prático de *APIs* sem problematizá-lo, ou apenas tangenciam a questão ao sugerir o *self-hosting* sem lhe atribuir centralidade.

Diante desse quadro, propõe-se uma comparação empírica controlada entre modelos leves de execução local (*Gemma 3 QAT*, *Qwen3*, *Granite 4.0 Tiny* e *Llama 3.2*) e um modelo comercial de referência (*Claude*), conduzida sob um único protocolo de *prompting* e sobre o mesmo conjunto de dados, de modo a avaliar simultaneamente

o desempenho de classificação, o custo operacional e as implicações de privacidade. A seleção atualiza as famílias abertas investigadas por Zhang et al. (2025), adotando sucessores mais enxutos das linhagens que naquele estudo melhor responderam à tarefa, e acrescenta um modelo sem equivalente direto na literatura analisada (*Granite 4.0 Tiny*); os critérios e a justificativa de cada modelo são detalhados no Capítulo 5.

A opção por um *prompt*-base único responde ao achado de Heiding et al. (2024), em que uma simples alteração na formulação da pergunta elevou a detecção de *phishing* sofisticado de 25% para 100% no melhor modelo avaliado. Fixar um *prompt* idêntico para todos os modelos assegura que as diferenças observadas decorram do modelo, e não da redação da instrução. O conjunto de dados, por sua vez, é construído a partir de dois *corpora* públicos e rastreáveis, dedicados a cada classe: o Nazario Phishing Corpus, para o *phishing*, e o Enron Corpus, para os *e-mails* legítimos (NAZARIO, 2005; KLIMT; YANG, 2004), cuja composição e curadoria são detalhadas no Capítulo 5. Ao adotar versões atualizadas das famílias abertas e ao posicionar a privacidade dos dados como critério central, este trabalho busca preencher a lacuna identificada e fornecer evidência empírica para decisões de implantação em organizações que precisem conciliar eficácia defensiva e soberania sobre seus próprios dados.

4 CONSTRUÇÃO DA ABORDAGEM

Este capítulo apresenta a abordagem construída para responder à questão central do trabalho: comparar, sob condições idênticas, um modelo comercial de grande porte e modelos abertos menores na detecção de *e-mails* de *phishing*. A fundamentação dos Capítulos 2 e 3 dá o embasamento; neste capítulo o trabalho passa do problema à proposta de solução. O foco recai sobre o que foi decidido no projeto e por que, ficando o detalhamento da execução para o Capítulo 5.

A Seção 4.1 classifica a pesquisa quanto à natureza, à abordagem, aos objetivos e aos procedimentos técnicos. A Seção 4.2 descreve a arquitetura da ferramenta de *benchmark* e a decisão de projeto que isola o modelo como única variável manipulada. A Seção 4.3 justifica as escolhas tecnológicas centrais, da linguagem ao ambiente de execução local. A Seção 4.4 percorre o fluxo de funcionamento da aplicação, do carregamento do *dataset* à consolidação das métricas.

4.1 CARACTERIZAÇÃO DA PESQUISA

Esta pesquisa é classificada sob quatro aspectos: sua natureza, a abordagem do problema, os objetivos e os procedimentos técnicos (PRODANOV; FREITAS, 2013).

Quanto à natureza, trata-se de pesquisa aplicada, pois gera conhecimento de uso prático imediato, voltado a apoiar as equipes de segurança da informação na escolha, com base em evidências, entre modelos comerciais e modelos abertos para a triagem de *e-mails* suspeitos de *phishing* (PRODANOV; FREITAS, 2013).

Quanto à abordagem, é quantitativa, já que o desempenho dos modelos é medido com métricas estatísticas de classificação e indicadores operacionais, o que permite compará-los numericamente (PRODANOV; FREITAS, 2013).

Quanto aos objetivos, é ao mesmo tempo descritiva e comparativa. Descritiva porque caracteriza o desempenho de cada modelo; comparativa porque o foco está em contrastar modelos de arquiteturas, tamanhos e formas de execução distintas, principalmente um modelo comercial de grande porte, acessado via *API*, frente a modelos abertos menores, executados localmente (PRODANOV; FREITAS, 2013).

Quanto aos procedimentos técnicos, é experimental, pois manipula de forma controlada uma variável independente, o modelo de linguagem, sobre um mesmo conjunto de dados e observa seus efeitos nas métricas de saída, com os demais fatores fixados pelas variáveis controladas descritas na Seção 5.1 (PRODANOV; FREITAS, 2013).

4.2 ARQUITETURA DA SOLUÇÃO

A solução construída é uma aplicação de *benchmark*, isto é, uma ferramenta de orquestração que submete cada *e-mail* do *dataset* a cada modelo com um único *prompt* padronizado (Seção 5.2.3) e coleta a classificação e as métricas operacionais. O objetivo é comparar os modelos sob condições idênticas, e não entregar um sistema de produção. Dessa forma, o código privilegia o controle do experimento, escalabilidade e a reprodutibilidade, em vez de robustez e facilidade de uso (SCHEIDT, 2026). Deu-se destaque a escalabilidade para que esta mesma *pipeline* possa ser utilizada por possíveis trabalhos futuros, utilizando mais modelos e outros *datasets* (SCHEIDT, 2026).

A Figura 3 resume a aplicação em quatro grupos de componentes. O primeiro é a fonte de dados, com o *dataset* de *e-mails* rotulados em *CSV*. O segundo é a camada de orquestração, que carrega os dados, percorre modelos e mensagens e monta os *prompts*. O terceiro reúne os provedores de inferência, a saber, a *API* da Anthropic para o Claude e o *runtime* local Docker Model Runner para os modelos abertos (Docker Inc., 2026). O quarto é a coleta de resultados, que grava classificação, latência e *tokens* em um relatório comparativo (SCHEIDT, 2026).

A decisão central de projeto consiste em isolar a variável independente, de modo que apenas o modelo varie entre as execuções. Para tanto, a orquestração interage com uma única interface de classificação, atendida por dois clientes intercambiáveis, um para cada provedor de inferência, que expõem o mesmo método e devolvem a mesma estrutura. A troca de modelo não altera o restante do código. Essa organização também sustenta o argumento de privacidade, pois os modelos abertos são executados integralmente na máquina local e nenhum *e-mail* deixa o ambiente do usuário; somente o Claude, tomado como referência, envia as mensagens a um serviço externo.

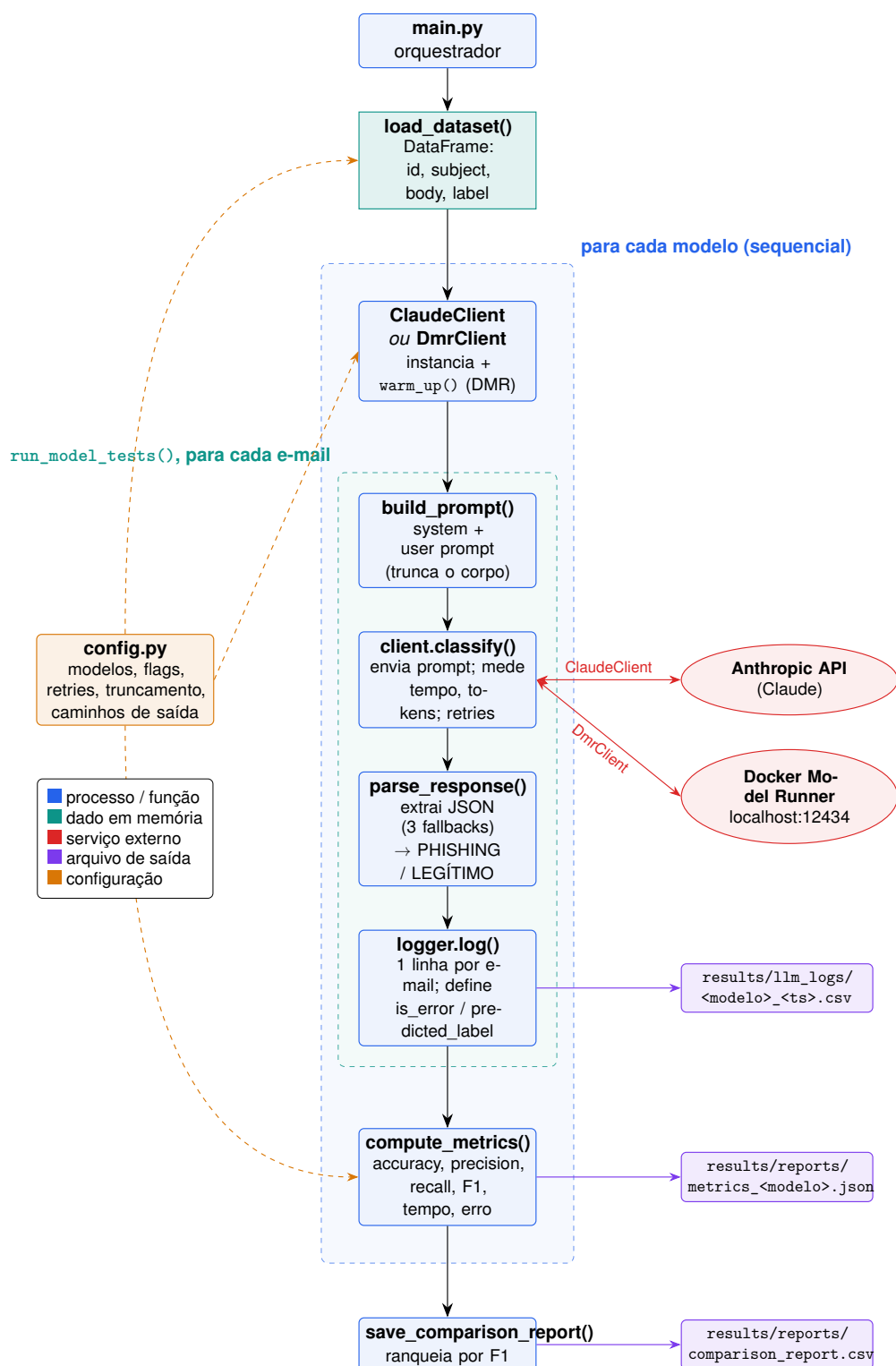


Figura 3 – Visão geral da arquitetura da solução.

Fonte: Elaborado pelo autor.

4.3 TECNOLOGIAS EMPREGADAS

A ferramenta de *benchmark* apoia-se em três escolhas tecnológicas centrais: a linguagem de programação, o ambiente de execução dos modelos locais e a biblioteca de acesso aos modelos. As subseções a seguir justificam cada uma delas. A configuração detalhada e o uso concreto são tratados no Capítulo 5; nesta seção são tratadas as justificativas de cada decisão.

4.3.1 Linguagem de Programação

A aplicação foi escrita em Python (Python Software Foundation, 2026), pela maturidade de seu ecossistema para trabalhos com modelos de linguagem e ciência de dados. Os provedores de inferência adotados neste trabalho disponibilizam bibliotecas oficiais para a linguagem, o que facilita a integração e reduz o risco de incompatibilidades. O cálculo das métricas também é direto, pois bibliotecas consolidadas como o *scikit-learn* (PEDREGOSA et al., 2011) implementam precisão, revocação, F1-Score e matriz de confusão, o que dispensa implementação manual. Adicionalmente, o suporte nativo a *JSON*, formato exigido na saída dos modelos (Seção 5.2.3), simplifica o *parsing* das respostas.

4.3.2 Containerização e Execução Local

Os quatro modelos abertos executam localmente, e não por um serviço remoto, sendo esta a decisão que sustenta na prática o argumento de privacidade que motiva o trabalho. Quando o modelo é executado na máquina do usuário, nenhum *e-mail* precisa ser enviado a terceiros, e o risco de exposição de dados pessoais discutido na Seção 2.3.4 deixa de se aplicar. Manter o processamento no ambiente local é também a postura mais alinhada à LGPD (BRASIL, 2018) e ao GDPR (European Parliament; Council of the European Union, 2016), que regulam a transferência e o tratamento desses dados.

A execução local foi feita com o *Docker Model Runner*, componente do Docker voltado à execução de modelos de linguagem na própria máquina (Docker Inc., 2026). A escolha traz dois ganhos para o experimento. O primeiro é a reprodutibilidade, pois o *runtime* padroniza o carregamento dos pesos, a quantização e a interface de inferência, de modo que a mesma configuração se repete a cada execução e entre máquinas diferentes. O segundo é o isolamento, já que o ambiente fica contido e versionado, sem depender de instalações manuais sujeitas a variação. Esse isolamento ainda reforça a privacidade, pois, por padrão, os modelos são executados sem acesso à internet, e os *e-mails* processados não têm como deixar a máquina, mesmo caso um modelo o tente (Docker Inc., 2026). O *Docker Model Runner* também administra a memória de vídeo de forma econômica, mantendo um único modelo carregado por

vez e substituindo-o sob demanda, o que se ajusta à restrição de *hardware* assumida no trabalho. A configuração exata de *hardware* e *software* dos experimentos está consolidada no Quadro 5 (Seção 5.2.2.4).

4.3.3 Biblioteca de Acesso aos Modelos

O acesso aos modelos apoia-se na *API* no estilo da OpenAI, convenção amplamente adotada para chamadas a modelos locais. O *Docker Model Runner* expõe um *endpoint* compatível com essa *API* (Docker Inc., 2026), o que permite empregar a biblioteca oficial `openai`, em Python, como cliente único para os quatro modelos locais. Como todos atendem pelo mesmo *endpoint*, basta trocar a *tag* do modelo para alternar entre eles, sem alterar o restante do código. Dessa forma, a inclusão de um novo modelo local resume-se a adicionar a *tag* correspondente no arquivo de configuração e a baixar o modelo pela interface do *Docker Model Runner* (SCHEIDT, 2026).

A padronização pela *API* da OpenAI traz outra vantagem além da uniformidade. Por ser uma convenção amplamente adotada, o mesmo cliente pode ser reapontado, via `base_url`, para outros provedores que a sigam, locais ou remotos, sem reescrever a camada de acesso. O Claude constitui a exceção, acessado pela biblioteca oficial da Anthropic, de interface própria. Para que essa diferença não se propague pelo código, os dois clientes implementam a mesma interface de classificação apresentada na Seção 4.2, com o mesmo método e a mesma estrutura de retorno. Assim, a orquestração opera sobre um contrato único, e a troca de modelo, mesmo entre provedores distintos, não altera a lógica do *benchmark*.

4.4 FUNCIONAMENTO DA APLICAÇÃO

O fluxo de execução da aplicação se resume a dois laços encaixados: um laço externo que percorre os modelos e um laço interno que percorre os *e-mails* do *dataset*. O Algoritmo 1 descreve essa lógica. O *prompt*, o *dataset*, os parâmetros de inferência e a forma de medir permanecem fixos, tendo como única variável manipulada o modelo de linguagem, o que sustenta a comparação sob condições idênticas. O código completo da aplicação está disponível no repositório do trabalho (SCHEIDT, 2026).

Algoritmo 1: Laço de avaliação dos modelos sobre o dataset.

Input: conjunto de modelos $M = \{\text{Claude}\} \cup \text{modelos locais}$, *prompt* padronizado P , caminho do *dataset* C_D

Output: relatório comparativo de métricas, ordenado por F1-Score

```

1  $D \leftarrow \text{carregarDataset}(C_D)$ 
2  $Metricas \leftarrow \emptyset$ 
3 foreach  $modelo \in M$  do
4    $cliente \leftarrow \text{criarCliente}(modelo)$ 
5   if  $modelo$  é local then
6      $\text{preAquecer}(cliente)$  // carrega o modelo em memória, serve para
        que o primeiro teste de um modelo local não sofra o impacto
        do carregamento na latência
7   end
8    $R \leftarrow \emptyset$ 
9   foreach  $email \in D$  do
10     $(sistema, usuario) \leftarrow \text{montarPrompt}(P, email)$ 
11     $resposta \leftarrow cliente.classificar(sistema, usuario)$ 
12     $(predicao, houveCorrecao, truncada) \leftarrow \text{parsearResposta}(resposta)$ 
13    if  $truncada$  then
14       $resposta \leftarrow cliente.classificar(sistema, usuario, limiteAmpliado)$ 
        // re-envia com limite maior de tokens de resposta para
        tentar resgatar a predição truncada
15       $(predicao, houveCorrecao, truncada) \leftarrow \text{parsearResposta}(resposta)$ 
16    end
17     $R_{\text{parcial}} \leftarrow$ 
         $\{(email.rotulo, predicao, resposta.tempo, houveCorrecao, resposta.tokens)\}$ 
18     $\text{salvarNoLog}(modelo.nome, R_{\text{parcial}})$ 
19     $R \leftarrow R \cup R_{\text{parcial}}$ 
20  end
21   $Metricas \leftarrow Metricas \cup \{\text{calcularMetricas}(R)\}$ 
22 end
23  $\text{salvarRelatorioComparativo}(Metricas)$ 

```

A execução inicia-se com o carregamento do *dataset*, realizado uma única vez. Cada linha do CSV é convertida em um registro composto por identificador, remetente, assunto, corpo e rótulo verdadeiro, no qual 1 indica *phishing* e 0 indica mensagem legítima. Esse mesmo conjunto é reaproveitado por todos os modelos, de modo a assegurar que a avaliação recaia exatamente sobre as mesmas mensagens.

O laço externo processa um modelo por vez, para cada um, instancia-se o cliente

adequado, o da *API* da Anthropic para o Claude ou o do Docker Model Runner para os modelos abertos. Como ambos os clientes expõem a mesma interface de classificação, o restante do laço não se altera conforme o modelo. Os modelos são testados em sequência, pois o *Docker Model Runner* mantém apenas um modelo carregado em memória por vez e realiza a troca sob demanda, sem ocupar recursos desnecessários (Docker Inc., 2026). Antes do laço interno de um modelo local, executa-se uma rotina de pré-aquecimento, que força o carregamento do modelo com uma requisição inicial. Sem essa etapa, o primeiro *e-mail* de cada modelo arcaria com o custo de carregar os pesos, o que contaminaria a medição de latência (Docker Inc., 2026; SCHEIDT, 2026).

O laço interno percorre os *e-mails*. Para cada mensagem, a rotina de montagem combina o *prompt* padronizado com o remetente, o assunto e o corpo, e devolve as partes de sistema e de usuário. O corpo é truncado quando excede um limite de caracteres, providência que protege os modelos menores, propensos a quebrar o formato de saída diante de entradas muito longas. Em seguida, o cliente classifica a mensagem e retorna a resposta bruta acompanhada do tempo decorrido e da contagem de *tokens*. Os parâmetros de inferência são idênticos para todos os modelos: temperatura igual a 0 e top-p igual a 1, fixados para reduzir a aleatoriedade da geração e favorecer a reprodutibilidade, conforme detalhado na Seção 5.2.3.

A resposta bruta passa por uma etapa de *parsing* defensivo. Espera-se que o modelo responda com um único objeto *JSON* contendo a classificação, mas os modelos pequenos nem sempre cumprem integralmente o formato. O *parser* extrai o *JSON* ainda que ele venha cercado por texto explicativo ou por marcações de bloco de código.

Do *JSON* extraído, a predição final não provém do campo textual *classification*, e sim da probabilidade estimada que o *prompt* também solicita (Seção 5.2.3). A mensagem é tratada como *phishing* quando essa probabilidade (*phishing_likelihood*) atinge um limiar ou o supera, e como legítima abaixo desse valor, o valor específico do limiar é discutido na Seção 5.2.4.2. O campo textual permanece na resposta e auxilia a auditoria da decisão, porém não define o rótulo. Esta escolha corrige uma incoerência observada em alguns modelos, com destaque para o *Gemma 3*, que por vezes marca um e-mail como *phishing* no campo textual, mas lhe atribui probabilidade baixa, contradizendo a própria saída. Acatar somente o campo textual elevaria o número de falsos positivos.

Parte das respostas chega como um *JSON* parcialmente malformatado, truncado por exceder o limite de *tokens* ou com aspas não escapadas no interior de uma string. Para esses casos, o *parser* realiza uma última tentativa de reparo heurístico: fecha chaves (*{ }*) ou colchetes (*[]*) abertos, escapa as aspas órfãs ¹ e recupera a predição que de outro modo se perderia. Uma resposta legível apenas após o reparo recebe uma marca

¹ eventualmente um modelo insere aspas duplas dentro do texto, e o formato *JSON* as interpreta como fechamento de uma *string*, o que quebra a formatação esperada.

de correção, representada no Algoritmo 1 como `houveCorrecao`. A predição resgatada é preservada, mas, como a saída bruta não constituía um *JSON* válido, a ocorrência também é contabilizada como erro de formatação. Somente quando nem o reparo recupera a resposta, ou a chamada de fato falha, a predição recebe um valor reservado e a mensagem é registrada como erro irrecoverável, sem interromper a execução.

O reparo automático cobre os casos mais frequentes, mas não a totalidade deles. Por isso, encerrada a coleta, as respostas ainda marcadas como erro passaram por revisão manual. O objetivo foi resgatar a predição sempre que ela estivesse legível na saída do modelo, mesmo em formatos não previstos pelo reparo heurístico, e manter como erro irrecoverável apenas o que de fato não trazia classificação aproveitável. Dessa forma, nenhuma predição recuperável é descartada por uma limitação do reparo automático, e a contagem de erro de formato reflete somente o que o modelo realmente deixou de entregar. Em alguns modelos, a resposta foi truncada pelo limite de *tokens* de saída; nesses casos, os *e-mails* foram reenviados com um limite maior, na tentativa de resgatar a predição. Persistindo o truncamento, o caso foi mantido como erro irrecoverável.

Marcar a correção, em vez de descartar a resposta, permite registrar com que frequência cada modelo quebra o formato solicitado sem perder a classificação efetivamente produzida. Os modelos menores tendem a falhar mais no formato, e contabilizar essas falhas à parte mantém a taxa de erro de formato separada do desempenho de classificação.

Ao terminar o laço interno de um modelo, a aplicação calcula as métricas a partir dos rótulos verdadeiros e das predições. A classe *phishing* é tomada como classe positiva. São computadas acurácia, precisão, revocação e *F1-Score*, taxa de falsos positivos (*FPR*) e taxa de falsos negativos (*FNR*), além da taxa de erro de formato, da contagem de *tokens* (de entrada e saída) e do tempo médio de resposta. As métricas de classificação são calculadas apenas sobre as respostas válidas, incluídas as recuperadas pelo reparo descrito anteriormente, já que essa correção recompõe o formato sem alterar a decisão do modelo. As métricas de cada modelo são acumuladas e, ao final, consolidadas em um relatório comparativo ordenado pelo *F1-Score*. Os resultados são apresentados no Capítulo 6.

5 METODOLOGIA EXPERIMENTAL

O Capítulo 4 caracterizou a pesquisa quanto à natureza, à abordagem, aos objetivos e aos procedimentos. Este capítulo, por sua vez, trata da parte prática, ao descrever como o experimento foi desenhado e conduzido, do estabelecimento das variáveis à coleta das métricas.

A pergunta que orienta o trabalho é comparativa. Para que seja respondida com rigor, é necessário que a única diferença entre as condições testadas seja o modelo de linguagem, mantendo-se constante todo o restante. A Seção 5.1 estabelece esse desenho, ao separar o que varia, o que é medido e o que permanece fixo. Em seguida, a Seção 5.2 percorre as etapas que vão da preparação dos dados à coleta dos resultados, detalhando cada decisão prática.

5.1 VARIÁVEIS DO EXPERIMENTO

O experimento envolve três tipos de variáveis: as independentes, que são manipuladas para criar as condições comparadas; as dependentes, que são medidas em resposta a essa manipulação; e as controladas, que são mantidas fixas para garantir que as diferenças observadas venham de fato das variáveis independentes.

A variável independente é o modelo de linguagem, com cinco níveis:

- *Claude Haiku 4.5 (Anthropic)*, acessado via *API* e usado como *baseline* de referência por representar a categoria comercial de grande porte;
- *Gemma 3 QAT 4B (Google)*, executado localmente via *Docker Model Runner*;
- *Qwen3-4B (Alibaba)*, executado localmente via *Docker Model Runner*;
- *Granite 4.0 Tiny (IBM)*, executado localmente via *Docker Model Runner*;
- *Llama 3.2 3B (Meta)*, executado localmente via *Docker Model Runner*.

A técnica de *prompting* não é manipulada: adota-se exclusivamente o *prompting zero-shot*, em que o modelo recebe apenas a descrição da tarefa, sem exemplos rotulados. A escolha acompanha a literatura comparativa de detecção de *phishing* com *LLMs*, na qual o *zero-shot* com saída estruturada é a configuração predominante, inclusive nos três trabalhos correlatos selecionados no Capítulo 3 (ZHANG et al., 2025; CHATAUT; GYAWALI; USMAN, 2024; HEIDING et al., 2024). Com isso, o experimento reduz-se a cinco condições, uma por modelo avaliado.

As variáveis dependentes são as métricas registradas a cada inferência, divididas em dois grupos. As métricas de classificação medem a qualidade das decisões do

modelo: precisão, revocação, *F1-Score*, acurácia, taxa de falsos positivos e taxa de falsos negativos, além da matriz de confusão. As métricas operacionais medem o custo de execução: latência por inferência, resumida pela média e pelo percentil 95, contagem de *tokens* de entrada e saída e custo estimado por inferência. Esse custo é medido em valor monetário no caso do modelo comercial e em consumo de memória de vídeo (*VRAM*) e de sistema (*RAM*) e tempo de processamento no caso dos modelos locais. Essa separação é relevante para o trabalho, pois um modelo só constitui alternativa viável se aliar boa qualidade de classificação a um custo de execução sustentável.

A essas se soma a taxa de erro de formato, que mede com que frequência o modelo deixa de entregar uma resposta no formato *JSON* exigido. Essa métrica é pertinente porque os modelos menores tendem a violar o formato, e uma resposta mal formada equivale, na prática, a uma classificação perdida. As métricas de classificação são calculadas sobre as respostas válidas, incluídas as recuperadas pelo reparo heurístico descrito na Seção 4.4, que apenas recompõe o formato, sem alterar a decisão do modelo; a taxa de erro de formato é registrada à parte, por modelo.

São mantidas fixas em todas as execuções, para isolar o efeito das variáveis independentes, as seguintes variáveis controladas:

1. o *dataset*, em sua composição e ordem de submissão;
2. o *prompt*-base, idêntico para todos os modelos, de modo que a única diferença entre as condições seja o modelo avaliado;
3. os parâmetros de decodificação: temperatura fixada em 0 (ou no menor valor permitido pelo modelo) e top-p em 1, com o objetivo de maximizar a reprodutibilidade ao reduzir a variabilidade estocástica da geração;
4. o formato de saída exigido (*JSON* estruturado, conforme detalhado na Seção 5.2.3);
5. o ambiente de execução dos modelos locais, com mesma máquina e mesma versão da plataforma de execução (*Docker Model Runner*), e o artefato de cada modelo fixado por *tag* e *digest* (Apêndice B). A quantização não é uniformizada entre os modelos, pois é característica do artefato distribuído e integra, portanto, a condição experimental de cada um;
6. a versão exata do modelo comercial Claude, registrada no momento dos experimentos, dado que modelos proprietários sofrem atualizações silenciosas que podem afetar a reprodutibilidade.

5.2 ETAPAS METODOLÓGICAS

O experimento avança em quatro etapas encadeadas. A primeira trata do *dataset* de e-mails. A segunda seleciona e configura os modelos avaliados. Na terceira, define-se o *prompt*-base comum a todos eles. A quarta submete os dados aos modelos e organiza a coleta das métricas. As subseções a seguir detalham cada uma.

5.2.1 Escolha e Preparo do *Dataset*

A primeira etapa prepara os dados que alimentam o experimento. Optou-se por uma base pública já existente, e não por montar um conjunto próprio. Essa decisão apoia-se em três motivos: aproveita material já rotulado, o que afasta o erro de uma rotulação manual; facilita a comparação com a literatura, que recorre a bases semelhantes (SIVANESWARAN et al., 2026); e favorece a reprodutibilidade, já que qualquer pessoa pode obter os mesmos dados de origem. Notar-se-á que essa escolha implica o risco discutido na Seção 2.3.3, a contaminação com dados de treinamento dos modelos. Modelos de linguagem são treinados sobre grandes volumes de dados (XIAO; ZHU, 2025), e é provável que parte desse material inclua *e-mails* deste *dataset* (MAGAR; SCHWARTZ, 2022; SAINZ et al., 2023), principalmente no caso do Claude, treinado com um volume de dados muito superior. Como a contaminação se manifesta com mais força em modelos maiores (CARLINI et al., 2022), seu efeito apenas reforçaria o que já se espera: um bom desempenho do Claude na detecção de *phishing*, o que torna o teste ainda mais exigente para os modelos locais.

5.2.1.1 Fonte e Composição

A primeira opção considerada foi o conjunto público *Phishing Email Dataset*, disponível no Kaggle (ALAM, 2024). Seu atrativo era a diversidade de origens, pois a base reúne seis *corpora* clássicos de e-mail em um só conjunto (*Enron*, *Ling*, *CEAS*, *Nazario*, *Nigerian Fraud* e *SpamAssassin*). Uma inspeção mais atenta das amostras, porém, revelou um problema que inviabilizou seu uso: embora a base rotule esses *e-mails* como *phishing*, boa parte deles trata na realidade de *spam* comum, como propaganda não solicitada, golpes genéricos e correntes, e não de *phishing* no sentido estrito. O Quadro 3 traz um caso dessa rotulação equivocada.

Campo	Conteúdo
Remetente (<i>from</i>)	Jonas Romero <dwteasm@teas.sk>
Assunto (<i>subject</i>)	Money back guarantee to ensure results!
Corpo (<i>body</i>)	Let me describe you how to quit smoking in a healthy way. Order herbal product LiveFree anti-smoking patch that has helped thousands of people to forget about this habit. Guaranteed results !
Rótulo na base (<i>label</i>)	<i>phishing</i>

Fonte: Elaborado pelo autor, a partir de (ALAM, 2024).

Quadro 3: Exemplo de e-mail rotulado como *phishing* na base, mas que constitui *spam*.

O motivo é que, apesar do nome dado ao *dataset* (*Phishing Email Dataset: Phish No More*), as bases das quais ele deriva, como CEAS_08, Ling, Enron e SpamAssassin, foram montadas em sua maioria para pesquisa de filtragem de *spam*, e tratam de *spam*, não de *phishing*. Avaliar um detector de *phishing* sobre esse material mediria, na prática, a separação entre *spam* e e-mail legítimo, o que desvia da questão central deste trabalho.

Optou-se, então, por montar a base a partir de fontes dedicadas a cada classe. O *phishing* provém do Nazario Phishing Corpus (NAZARIO, 2005), coleção reconhecida, composta exclusivamente de *e-mails* de *phishing* reais e atualizada ano a ano. Aproveitaram-se os recortes mais recentes, de 2024 e 2025, com 403 e 481 *e-mails* respectivamente. Como os dois somam menos do que a meta de mil *e-mails* da classe, a coleta foi completada com um conjunto público no Kaggle que reúne o *Nazario 5* e o *TREC-2007* (SOOD, 2024) (deste foram utilizados somente os do *Nazario 5*), ainda dentro do mesmo universo de *phishing* real. De outra forma, os e-mails legítimos provêm do Enron Corpus (KLIMT; YANG, 2004), amplo acervo de mensagens corporativas reais, muito usado como fonte de e-mail legítimo na literatura. A reunião dessas fontes garante que cada classe corresponda de fato ao fenômeno que este trabalho pretende medir. Todas são bases públicas e já rotuladas pela própria origem, o que preserva as vantagens de reaproveitar material existente e dispensa qualquer rotulação manual: o rótulo de cada e-mail decorre da fonte de onde foi extraído, *phishing* para o Nazario e para o complemento do Kaggle, legítimo para o Enron.

5.2.1.2 Inclusão e Pré-processamento

Como os dois *corpora* já chegam rotulados pela própria origem, dispensou-se um trabalho extenso de limpeza. Ainda assim, aplicou-se um pré-processamento mínimo. Antes da amostragem, todo e-mail candidato passou por um filtro de qualidade semântica, que verifica se o corpo contém texto legível em quantidade suficiente para a análise por um modelo de linguagem. O filtro remove a marcação *HTML*, reduz

as sequências de espaços, quebras de linha e tabulações a um único espaço e, em seguida, descarta o e-mail em três situações: quando o texto restante é curto demais; quando resta pouca proporção de caracteres alfabéticos, indício de conteúdo codificado como blocos em *base64*; ou quando, retiradas as URLs e os endereços de e-mail, sobram poucas palavras com letras. Dessa forma, o critério elimina *HTML* vazio, corpos compostos apenas por *links*, *tokens* codificados e fragmentos minúsculos, casos em que não há mensagem real a classificar e cuja presença distorceria as métricas.

A base também foi deduplicada pelo corpo do e-mail. *Templates* de *phishing* costumam repetir o mesmo corpo sob assuntos diferentes, e mensagens idênticas inflariam artificialmente a contagem de uma classe. A remoção de duplicatas vale inclusive entre as classes, de modo que nenhum corpo apareça simultaneamente como *phishing* e como legítimo, o que evita rótulos conflitantes para o mesmo conteúdo. O Algoritmo 2 resume o filtro de qualidade e a deduplicação.

Algoritmo 2: Filtro de qualidade semântica e deduplicação por corpo.

Input: conjunto de e-mails candidatos E , cada um com corpo e rótulo

Output: base filtrada B , sem duplicatas e só com corpos analisáveis

```

1  $B \leftarrow \emptyset$ 
2  $corposVistos \leftarrow \emptyset$ 
3 foreach  $email \in E$  do
4    $texto \leftarrow \text{removeHTML}(email.corpo)$            // tira tags e normaliza os
     espaços em branco
5   if  $tamanho(texto) < MIN\_CARACTERES$  then
6     | descartar e ir ao próximo                       // fragmento minúsculo
7   end
8   if  $razãoAlfabética(texto) < MIN\_RAZÃO$  then
9     | descartar e ir ao próximo                       // conteúdo codificado
10  end
11   $palavras \leftarrow \text{palavrasComLetras}(\text{removeURLs}(texto))$ 
12  if  $tamanho(palavras) < MIN\_PALAVRAS$  then
13    | descartar e ir ao próximo                       // corpo só-URL ou vazio
14  end
15  if  $email.corpo \in corposVistos$  then
16    | descartar e ir ao próximo // duplicata, inclusive entre classes
17  end
18   $corposVistos \leftarrow corposVistos \cup \{email.corpo\}$ 
19   $B \leftarrow B \cup \{email\}$ 
20 end
```

Aplicou-se também o truncamento dos e-mails com mais de 5 mil caracteres. A medida atende a uma limitação prática: os maiores e-mails da base excedem a janela

de contexto dos modelos avaliados, e um texto acima desse limite levaria o modelo a falhar, impossibilitando a avaliação do conteúdo. O limite foi escolhido como ponto de equilíbrio, de forma a afetar apenas uma fração pequena das amostras, mantendo o conteúdo dentro da janela de contexto de todos os modelos e reduzindo o tempo de processamento. Acima desse valor os ganhos são marginais, pois cada acréscimo alcança um número cada vez menor de e-mails à medida que se aproxima do teto de contexto. O truncamento também auxilia a prevenir ataques de negação de serviço, nos quais um e-mail malicioso é deliberadamente longo para sobrecarregar o modelo, consumindo muitos *tokens* nos modelos pagos ou capacidade de processamento nos gratuitos. A Tabela 1 mostra, para diferentes limites, quantos e-mails caberiam por inteiro e quantos seriam truncados, evidenciando esses ganhos decrescentes.

Tabela 1 – E-mails preservados por inteiro e truncados, por limite de caracteres.

Caracteres	% de e-mails	Qtd.
500	40,0%	800
1000	64,2%	1284
1500	74,5%	1490
2000	81,5%	1630
2500	85,9%	1718
3000	88,9%	1778
4000	93,7%	1873
5000	96,2%	1923
6000	97,2%	1944
8000	98,5%	1970
10000	98,8%	1975
15000	99,2%	1985
20000	99,5%	1991

Fonte: Elaborado pelo autor.

Esse corte afeta pouco a classe de interesse. Por sua natureza, e-mails de *phishing* tendem a ser curtos: Jayatilaka, Arachchilage e Babar (2024) argumenta que as pessoas tendem a confiar mais em e-mails longos, embora "longo", para um ser humano, seja uma medida relativa. De fato, o truncamento recaiu principalmente sobre os e-mails legítimos, cerca de duas vezes mais frequentes nessa condição do que os de *phishing*, conforme a Tabela 2.

Tabela 2 – E-mails truncados (acima de 5000 caracteres) por classe.

Classe	Qtd. > 5000	% da própria classe
<i>Phishing</i> (1)	26	2,6% dos 1000
Legítimo (0)	51	5,1% dos 1000
Total	77	3,9% dos 2000

Fonte: Elaborado pelo autor.

Essas etapas distinguem-se de uma reescrita do conteúdo. O filtro de qualidade e a deduplicação decidem quais e-mails entram na base, e o truncamento limita o comprimento, porém nenhum deles altera o texto que permanece. O corpo dos e-mails selecionados é preservado como aparece na origem, sem normalizações adicionais nem reescrita de cabeçalhos. Pretende-se manter a entrada próxima de um e-mail real, que chega ao usuário sem tratamento prévio, e evitar que etapas de limpeza introduzam artefatos capazes de mascarar o desempenho dos modelos. Busca-se testar o modelo de linguagem sobre um *e-mail* de significado semântico real, e não sobre um texto composto majoritariamente por *tags HTML* ou sobre e-mails demasiado curtos, que não refletem o cenário real de uso. O filtro e a deduplicação apenas eliminam os casos em que o modelo não teria material suficiente para a análise semântica, ou em que a presença de duplicatas distorceria as métricas. A seguir, o Quadro 4 reúne exemplos de e-mails descartados, com o filtro responsável por cada exclusão.

Filtro	Assunto	Corpo	Explicação
Tamanho mínimo (MIN_CARACTERES = 20)	Re: lunch	Yes Bob ok go now	Possui 17 caracteres, abaixo do mínimo de 20. É curto demais para a análise do modelo, assim como outras respostas breves do tipo "ok", "thanks"ou "see attached".
Razão alfabética mínima (MIN_RAZÃO = 0,35)	tracking	Order a1 b2 c3 d4 e5 0000000000 1111111111 2222222222 ///// ++++++ =====	Possui 74 caracteres e seis "palavras", mas apenas 16% de caracteres alfabéticos, abaixo de 0,35. Descarta corpos compostos essencialmente por <i>base64</i> ou <i>quoted-printable</i> , identificadores de rastreamento e blobs numéricos, sem texto natural a interpretar.
Mínimo de palavras (MIN_PALAVRAS = 5)	Verify your account	Verify now: <hxtps://secure-login[.]example[.]com/account?id=99> <hxtp://bit[.]ly/aZ9> ^a	Possui 76 caracteres e 75% de caracteres alfabéticos, passando nos dois filtros anteriores, porém, após a remoção das URLs e dos endereços de e-mail, restam apenas duas palavras com letras ("Verify"e "now"), abaixo de cinco. Descarta os e-mails que se resumem a um link, sem conteúdo textual real.

^a Dado que o link se trata de um URL de *phishing*, ele foi desativado (*defanging*) (Inventive HQ Team, 2025).

Fonte: Elaborado pelo autor.

Quadro 4: Exemplos de e-mails descartados pelo filtro de qualidade semântica.

5.2.1.3 Amostragem e Balanceamento

A partir da base filtrada, sorteou-se uma amostra aleatória de 2 mil *e-mails*, mil de *phishing* e mil legítimos, em equilíbrio exato entre as duas classes. Adotou-se essa proporção pois, em uma base desbalanceada, a acurácia infla com a classe majoritária e oculta o desempenho real do modelo; com as classes em igual proporção, métricas como precisão, revocação e *F1-Score* tornam-se diretamente comparáveis entre os cinco modelos, conforme detalhado em 2.4.

O tamanho da amostra concilia dois requisitos. De um lado, a amostra precisa

ser grande o bastante para gerar estimativas estáveis das métricas; de outro, deve permanecer pequena o suficiente para manter o experimento viável, já que cada *e-mail* é submetido a cinco modelos, quatro deles limitados pela latência da inferência local e um pelo custo de uso da *API* comercial. O sorteio aleatório, por fim, afasta o viés de uma seleção manual.

Para assegurar a reprodutibilidade, a lista exata dos identificadores dos *e-mails* sorteados, junto com o *script* de amostragem, está versionada no repositório do trabalho (SCHEIDT, 2026). Dessa forma, a mesma amostra pode ser reconstruída sem que os IDs precisem constar no corpo do texto.

5.2.2 Seleção e Configuração dos Modelos de LLMs

A segunda etapa define quais modelos integram o experimento, por que foram escolhidos e como são configurados para a inferência. As escolhas partem dos critérios apresentados a seguir e respeitam o limite de *hardware* adotado no trabalho.

5.2.2.1 Critérios de Seleção

A seleção seguiu quatro critérios: a contagem de parâmetros, a arquitetura, a viabilidade de execução local em *hardware* acessível e a representatividade do estado da arte comercial. Os quatro modelos abertos foram escolhidos por caberem em uma placa de vídeo de baixo a médio custo (adotou-se 6 GB de *VRAM* como referência) e por cobrirem estratégias arquiteturais distintas, descritas na Seção 2.2: dois modelos densos (*Qwen3-4B* e *Llama 3.2 3B*), um modelo denso quantizado via *QAT* (*Gemma 3 QAT 4B*) e um modelo *Mixture-of-Experts* (*Granite 4.0 Tiny*). O modelo comercial entra como referência de desempenho.

Todos os modelos são avaliados em sua versão base, sem *fine-tuning*. Pretende-se medir a capacidade dos modelos tal como são entregues ao usuário comum, cenário que reflete o uso real em equipes sem infraestrutura para treino adicional. A tarefa é tratada como classificação binária (*phishing* ou legítimo), e não multiclasse, pois a decisão operacional relevante na triagem consiste apenas em separar as mensagens suspeitas das demais.

5.2.2.2 Modelos Selecionados

O modelo comercial Claude Haiku 4.5 (Anthropic) foi incluído como *baseline* de referência, pois representa a categoria de modelos proprietários de grande porte acessados via *API* externa. Estabelece-se com ele o patamar de desempenho contra o qual os modelos locais são comparados. Ademais, trata-se de um modelo amplamente empregado na literatura, como em Heiding et al. (2024), Trad e Chehab (2024), Zhang et al. (2025) e Nasution et al. (2025).

A escolha do *Haiku*, e não de um *tier* superior da mesma família, como o *Sonnet* ou o *Opus*, atende à natureza da tarefa estudada. A detecção de *phishing* é um problema de triagem de alto volume, em que cada mensagem recebida demanda uma inferência; nesse cenário, o custo por chamada à *API*, multiplicado pelo fluxo de *e-mails* de uma organização, torna-se o fator econômico determinante. O *Haiku* é o *tier* de produção economicamente plausível para esse uso, por reunir o menor custo por inferência da família a uma latência compatível com o processamento em escala. A adoção de um modelo maior elevaria o teto de desempenho da referência, mas a um custo que inviabilizaria a triagem contínua, justamente o uso que se pretende contrastar com a alternativa local. Cabe registrar que essa decisão torna a comparação mais conservadora: confrontam-se os modelos locais com o *tier* comercial mais acessível, e não com o de maior capacidade, de modo que eventuais vantagens dos modelos abertos não decorrem de uma referência comercial deliberadamente enfraquecida.

Os modelos *Gemma 3 QAT 4B* (Google) e *Granite 4.0 Tiny* (IBM) foram selecionados a partir do trabalho de Sousa (2025), que investiga a anonimização como pilar de segurança em interações com modelos de linguagem e aponta esses modelos como opções locais viáveis para cenários sensíveis à privacidade. Já que o argumento central deste trabalho também é a privacidade, a adoção de modelos já avaliados sob essa ótica reforça a coerência da proposta. Adicionalmente, o *Gemma* figura, em sua família anterior (*Gemma-2-9B*), entre os modelos abertos comparados por Zhang et al. (2025); adota-se aqui a versão sucessora *Gemma 3*¹ (YVINEC; CULLITON, 2025), o que sustenta sua escolha.

O modelo *Qwen3-4B* (Alibaba) foi selecionado a partir de Zhang et al. (2025), cujo *benchmark* demonstra que modelos abertos leves podem igualar ou superar modelos proprietários na detecção de *phishing*. Embora Zhang et al. (2025) tenha avaliado a família *Qwen2* (*Qwen2-7B*), este trabalho adota a versão sucessora *Qwen3*¹ (Qwen Team; Alibaba Cloud, 2025), de modo a manter a linhagem de modelos enquanto se atualiza para o estado da arte aberto atual.

O modelo *Llama 3.2 3B* (Meta) também foi selecionado a partir de Zhang et al. (2025), no qual o *Llama-3-8b-Instruct* obteve o melhor desempenho de todo o *benchmark*, com *F1-score* de 0,9756, à frente de sua variante maior e de modelos proprietários de grande porte, como o *GPT-4o*. Dado que o estudo aponta a linhagem *Llama* como a de melhor resultado, justifica-se sua inclusão no experimento. Adota-se a versão sucessora *Llama 3.2*, em sua variante de 3 bilhões de parâmetros, a menor da família, a fim de verificar se o desempenho se mantém sob a restrição de *hardware* deste trabalho. A escolha encontra apoio adicional em Ei e Yongsiriwit (2025), que

¹ Apesar de serem atualizações, o *Gemma 3* e o *Qwen3* têm menos parâmetros (cerca de 4 bilhões) que os menores modelos avaliados por Zhang et al. (2025) (cerca de 7 bilhões). Ainda assim, a própria conclusão daquele trabalho indica que modelos menores podem superar os maiores na detecção de *phishing*.

emprega o *Llama 3.2* na classificação de *e-mails* de *phishing*.

5.2.2.3 Restrição de *Hardware*

A seleção dos modelos abertos foi condicionada pela capacidade da placa de vídeo empregada nos experimentos. Adotou-se como referência uma GPU com 6 GB de *VRAM*, configuração de bom custo-benefício e ampla disponibilidade no mercado de consumo. Esse limiar também define o que este trabalho denomina "modelos leves", ou seja, modelos que operam em até 6 GB de *VRAM* ². A restrição é deliberada e atende ao objetivo do estudo, que é avaliar modelos passíveis de execução local por uma organização pequena, ou mesmo por um usuário individual, sem investimento elevado em *hardware*.

Três dos quatro modelos abertos respeitam esse limite com folga. O Gemma 3 QAT 4B ocupa cerca de 4 GB de *VRAM*, em razão da quantização a 4 bits, e o Granite 4.0 Tiny opera dentro da faixa de 6 GB, beneficiado por ativar apenas 1 bilhão de seus 7 bilhões de parâmetros por inferência. O Llama 3.2 3B é o mais econômico do conjunto, pois reúne um número reduzido de parâmetros à quantização a 4 bits (*Q4_0*), com ocupação aproximada de 2 a 3 GB de *VRAM*. O Qwen3-4B constitui exceção parcial: seu artefato, o único do conjunto quantizado a 8 bits, tem recomendação de 6 a 7 GB e pode exceder a *VRAM* disponível, caso em que parte da carga é alocada na RAM, ao custo de latência adicional, sem impedir a execução; o efeito desse transbordo sobre a latência medida é discutido na Seção 6.4. Modelos maiores, que demandariam placas profissionais ou múltiplas GPUs, foram descartados por contrariarem a premissa de acessibilidade que orienta a pesquisa.

5.2.2.4 Ambiente de Execução

Para garantir a reprodutibilidade, todos os modelos locais foram executados na mesma máquina e na mesma versão da plataforma de execução. Os modelos abertos rodaram via *Docker Model Runner*, que padroniza o carregamento, a quantização e a interface de inferência. O modelo comercial foi acessado via *API* externa, com a versão e a data de acesso registradas para mitigar o efeito de atualizações silenciosas. O Quadro 5 consolida a configuração de *hardware* e *software* utilizada.

² Ainda que a placa utilizada disponha de 6 GB de *VRAM*, é possível executar modelos cuja recomendação exceda esse limite, embora com impacto na latência.

Componente / Software	Especificação
GPU (VRAM)	NVIDIA GeForce RTX 2060 (placa com 6 GB de VRAM)
CPU	AMD Ryzen 5 3600 6-core Processor
Memória RAM	24 GB (3 módulos de 8 GB cada)
Armazenamento	HD WDC WD10EZEX-00WN4A0 (1 TB)
Sistema operacional	Windows 10 Home (Versão 22H2), muito embora os modelos executem em <i>containers docker</i>
Docker / Docker Model Runner	Versão 28.5.1, build e180ab8

Fonte: Elaborado pelo autor.

Quadro 5: Configuração do ambiente experimental.

O Quadro 6 reúne os cinco modelos avaliados, com o desenvolvedor, a *tag* de execução e uma breve descrição de cada um. Para os modelos locais, a *tag* fixa a versão baixada; o *digest* SHA256 de cada artefato, que identifica de forma exata a versão e torna a configuração reproduzível, é listado no Apêndice B. O modelo comercial, acessado via *API* externa não expõe esse identificador, e sua versão é registrada no momento dos experimentos.

Nome	Claude Haiku 4.5	Gemma 3 QAT 4B	Qwen3-4B	Granite 4.0 Tiny	Llama 3.2 3B
Desenvolvedor	Anthropic	Google	Alibaba	IBM	Meta
Nome + tag	claude-haiku-4.5	gemma3-qat:4B-Q4_K_M	qwen3:4B-UD-Q8_K_XL	granite-4.0-h-tiny:7B-Q4_K_M	llama3.2:3B-Q4_0
Data de download	05/06/2026 (data de requisição à API)	07/05/2026	07/05/2026	07/05/2026	07/05/2026
VRAM recomendada	n.a. (via API)	cerca de 4 GB	cerca de 6-7 GB ^a	cerca de 5-6 GB	cerca de 2-3 GB
Descrição	Modelo proprietário de grande porte, acessado via API; <i>baseline</i> comercial de referência.	Modelo denso quantizado via QAT para 4 bits, com baixo uso de VRAM.	Modelo denso da família <i>Qwen3</i> (variante <i>Instruct</i> 2507); artefato em 8 bits.	Modelo híbrido <i>Mixture-of-Experts</i> ; ativa cerca de 1 B dos 7 B de parâmetros por inferência.	Modelo denso da família <i>Llama</i> 3.2, menor variante de texto (3 B); artefato quantizado a 4 bits.
Fontes	(Anthropic, 2025)	(YVINEC; CULLITON, 2025)	(YANG et al., 2025) e (Qwen Team; Alibaba Cloud, 2025)	(SOULE; BERGMANN, 2025) e (HAN; Unsloth Team, 2025)	(GRATTAFIORI et al., 2024) e (Meta AI, 2024)

^a Apesar deste custo possivelmente ultrapassar os limites do *hardware* utilizado, este valor representa a VRAM recomendada. O modelo pode ser executado com um valor levemente menor que o recomendado, mesmo que ao custo de latência.

Fonte: Elaborado pelo autor.

Quadro 6: Modelos de linguagem avaliados no experimento.

5.2.2.5 Configuração de Inferência

Os parâmetros de inferência foram padronizados entre todos os modelos para isolar o efeito do modelo de linguagem. A temperatura foi fixada em 0 (ou no menor valor permitido por cada modelo) e o top-p em 1, reduzindo a variabilidade estocástica da geração e favorecendo a reprodutibilidade. Exigiu-se também saída em JSON estruturado, o que permite o *parsing* automatizado das respostas e classificação binária, conforme detalhado na Seção 5.2.3.

5.2.3 Protocolo de Prompting

A terceira etapa define o *prompt*-base usado na inferência, que concretiza a variável controlada apresentada na Seção 5.1. O *prompt* é único e idêntico para os cinco modelos, descreve a tarefa uma só vez e não traz exemplos rotulados, conforme a

estratégia *zero-shot* adotada no trabalho. Ele foi redigido em inglês, idioma dos e-mails do *dataset* e configuração predominante na literatura, o que evita a perda de desempenho ligada ao possível diferencial entre o idioma do *prompt* e o do conteúdo analisado (JI et al., 2025).

O desenho do *prompt* combina técnicas convergentes nos três trabalhos correlatos que avaliam *LLMs* na detecção de *phishing*. A saída é exigida em JSON, formato que Chataut, Gyawali e Usman (2024) e Zhang et al. (2025) adotam para permitir o *parsing* automatizado das respostas e uma classificação binária sem ambiguidade. O *prompt* também enumera os indicadores que o modelo deve checar: remetente incoerente com a organização que diz representar, *links* ou anexos suspeitos, linguagem de urgência ou ameaça e pedidos de dados sensíveis. Essa lista segue Chataut, Gyawali e Usman (2024) e orienta a atenção do modelo para os sinais relevantes. Heiding et al. (2024) reforçam a escolha, pois, em seus testes, pedir que o modelo procurasse ativamente sinais de suspeita elevou de forma expressiva a taxa de detecção, em comparação com uma leitura sem orientação.

Além da classificação textual, o *prompt* pede outras duas saídas. A primeira é uma estimativa de probabilidade, de 0 a 100, de o e-mail ser *phishing*. Esse grau de confiança aparece nos três trabalhos, como *confidence score* em Chataut, Gyawali e Usman (2024) e como percentual de suspeição em Heiding et al. (2024) e Zhang et al. (2025). Neste trabalho, a probabilidade assume papel central, pois é ela, e não o campo textual, que define a classificação final de cada e-mail, conforme detalhado na Seção 4.4. A segunda saída é uma justificativa curta, com as principais razões da decisão, no formato proposto por Zhang et al. (2025), o que torna a resposta auditável.

Um recurso do *prompt* de Chataut, Gyawali e Usman (2024) foi deixado de fora: o enquadramento do modelo como especialista em segurança. Hu, Rostami e Thomason (2026), em experimentos recentes ainda não publicados, questionam o uso do *persona prompting*, pois evidências preliminares sugerem que atribuir um papel ao modelo o faz soar mais como um especialista, porém degrada a qualidade efetiva de suas respostas. Além de acompanhar esse indício, deixar o *prompt* sem papéis simplifica o desenho do teste, já que remove um fator de variação que não é objeto deste trabalho. O *prompt* resultante descreve a tarefa de forma direta, sem atribuir papéis ao modelo. O Quadro 7 traz o *prompt*-base completo; os conteúdos do e-mail sob análise entram no lugar dos marcadores *from*, *subject* e *body*. Fornecer o remetente ao modelo torna verificável o primeiro indicador da lista, a coerência entre quem a mensagem diz representar e o endereço de origem, que sem esse campo não teria como ser avaliado.

```
SYSTEM_PROMPT = """You are given the full content of an email. Your task is to
decide whether the
email is a phishing attempt or a legitimate message.

Analyze the email for common phishing indicators, such as:
- Sender information that is unusual or inconsistent with the organization it
  claims to come from (for example, a display name or address that does not
  match the expected domain);
- Suspicious links or attachments, such as shortened URLs, links whose visible
  text does not match their real destination, or unexpected attachments;
- Urgent, threatening, or pressuring language meant to force a hasty action;
- Requests for sensitive information, such as passwords, financial data, or
  personal identification.

Estimate how likely the email is to be a phishing attempt
on a scale from 0 to 100, and give a brief justification, then make a decision.

Respond ONLY with a single JSON object, with no extra text, in exactly this format:

{
  "indicators": [<short indicator>", "<short indicator>", ...],
  "phishing_likelihood": <integer from 0 to 100>,
  "classification": "phishing" or "legitimate"
}"""

USER_PROMPT_TEMPLATE = """Email to analyse:

From: {from}

Subject: {subject}

Body:
{body}"""
```

Fonte: Elaborado pelo autor, com base em Chataut, Gyawali e Usman (2024), Heiding et al. (2024) e Zhang et al. (2025).

Quadro 7: *Prompt-base zero-shot* utilizado na inferência.

A composição, a fonte e o preparo do *dataset* foram descritos na Seção 5.2 (amostra balanceada entre as classes de *phishing* e legítimo, extraídas respectivamente do Nazario Phishing Corpus e do Enron Corpus) e não são retomados aqui. Para tornar concreto o que foi efetivamente submetido aos modelos, o Quadro 8 mostra os campos de um registro e um exemplo de e-mail de *phishing* da base (nominalmente, o *e-mail* de id 1393).

Campo	Conteúdo
Remetente (<i>from</i>)	"Lockwood, Frederick (Rick)<flockwood@coastline.edu>
Assunto (<i>subject</i>)	Re: WARNING
Corpo (<i>body</i>)	Dear User, Your E-mailbox password will soon expire. To keep your password active. Click Here to update Cheers! ITS HelpCentre ©2018 Web Administrator. All rights reserved. Dear User, Your E-mailbox password will soon expire. To keep your password active. Click Here to update Cheers! ITS HelpCentre ©2018 Web Administrator. All rights reserved.

Fonte: Elaborado pelo autor, a partir de (NAZARIO, 2005).

Quadro 8: Campos de um registro e exemplo de entrada do *dataset*.

5.2.4 Execução dos Experimentos e Análise

A submissão dos *e-mails* aos modelos é conduzida por uma *pipeline* programática, escrito em Python e versionado em repositório (SCHEIDT, 2026). Essa opção difere da adotada em parte dos trabalhos correlatos: Chataut, Gyawali e Usman (2024) e Heiding et al. (2024) interagiram com os modelos por interfaces de *chat*, e Zhang et al. (2025) recorreu a executores prontos, como o Ollama e o *text-generation-webui*. Conduzir a avaliação por código confere maior controle sobre o experimento e favorece a reprodutibilidade, pois cada etapa, do envio do *prompt* ao registro das métricas, fica explícita e repetível, sem depender de ações manuais em uma interface.

5.2.4.1 Protocolo de Execução

A execução segue o desenho de variável única descrito na Seção 5.1: cada *e-mail* da amostra passa pelos cinco modelos sob as mesmas condições, alterando-se apenas o modelo avaliado. Os modelos são percorridos em sequência, um de cada vez, pois o *Docker Model Runner* mantém apenas um modelo carregado por vez (Seção 4.3.2). Para cada inferência, o *pipeline* registra a resposta bruta do modelo, a predição dela derivada, a latência medida e a contagem de *tokens* de entrada e de saída, além de uma marca que sinaliza se a resposta precisou de reparo. O fluxo completo, do carregamento do *dataset* à consolidação das métricas, está descrito no Algoritmo 1 (Seção 4.4).

O registro é salvo de forma incremental, isto é, cada resultado é gravado no *log* assim que a inferência termina, e não apenas ao fim do laço. Esse procedimento protege a coleta contra falhas no meio da execução, como uma queda de conexão com a *API* ou um erro pontual durante a execução de um modelo local, que de outro modo poderiam descartar horas de processamento, e ainda permite retomar a execução a partir do último *e-mail* registrado no *log*. Antes de iniciar cada modelo local, uma rotina

de pré-aquecimento força o carregamento dos pesos com uma requisição inicial, de modo que o custo de carga não contamine a medição de latência do primeiro *e-mail* (Seção 4.4).

5.2.4.2 Plano de Análise

Concluída a submissão, a análise parte dos rótulos verdadeiros e das predições registradas. Para cada condição calculam-se as métricas de classificação definidas no Capítulo 2 (Equações 2.1 a 2.6): acurácia, precisão, revocação, *F1-Score*, taxa de falsos positivos e taxa de falsos negativos, com o *phishing* tomado como classe positiva. Cada métrica é calculada sobre as respostas válidas de cada condição, na forma estabelecida na Seção 5.1. Os casos que o reparo não recuperou são identificados individualmente em cada modelo, pois o exame de sua saída bruta pode evidenciar limitações pontuais dos modelos mais leves, como a dificuldade diante de conteúdo fora do inglês. A esse conjunto somam-se as métricas operacionais: a latência por inferência, resumida pela média e pelo percentil 95 (Seção 2.4), e o custo por inferência, monetário no caso do Claude e em *VRAM*, *RAM* e tempo no caso dos modelos locais. Reporta-se ainda o coeficiente de correlação de *Matthews* (MCC, Equação 2.7) como medida-síntese complementar, pois, ao incorporar as quatro células da matriz de confusão, é mais sensível ao desbalanceamento entre as classes que o *F1-Score*.

A leitura comparativa adota o *F1-Score* como métrica principal, pela razão exposta no Capítulo 2: ele resume precisão e revocação em um único número resistente ao efeito de classes desbalanceadas. As demais métricas entram como complemento, pois organizações com prioridades distintas podem pesar de formas diferentes os falsos positivos e os falsos negativos.

Além da classificação binária, o *prompt* pede ao modelo um *phishing_likelihood*, um inteiro de 0 a 100 que mede o quanto ele julga a mensagem suspeita (Seção 5.2.3). Ter, ao mesmo tempo, o rótulo direto e a pontuação contínua permite avaliar a decisão sob três estratégias. A primeira confia apenas no campo de classificação, isto é, usa o rótulo que o próprio modelo escreveu. A segunda aplica um limiar fixo sobre a pontuação, classificando como *phishing* toda mensagem com *likelihood* maior ou igual a 50 e tratando o meio da escala como ponto neutro. A terceira decorre de uma análise do ponto de corte, que varre todos os limiares possíveis e adota aquele que maximiza o *F1-Score* na amostra, revelando o melhor desempenho que o modelo alcançaria se o corte fosse ajustado à sua escala. Cada estratégia equivale a um ponto no plano ROC, descrito pela taxa de verdadeiros positivos e pela taxa de falsos positivos (Equações 2.3 e 2.4), e os três pontos são reunidos em uma tabela por modelo. Confrontá-los mede a coerência da decisão do modelo com a própria pontuação e a sua calibração, ou seja, o quanto o valor 50 já está próximo do corte ideal. Registra-se que o ponto de corte ótimo é estimado sobre a mesma amostra em que é avaliado, de modo que indica o

potencial do modelo, e não uma garantia transferível a dados novos. Para dimensionar o sobreajuste decorrente dessa estimativa, o corte é submetido a uma validação por partição: a amostra é dividida aleatoriamente em 70% para a estimativa do limiar e 30% para a avaliação, e o F1-Score obtido com o limiar aprendido na primeira partição é confrontado entre as duas. O resultado dessa verificação acompanha a análise comparativa no Capítulo 6.

Para sustentar as afirmações comparativas, sem apoiá-las apenas na diferença numérica bruta entre os modelos, aplica-se o teste de McNemar (DIETTERICH, 1998). Como todos os modelos classificam exatamente os mesmos e-mails, suas predições são pareadas, o que torna o teste adequado: ele compara dois classificadores sobre um único conjunto de teste, considerando apenas os casos em que um acerta e o outro erra. O teste é aplicado entre o Claude, tomado como *baseline*, e cada modelo local, para verificar se a diferença de desempenho observada é estatisticamente significativa ou se cabe dentro da variação esperada.

O teste de McNemar atesta a existência da diferença, mas não a sua magnitude. Para quantificá-la, aplica-se um *bootstrap* pareado, que reamostra com reposição os *e-mails* da amostra e, a cada reamostragem, recalcula a diferença entre cada modelo local e o Claude no F1-Score, na AUC e no MCC, métricas definidas no Capítulo 2. Por operar sobre as mesmas predições pareadas, o procedimento estima a incerteza dessas diferenças, na forma de intervalos de confiança, sem supor uma distribuição teórica para as métricas. Os resultados de ambas as análises são apresentados no Capítulo 6.

6 ANÁLISE DOS RESULTADOS

Este capítulo apresenta e discute os resultados obtidos com a aplicação do protocolo experimental descrito no Capítulo 5. A exposição parte de uma visão geral, desce ao exame individual de cada modelo e retorna ao plano comparativo: a Seção 6.1 consolida as métricas de classificação das cinco condições em uma visão de conjunto; a Seção 6.2 examina cada modelo separadamente, com sua matriz de confusão e seu padrão de erro característico; a Seção 6.3 confronta os modelos entre si, tendo o *F1-Score* como eixo e o teste de McNemar (DIETTERICH, 1998) como apoio; a Seção 6.4 trata das métricas operacionais; e a Seção 6.5 discute os achados à luz da questão de pesquisa, integrando desempenho, custo, segurança e limitações.

6.1 RESULTADOS DE CLASSIFICAÇÃO

A Tabela 3 reúne as métricas iniciais de classificação das cinco condições e oferece a visão de conjunto a partir da qual os modelos são detalhados individualmente na Seção 6.2 e comparados na Seção 6.3.

Tabela 3 – Métricas de classificação por modelo (com *limiar* ≥ 50 fixo).

Modelo	F1	Prec.	Rev.	Acur.	FPR.	FNR.	Erros totais	Erros irrep.
Claude Haiku 4.5	92,67%	86,88%	99,30%	92,15%	15,00%	0,70%	0	0
Llama 3.2 3B	89,05%	82,39%	96,90%	88,12%	20,60%	3,11%	8	3
Qwen3-4B	81,98%	69,86%	99,20%	78,20%	42,80%	0,80%	2	0
Granite 4.0 Tiny	78,55%	66,28%	96,40%	73,67%	49,05%	3,60%	12	2
Gemma 3 QAT 4B	71,74%	56,19%	99,20%	60,96%	77,20%	0,80%	12	2

Fonte: Elaborado pelo autor.

Os valores reportados correspondem a um limiar fixo de decisão de 50 e foram calculados sobre as amostras com saída válida em cada condição. As saídas com JSON malformatado passaram por uma rotina de reparo, cujo efeito sobre as métricas foi desprezível (inferior a 0,7% no caso mais expressivo); a taxa de erro de formato de cada modelo é registrada na Seção 6.2 apenas a título de documentação. A Tabela 3 fornece, portanto, a visão geral do desempenho de classificação, embora não reflita o potencial máximo de cada modelo, alcançável com o ajuste do limiar de decisão. Para cada modelo, a Seção 6.2 apresenta a tabela ROC com as três estratégias de decisão (classificação direta, limiar fixo e ponto de corte ótimo) e discute a respectiva calibração.

6.2 ANÁLISE POR MODELO

As respostas de cada modelo são avaliadas sob as três estratégias de decisão estabelecidas no plano de análise (Seção 5.2.4.2), assim identificadas nas figuras e tabelas deste capítulo:

1. *classification*: adota o rótulo emitido pelo próprio modelo no campo textual da resposta, sem recorrer à pontuação;
2. *likelihood* ≥ 50 : aplica um limiar fixo sobre a pontuação, classificando como *phishing* toda mensagem com *phishing_likelihood* maior ou igual a 50;
3. *F1-Score* ótimo: adota o corte que maximiza o *F1-Score* na amostra, isto é, o melhor desempenho que o modelo alcançaria se o limiar fosse calibrado à sua escala.

Cada estratégia corresponde a um ponto de operação na curva ROC do modelo, e a distância entre elas mede a calibração de cada um, ou seja, o quanto a decisão direta e o limiar fixo se aproximam do corte ideal. Como o corte ótimo é estimado sobre a mesma amostra em que é avaliado, sua estabilidade é verificada na Seção 6.3.1.

6.2.1 Claude Haiku 4.5

O Claude Haiku 4.5 atua como referência comercial do estudo e estabelece o patamar contra o qual os modelos locais são lidos nas subseções seguintes. Sua curva ROC (Figura 4) aproxima-se do ideal, com AUC de 0,991; no ponto de corte ótimo, o limiar de 76,5 maximiza o *F1-Score* em 96,3%. Foi também o único modelo a respeitar o formato de saída nas 2000 amostras submetidas, sem nenhuma resposta malformada, de modo que a rotina de reparo sequer foi acionada.

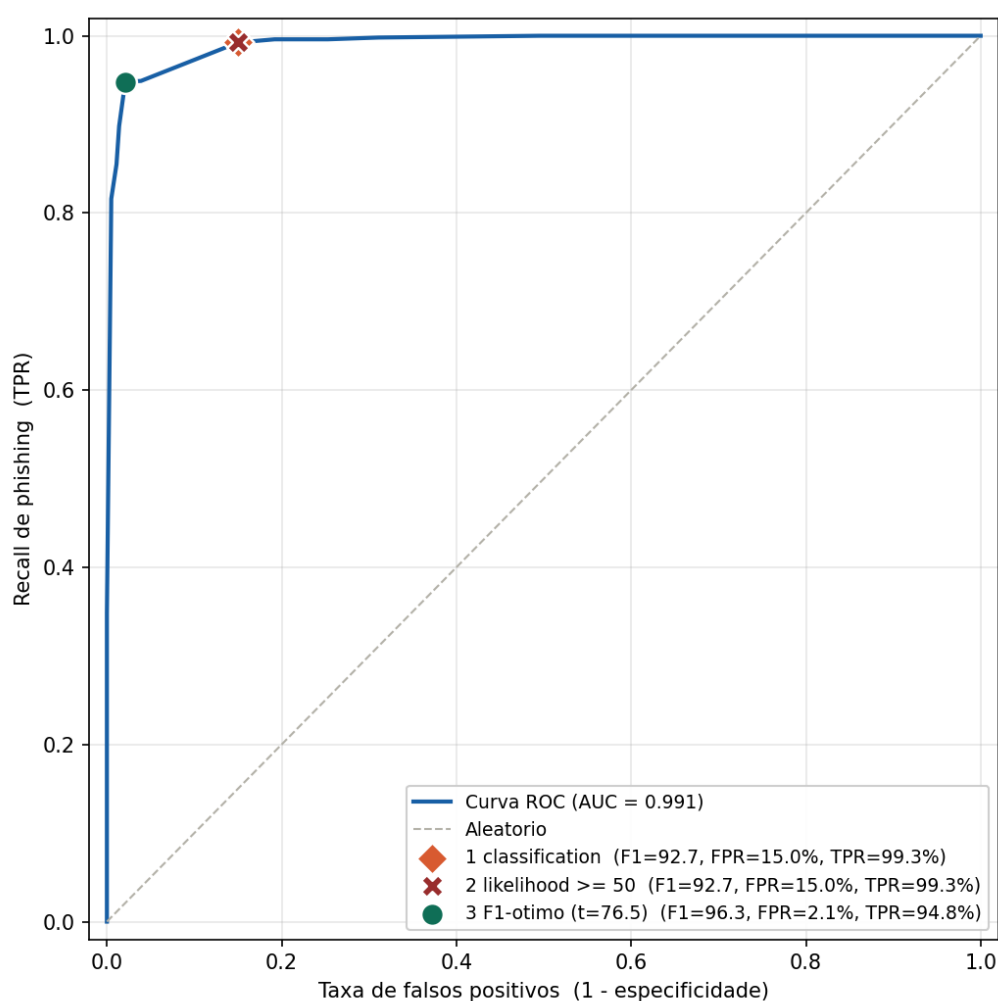


Figura 4 – Curva ROC do Claude Haiku 4.5.

Fonte: Elaborado pelo autor.

A Figura 5 apresenta a matriz de confusão no ponto de corte ótimo. A distribuição dos erros entre as duas classes é a mais equilibrada entre os cinco modelos avaliados: 52 e-mails de *phishing* não foram detectados (FN) e 21 mensagens legítimas foram classificadas como suspeitas (FP), o menor volume de falsos positivos do estudo.

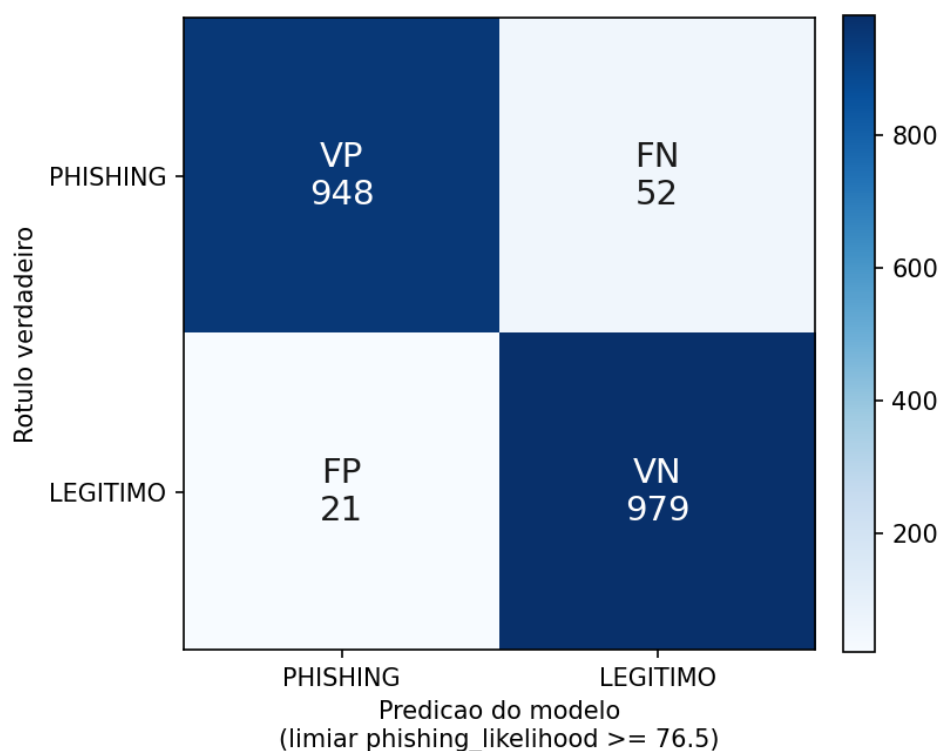


Figura 5 – Matriz de confusão do Claude Haiku 4.5.

Fonte: Elaborado pelo autor.

Essa separação reaparece na distribuição do *phishing_likelihood* (Figura 6), que dispõe a pontuação atribuída a cada e-mail, distinguida por classe, com os limiares fixo (50) e ótimo (76,5) em destaque: a maioria das mensagens de *phishing* situa-se acima do corte ótimo e as legítimas abaixo, o que corrobora o elevado poder de discriminação indicado pela AUC.

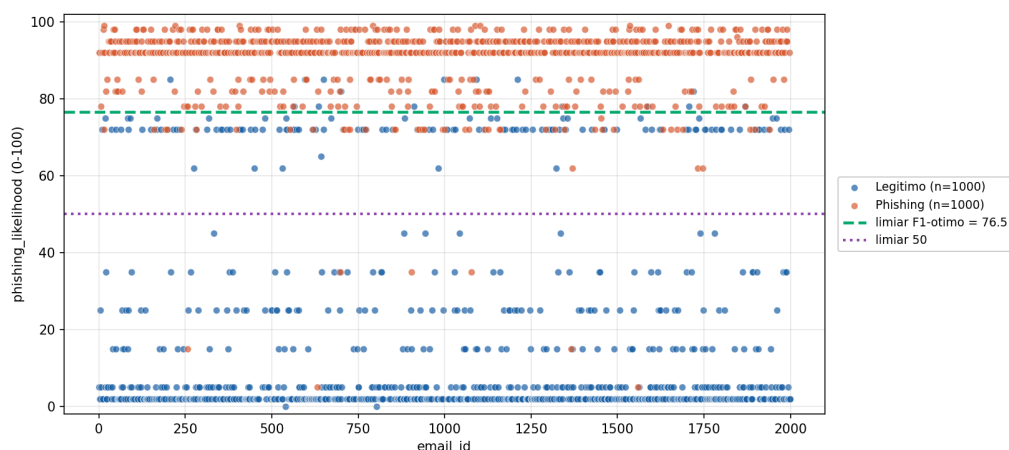


Figura 6 – Distribuição do *phishing_likelihood* por e-mail do Claude Haiku 4.5.

Fonte: Elaborado pelo autor.

6.2.2 Gemma 3 QAT 4B

O Gemma 3 QAT 4B, modelo denso quantizado a 4 bits via *Quantization-Aware Training*, apresentou um desequilíbrio acentuado entre os dois tipos de erro. Como mostra a matriz de confusão da Figura 7, os 279 falsos positivos superam expressivamente os 121 falsos negativos: o erro predominante está na classificação de e-mails legítimos como *phishing*, e é esse excesso de alarmes falsos que contém o *F1-Score* do modelo mesmo na melhor estratégia de decisão.

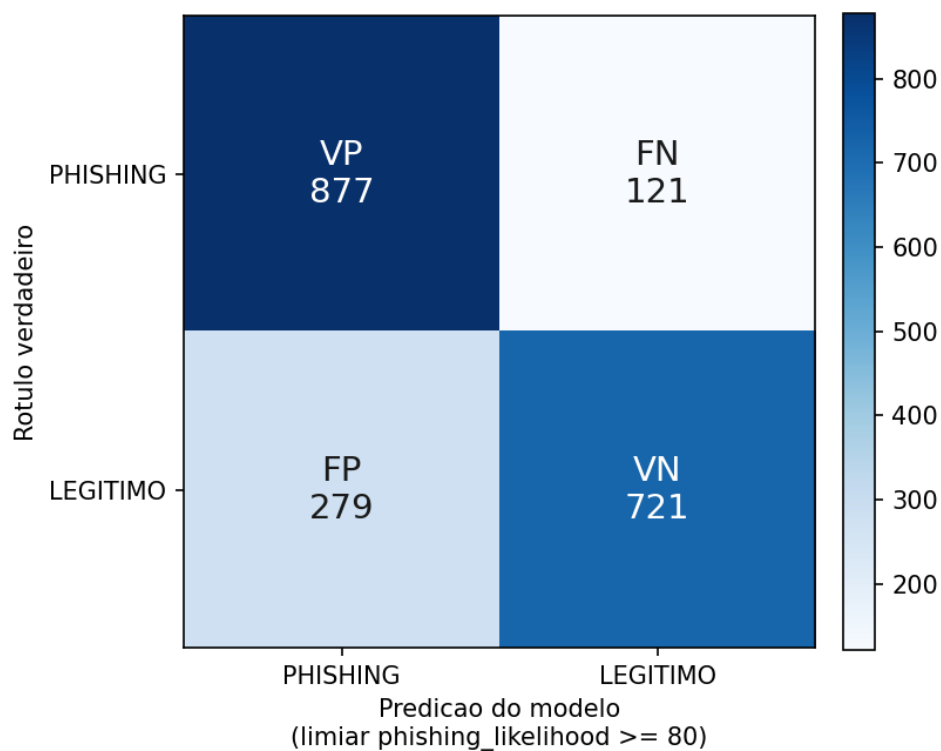


Figura 7 – Matriz de confusão do Gemma 3 QAT 4B.

Fonte: Elaborado pelo autor.

A origem do desequilíbrio aparece na distribuição do *phishing_likelihood* (Figura 8): entre 65 e 80 há uma faixa de sobreposição em que mensagens legítimas e de *phishing* recebem pontuações próximas, o que empurra o corte ótimo para 80 e, ainda assim, mantém muitas mensagens legítimas acima dele.

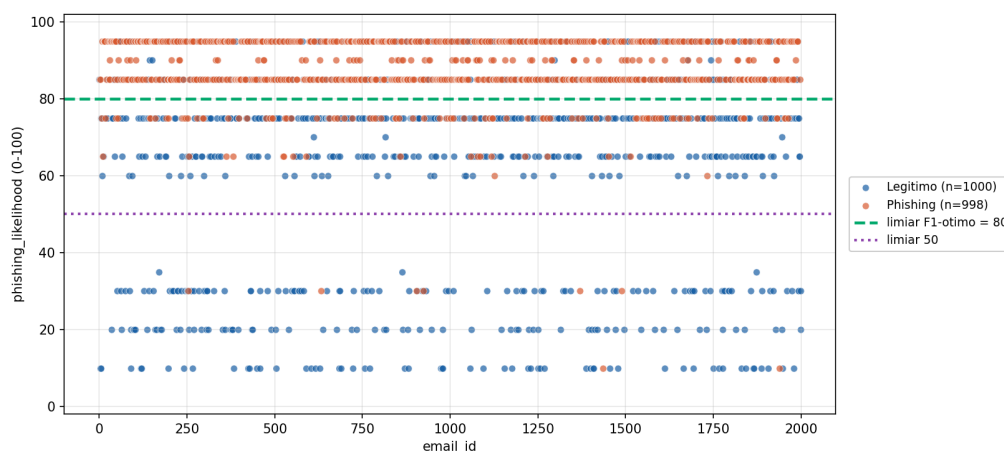


Figura 8 – Distribuição do *phishing_likelihood* por e-mail do Gemma 3 QAT 4B.

Fonte: Elaborado pelo autor.

A curva ROC (Figura 9) traduz esse comportamento em poder de discriminação: a AUC de 0,830 afasta a curva da diagonal, mas o joelho permanece distante do canto superior esquerdo, de modo que revocação alta só é obtida com elevado número de falsos positivos. Por isso, o *F1-Score* alcança apenas 81,4% mesmo no ponto de corte ótimo, no limiar de 80.

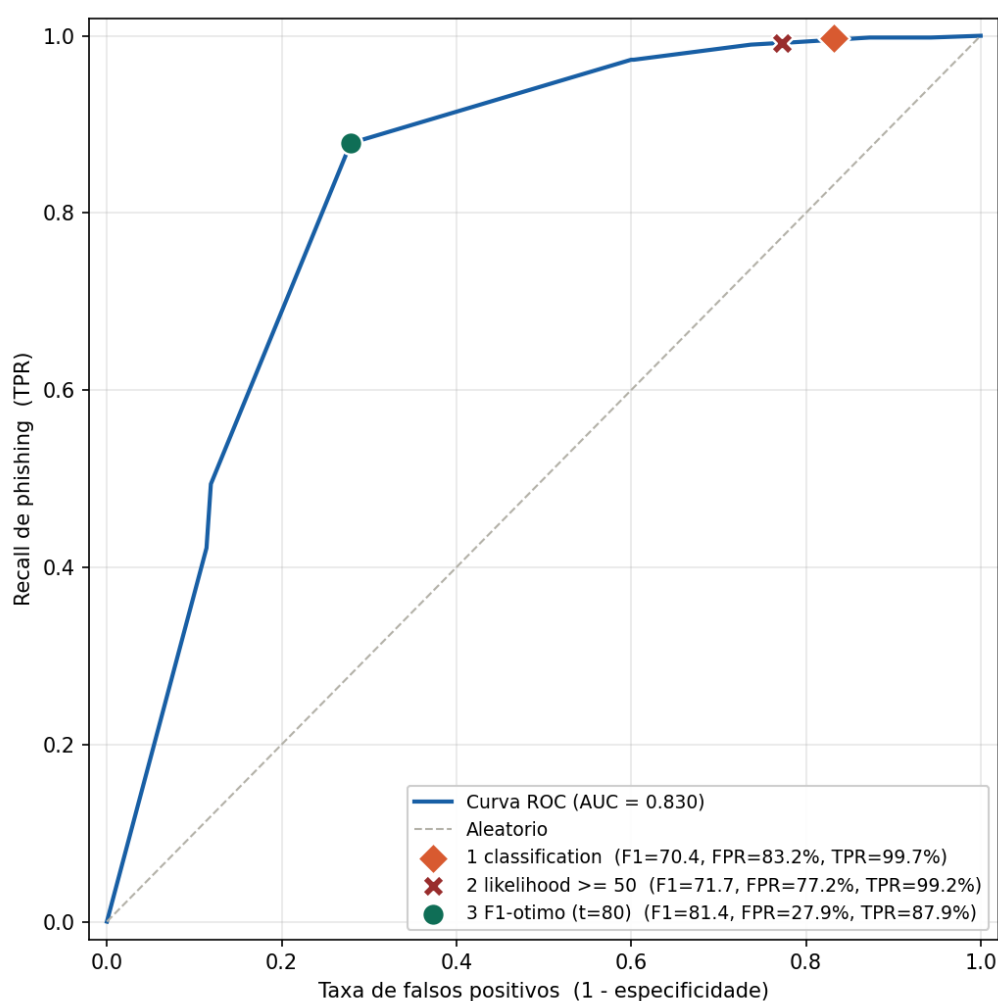


Figura 9 – Curva ROC do Gemma 3 QAT 4B.

Fonte: Elaborado pelo autor.

Quanto ao formato de saída, das 2000 amostras submetidas, 12 retornaram JSON malformatado; o reparo heurístico (Seção 4.4) recuperou 10 delas, e as 2 restantes, identificadas na Tabela 4, foram descartadas do cálculo das métricas.

Tabela 4 – E-mails com erro de formato irreparável no Gemma 3 QAT 4B.

<i>email_id</i>	Classe real	Resposta do Modelo
722	<i>phishing</i>	O modelo entrou em loop, reenviando a mesma <i>string</i> múltiplas vezes (parte de um <i>url</i>).
1127	<i>phishing</i>	O modelo entrou em loop, reenviando a mesma <i>string</i> múltiplas vezes (parte de um <i>url</i>).

Fonte: Elaborado pelo autor.

6.2.3 Qwen3-4B

O Qwen3-4B, modelo denso da família Qwen3, obteve a maior AUC entre os modelos locais (0,898) e, também entre eles, a melhor aderência ao formato de saída: apenas 2 das 2000 respostas vieram malformadas, ambas recuperadas pelo reparo heurístico, sem nenhum descarte. Sua curva ROC (Figura 10) tem o cotovelo próximo ao canto superior esquerdo, de modo que o modelo sustenta revocação alta com taxa de falsos positivos moderada.

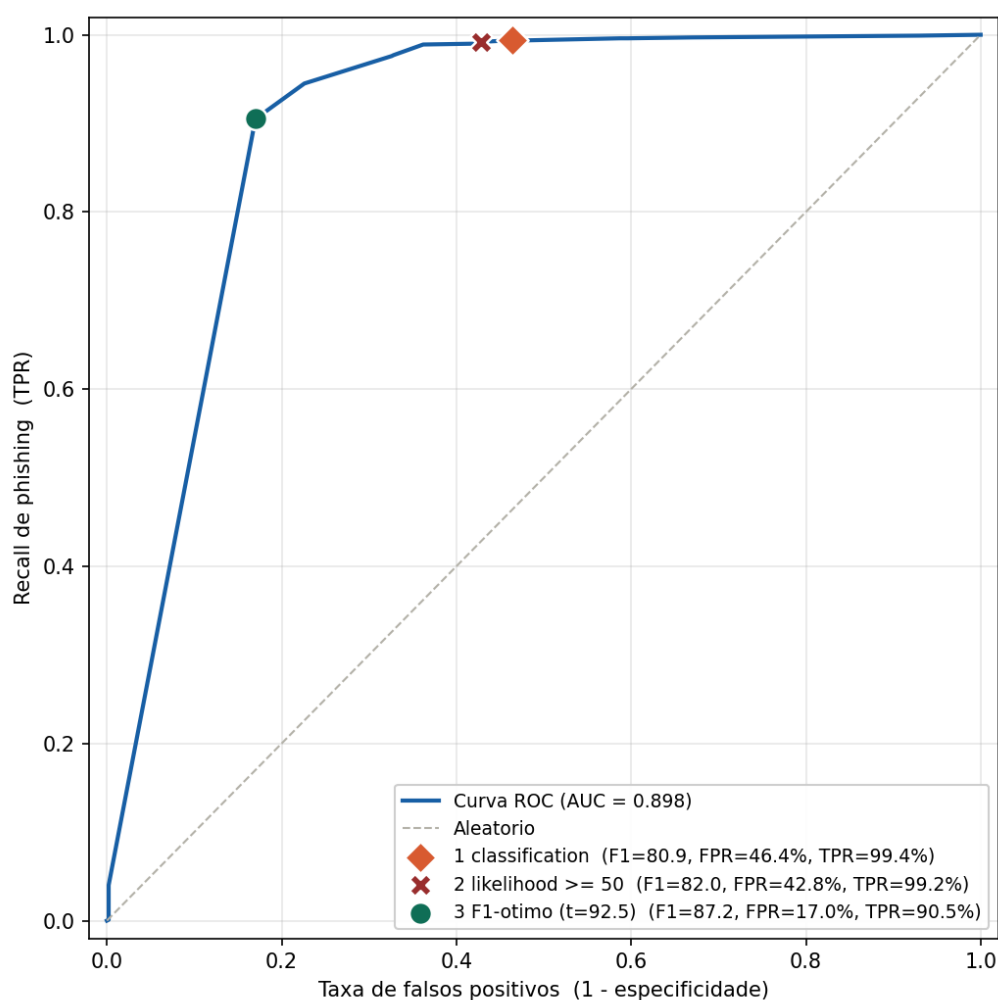


Figura 10 – Curva ROC do Qwen3-4B.

Fonte: Elaborado pelo autor.

Sua limitação não está na discriminação, e sim na calibração da escala. Como mostra a Figura 11, as mensagens de *phishing* concentram-se no topo da pontuação, mas parcela expressiva das legítimas também recebe valores entre 80 e 90, o que desloca o corte ótimo para 92,5. Apenas nesse limiar elevado o modelo atinge seu melhor *F1-Score*, de 87,2%; no limiar fixo de 50, distante da região em que as classes de fato se separam, o valor cai para 82,0% (Tabela 3).

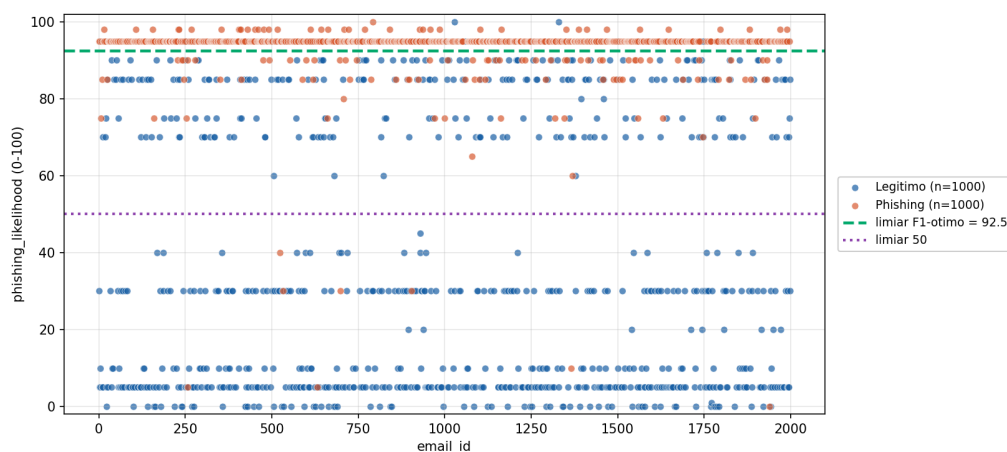


Figura 11 – Distribuição do *phishing_likelihood* por e-mail do Qwen3-4B.

Fonte: Elaborado pelo autor.

No corte ótimo, a matriz de confusão (Figura 12) registra o melhor equilíbrio de erros entre os modelos locais, com 170 falsos positivos contra 95 falsos negativos, distribuição coerente com a proximidade da curva ROC ao canto superior esquerdo.

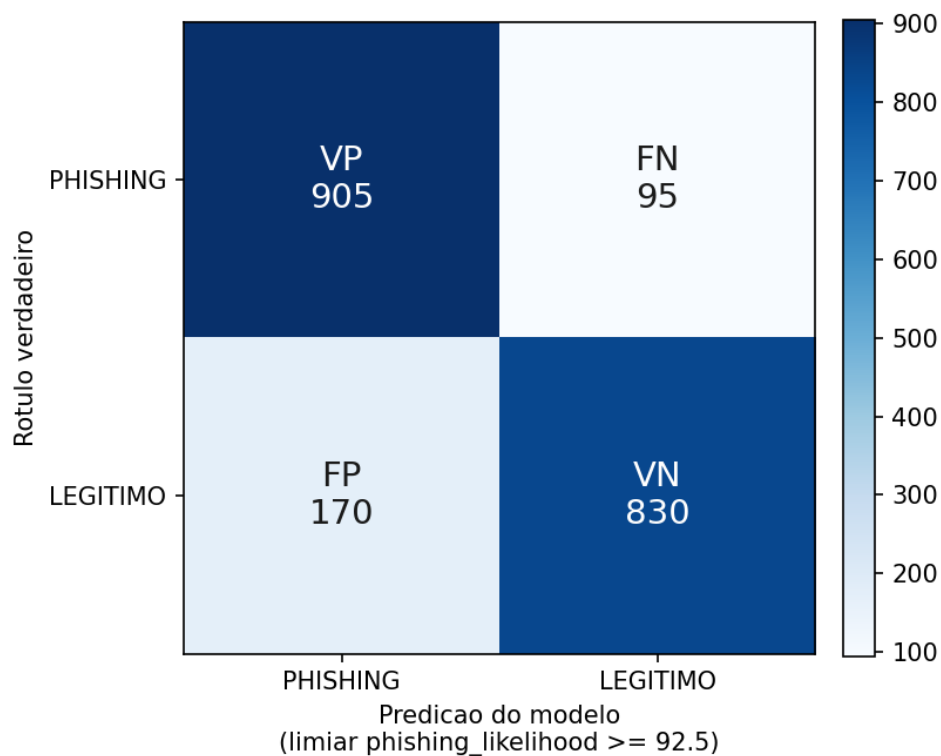


Figura 12 – Matriz de confusão do Qwen3-4B.

Fonte: Elaborado pelo autor.

6.2.4 Granite 4.0 Tiny

O Granite 4.0 Tiny distingue-se dos demais pela arquitetura *Mixture-of-Experts*, que ativa cerca de 1 bilhão de seus 7 bilhões de parâmetros a cada inferência. Seu comportamento de classificação foi o mais enviesado do estudo para a classe positiva: a matriz de confusão (Figura 13) registra revocação alta, com apenas 48 e-mails de *phishing* não detectados, ao custo de 376 falsos positivos, o maior volume de alarmes falsos entre os cinco modelos.

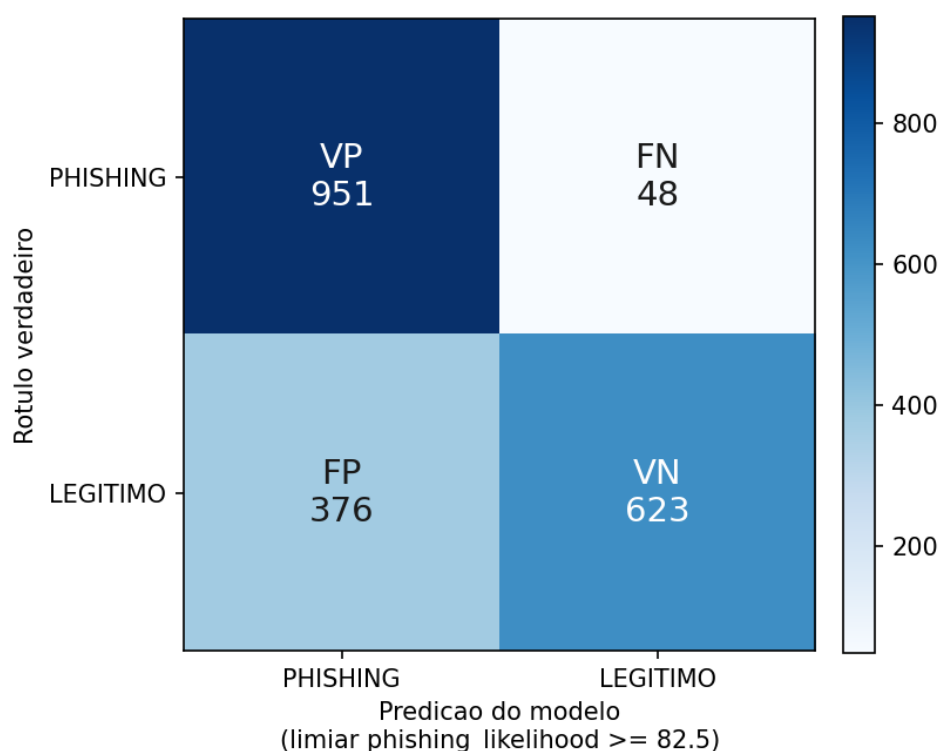


Figura 13 – Matriz de confusão do Granite 4.0 Tiny.

Fonte: Elaborado pelo autor.

A distribuição do *phishing_likelihood* (Figura 14) explica esse padrão: muitas mensagens legítimas recebem pontuação alta, na mesma faixa das de *phishing*, de modo que nem o corte ótimo, em 82,5, consegue separá-las.

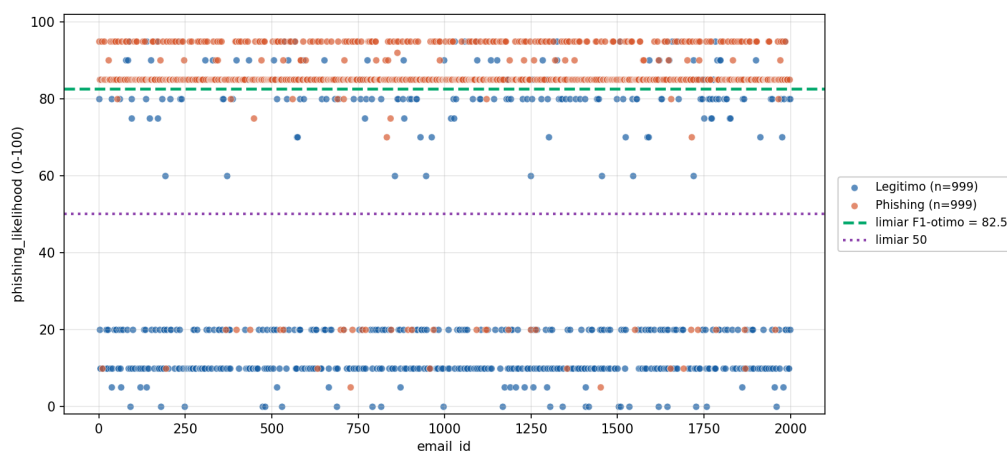


Figura 14 – Distribuição do *phishing_likelihood* por e-mail do Granite 4.0 Tiny.

Fonte: Elaborado pelo autor.

Na curva ROC (Figura 15), esse padrão se traduz na menor AUC do estudo, de 0,822, com o cotovelo distante do canto superior esquerdo: o ajuste do limiar reduz pouco a taxa de falsos positivos sem sacrificar a revocação, e o *F1-Score* limita-se a 81,8% mesmo na melhor estratégia.

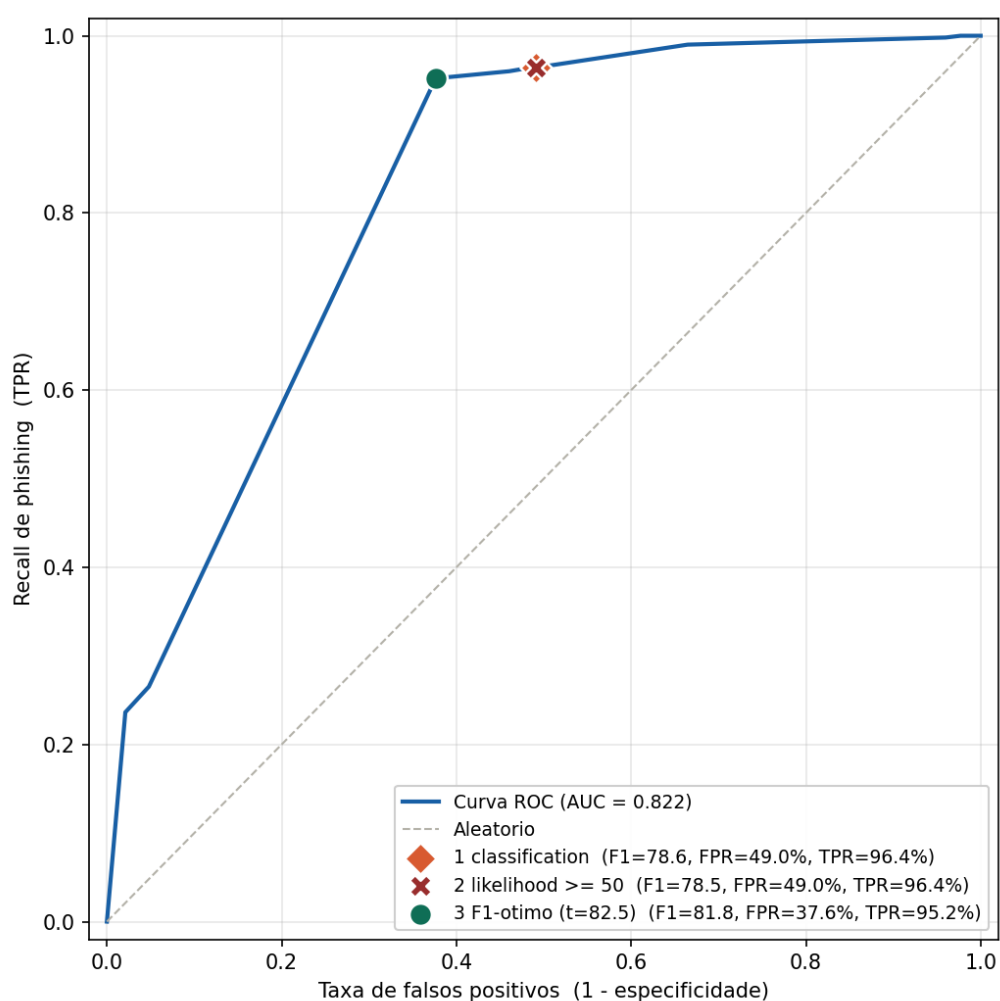


Figura 15 – Curva ROC do Granite 4.0 Tiny.

Fonte: Elaborado pelo autor.

No formato de saída, o Granite repetiu o padrão do Gemma: 12 respostas mal-formadas em 2000, das quais 10 foram recuperadas pelo reparo e 2 permaneceram irrecuperáveis, identificadas na Tabela 5 e descartadas do cálculo das métricas.

Tabela 5 – E-mails com erro de formato irreparável no Granite 4.0 Tiny.

<i>email_id</i>	<i>Classe real</i>	<i>Resposta do Modelo</i>
928	<i>phishing</i>	O modelo respondeu apenas com o caractere de abertura de chave ({) nas duas tentativas. Possivelmente isso ocorreu pois o e-mail é composto majoritariamente pelo alfabeto <i>abjad</i> (alfabeto este utilizado pelo árabes)
1334	<i>legítimo</i>	O modelo entrou em loop, reenviando a mesma <i>string</i> (parte de uma URL) múltiplas vezes.

Fonte: Elaborado pelo autor.

Entre os dois casos irreparáveis, o e-mail de identificador 928 merece registro, pois ilustra uma limitação característica dos modelos leves. Trata-se de um *phishing* redigido majoritariamente em árabe (alfabeto *abjad*), diante do qual o Granite emitiu apenas o caractere de abertura de chave nas duas tentativas, sem chegar a produzir uma classificação. A falha sugere que a competência do modelo degrada com conteúdo fora do inglês, idioma predominante tanto de seu treinamento quanto do *dataset*, e antecipa a ressalva de generalização para outros idiomas retomada na Seção 6.5.

6.2.5 Llama 3.2 3B

O Llama 3.2 3B é a menor variante de texto da família Llama, um modelo denso de código aberto quantizado a 4 bits e executado localmente. A Figura 16 apresenta a curva ROC, com AUC de 0,895 e os três pontos de operação. O cotovelo da curva situa-se próximo ao canto superior esquerdo, em patamar semelhante ao do Qwen3-4B (AUC de 0,898), até então o melhor entre os modelos locais. A diferença relevante está na calibração: o ponto de corte ótimo fica no limiar de 45, logo abaixo do limiar fixo de 50, e as três estratégias de decisão praticamente coincidem, todas com *F1-Score* em torno de 89%. A classificação direta atinge 89,2%, valor igual ao do corte ótimo. Dessa forma, o Llama decide de modo coerente com a própria pontuação, sem depender de um limiar elevado para alcançar seu melhor desempenho, ao contrário dos demais modelos locais, cujos cortes ótimos se deslocaram para 80 ou mais.

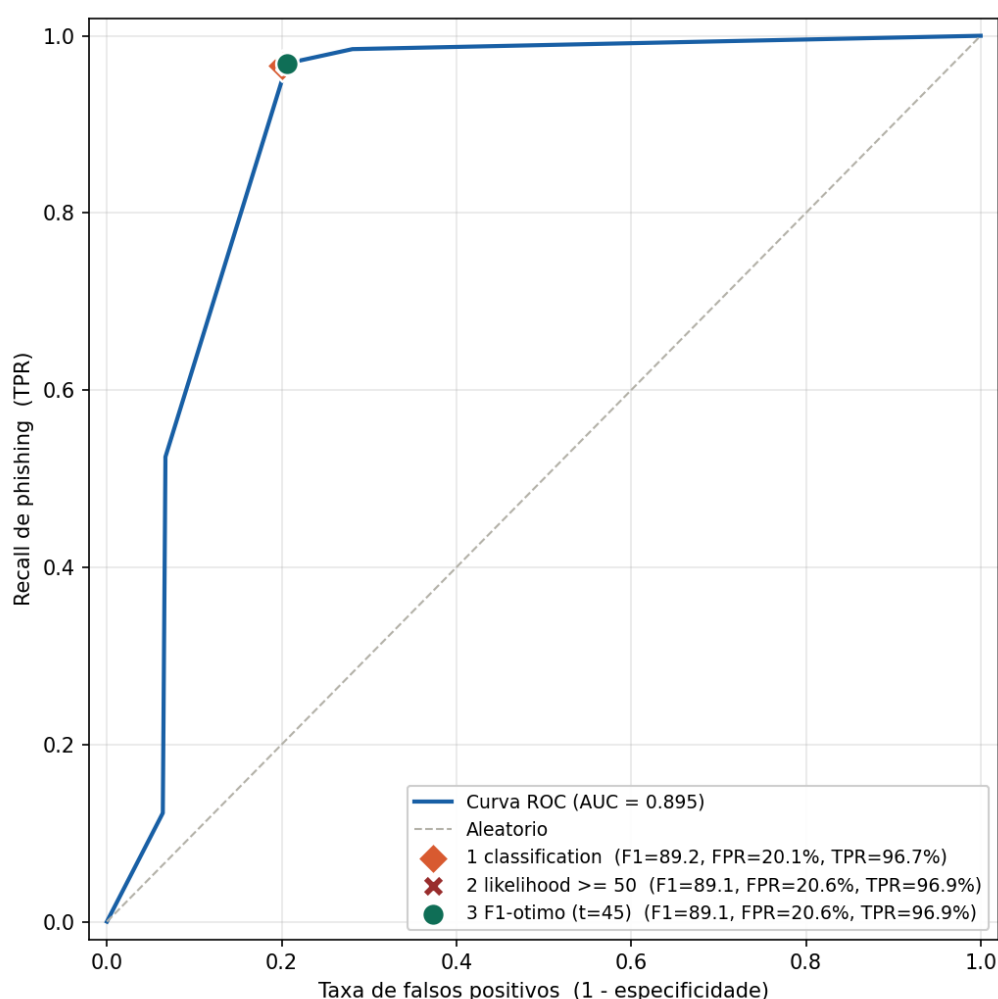


Figura 16 – Curva ROC do Llama 3.2 3B.

Fonte: Elaborado pelo autor.

Essa coincidência entre o corte ótimo e o limiar fixo, contudo, decorre menos de uma calibração fina do que da forma como o Llama distribui suas pontuações. A Tabela 6 mostra que o modelo não utilizou a escala contínua de 0 a 100, pois emitiu apenas sete valores distintos, concentrados em dois extremos. Mais de um terço das respostas (734) recebeu *likelihood* 0, e o restante se acumulou no topo da escala, com 983 respostas entre 80 e 90 e outras 187 em 100. Entre 20 e 70 há uma faixa praticamente vazia, com um único e-mail pontuado em 70. Qualquer limiar escolhido nessa lacuna, de 21 a 70, produz a mesma classificação, já que não existe pontuação a separar nesse intervalo. O ponto de corte ótimo recai em 45 não por ajuste preciso, mas por estar nessa banda sem dados, onde também se encontra o limiar fixo de 50. Portanto, as três estratégias coincidem porque a granularidade grosseira do modelo torna indiferente a escolha do corte em toda essa região.

Tabela 6 – Distribuição dos valores de *phishing_likelihood* emitidos pelo Llama 3.2 3B.

<i>phishing_likelihood</i>	Frequência	Proporção
0	734	36,8%
20	91	4,6%
70	1	0,1%
80	581	29,1%
90	402	20,1%
95	1	0,1%
100	187	9,4%
Total	1997	100,0%

Fonte: Elaborado pelo autor.

A Figura 17 ilustra graficamente essa concentração, ao dispor o *phishing_likelihood* de cada e-mail por classe e evidenciar a faixa vazia entre 21 e 79, em que recaem tanto o limiar fixo quanto o corte ótimo.

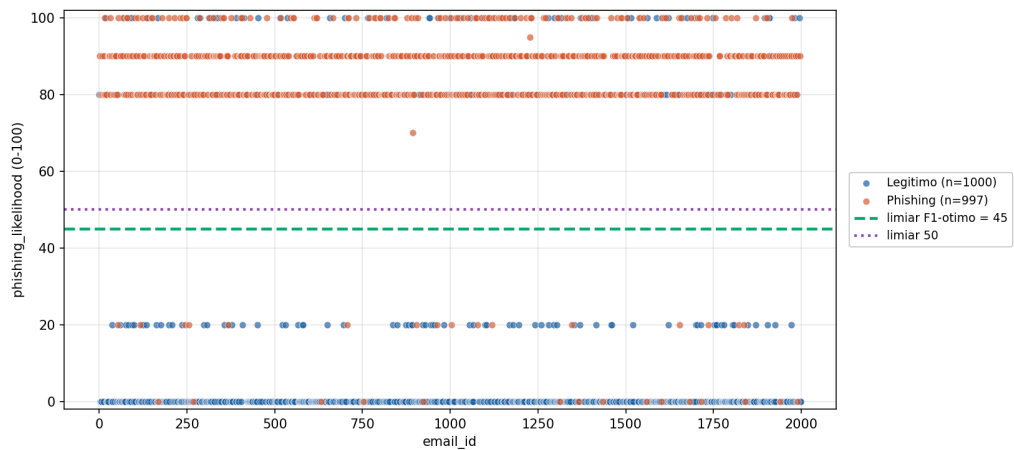


Figura 17 – Distribuição do *phishing_likelihood* por e-mail do Llama 3.2 3B.

Fonte: Elaborado pelo autor.

A Figura 18 apresenta a matriz de confusão no ponto de corte ótimo. A revocação é alta, pois apenas 31 e-mails de *phishing* não foram detectados (FN), o menor volume de falsos negativos entre os modelos locais. Os 206 falsos positivos indicam uma tendência a classificar e-mails legítimos como *phishing*, ainda que de modo mais contido que o Gemma (279) e o Granite (376), ficando atrás apenas do Qwen (170). Esse equilíbrio sustenta o *F1-Score* de 0,8905 da Tabela 3, o maior entre os modelos locais e o mais próximo do Claude Haiku 4.5.

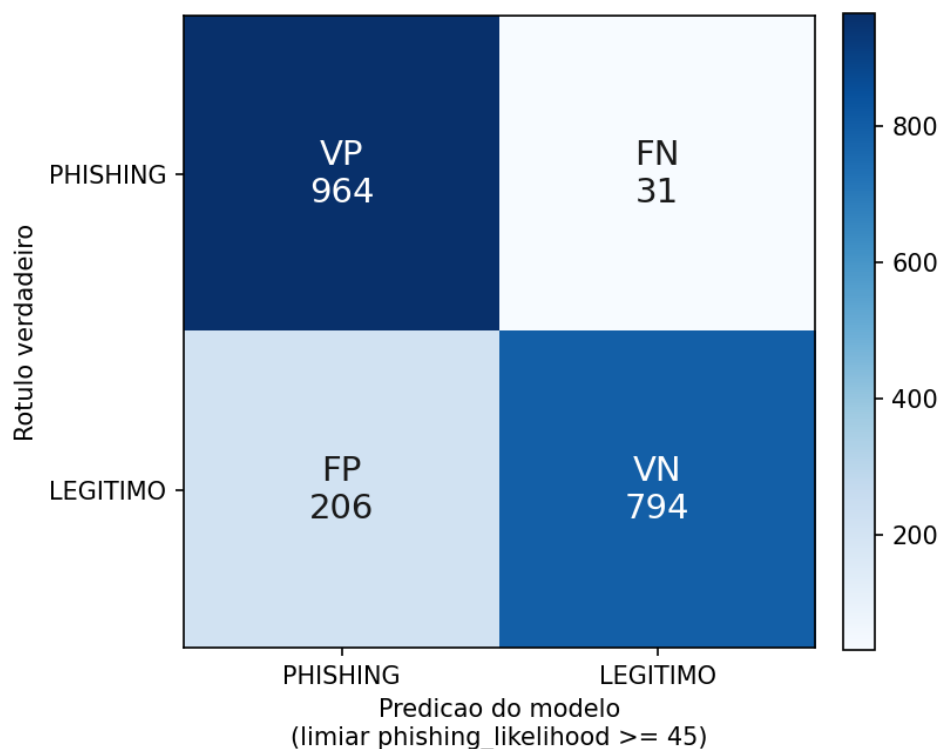


Figura 18 – Matriz de confusão do Llama 3.2 3B.

Fonte: Elaborado pelo autor.

No formato de saída, o Llama registrou 8 respostas malformadas em 2000, das quais 5 foram recuperadas pelo reparo heurístico; as 3 restantes, identificadas na Tabela 7, foram descartadas do cálculo das métricas.

Tabela 7 – E-mails com erro de formato irreparável no Llama 3.2 3B.

<i>email_id</i>	Classe real	Resposta do Modelo
450	<i>phishing</i>	O Modelo incluiu um segundo JSON dentro de " <i>Indicators</i> ", que o arquivo de <i>parsing</i> não suporta.
968	<i>phishing</i>	O Modelo gerou uma resposta muito longa, excedendo o limite de tokens.
1345	<i>phishing</i>	O Modelo incluiu um segundo JSON dentro de " <i>Indicators</i> ", que o arquivo de <i>parsing</i> não suporta. Adicionalmente, o modelo fechou a lista com chaves (}) ao invés de colchetes (]).

Fonte: Elaborado pelo autor.

O resultado confirma, na escala reduzida de 3 bilhões de parâmetros, o achado de

Zhang et al. (2025), em que a linhagem Llama liderou o *benchmark* de detecção de *phishing*. Ainda que seja o menor modelo do estudo, o Llama 3.2 3B foi o melhor entre os locais, tanto pela calibração adequada quanto pela proximidade com a referência comercial.

6.3 ANÁLISE COMPARATIVA ENTRE OS MODELOS

Nota-se que o Claude Haiku 4.5 obteve o melhor desempenho entre os cinco modelos, com a curva ROC mais próxima do ideal (AUC de 0,991) e a distribuição de erros mais equilibrada entre as classes. Tomado como referência, ele serve de base para situar os modelos locais, cuja ordem nesta comparação coincide com a observada nas análises individuais da Seção 6.2. O Llama 3.2 3B foi o mais próximo dessa referência, pois alcançou o maior *F1-Score* do grupo e, em razão da boa calibração, manteve esse desempenho já no limiar fixo, sem depender de um corte ajustado. O Qwen3-4B situou-se logo atrás, com poder de discriminação semelhante (AUC de 0,898 contra 0,895 do Llama), porém dependente de um limiar bem mais alto para atingir seu melhor *F1-Score*. O Gemma 3 QAT 4B e o Granite 4.0 Tiny, por sua vez, só alcançaram revocação alta ao custo de muitos falsos positivos, o que limitou o *F1-Score* de ambos mesmo no ponto de corte ótimo. A Figura 19 sintetiza esses resultados, confrontando o *F1-Score* dos cinco modelos sob as três regras de decisão. A Figura 20 reúne as curvas ROC dos cinco modelos para facilitar a comparação visual.

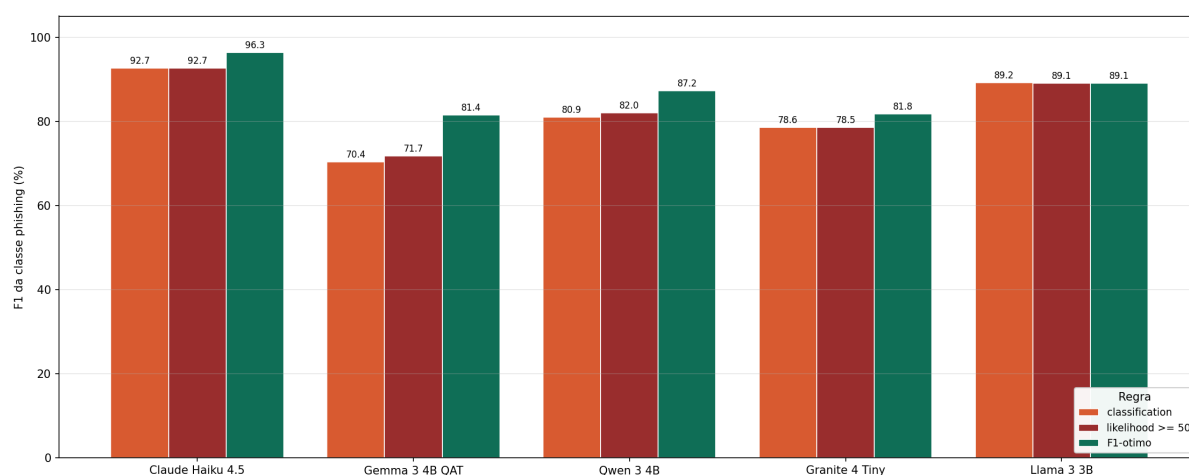


Figura 19 – *F1-Score* da classe *phishing* por modelo nas três regras de decisão.

Fonte: Elaborado pelo autor.

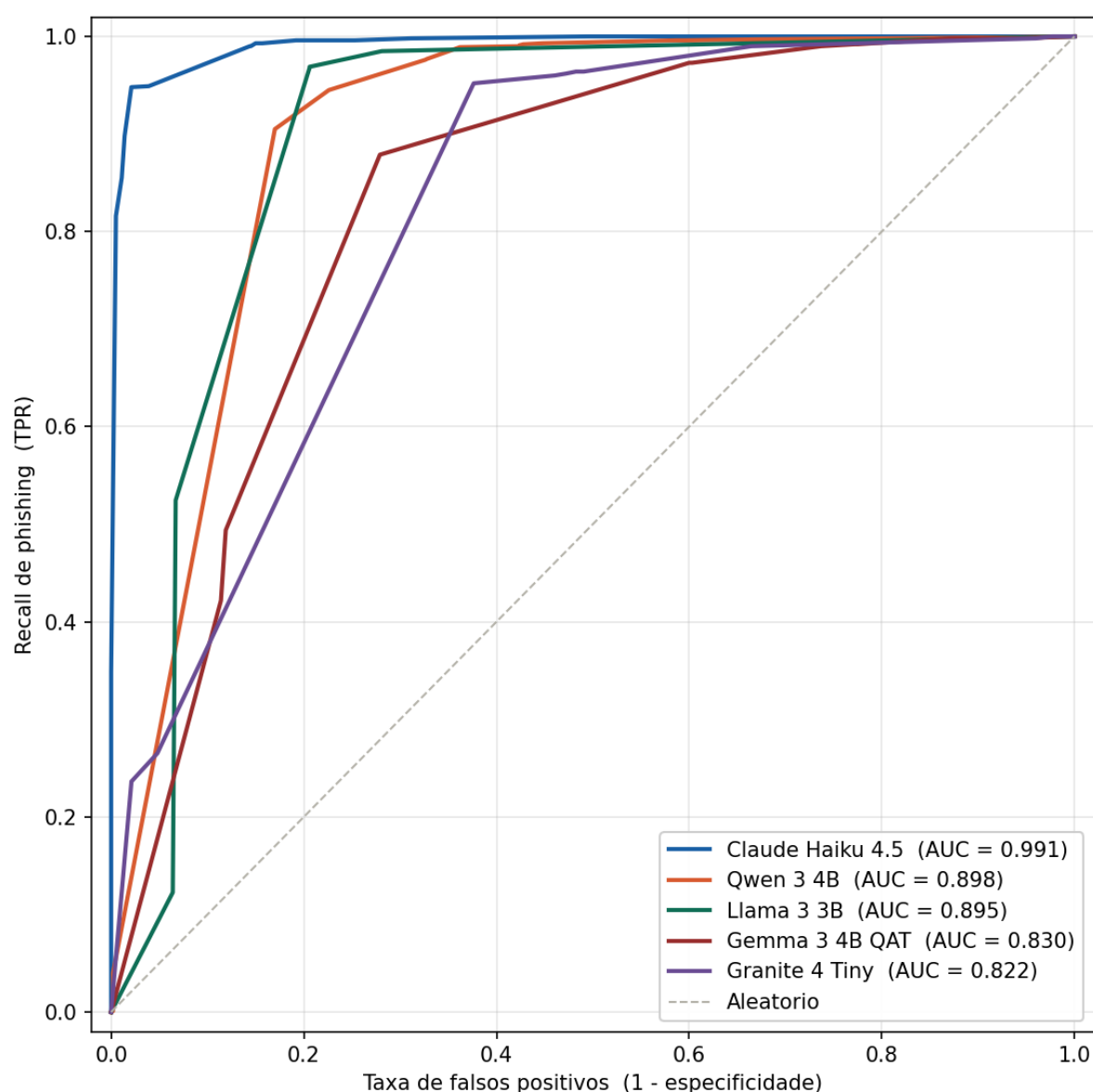


Figura 20 – Curvas ROC dos modelos comparados.

Fonte: Elaborado pelo autor.

6.3.1 Comparação Direta dos Modelos Contra o Claude Haiku 4.5

Para o estudo comparativo dos modelos, adotou-se a estratégia 3 da Seção 6.2, ou seja, o ponto de corte ótimo que maximiza o *F1-Score* de cada um. A escolha se justifica por representar o desempenho máximo de cada modelo, sem vincular a comparação a um limiar fixo que poderia favorecer um modelo em detrimento de outro. As métricas de classificação resultantes desse novo limiar estão reunidas na Tabela 8, que serve de base para os confrontos discutidos a seguir.

Tabela 8 – Métricas de classificação por modelo no ponto de corte ótimo (*F1-Score* máximo).

Modelo	Limiar	F1	Prec.	Rev.	Acur.	FPR	FNR
Claude Haiku 4.5	76,5	96,29%	97,83%	94,80%	96,35%	2,10%	5,20%
Llama 3.2 3B	45	89,07%	82,42%	96,89%	88,13%	20,60%	3,11%
Qwen3-4B	92,5	87,23%	84,19%	90,50%	86,75%	17,00%	9,50%
Granite 4.0 Tiny	82,5	81,77%	71,67%	95,20%	78,78%	37,64%	4,80%
Gemma 3 QAT 4B	80	81,43%	75,87%	87,88%	79,98%	27,90%	12,12%

Fonte: Elaborado pelo autor.

Essa escolha carrega um risco conhecido: o limiar ótimo é estimado sobre a mesma amostra em que as métricas são calculadas, de modo que o desempenho reportado poderia estar inflado pelo sobreajuste do corte aos dados de avaliação. Para dimensionar esse efeito, aplicou-se a validação por partição prevista na Seção 5.2.4.2, com o limiar estimado em 70% da amostra e o *F1-Score* recalculado nos 30% restantes. A Tabela 9 confronta os dois valores. O otimismo observado foi nulo em dois modelos e alcançou no máximo 0,4 ponto percentual (Qwen3-4B), o que indica que os limiares ótimos são estáveis nesta amostra e que o sobreajuste do corte não compromete a comparação conduzida a seguir. A ressalva remanescente é de generalização, dado que a amostra é ínfima frente ao universo de e-mails em circulação, de modo que a verificação afasta o sobreajuste interno, mas não garante que o valor exato de cada limiar se transfira a dados novos.

Tabela 9 – Validação do ponto de corte ótimo por partição 70/30: *F1-Score* (%) com o limiar estimado apenas na partição de estimativa.

Modelo	F1 na partição de estimativa (%)	F1 na partição de avaliação (%)	Otimismo (p.p.)
Claude Haiku 4.5	96,3	96,2	0,1
Llama 3.2 3B	89,1	89,1	0,0
Qwen3-4B	87,3	86,9	0,4
Granite 4.0 Tiny	81,8	81,7	0,1
Gemma 3 QAT 4B	81,4	81,4	0,0

Fonte: Elaborado pelo autor.

Em seguida, aplicou-se o teste de McNemar para confrontar o Claude Haiku 4.5 com cada um dos modelos locais, a partir das matrizes de confusão correspondentes ao ponto de corte ótimo. Os resultados indicaram que as diferenças entre o Claude e os modelos locais são estatisticamente significativas, com $p < 0,001$ nas quatro comparações.

A Figura 21 apresenta os pares discordantes de cada confronto, isto é, os e-mails em que um modelo acertou e o outro errou. A barra azul contabiliza os casos em que o

Claude acertou e o modelo local errou (*b*); a laranja, os casos opostos (*c*). Os pares concordantes, em que ambos acertam ou ambos erram, não entram no teste, já que o McNemar considera apenas as divergências entre os dois modelos. Em todos os confrontos, o volume de *b* é muito superior ao de *c*, ou seja, quando o Claude e um modelo local divergem, predomina o acerto do Claude.

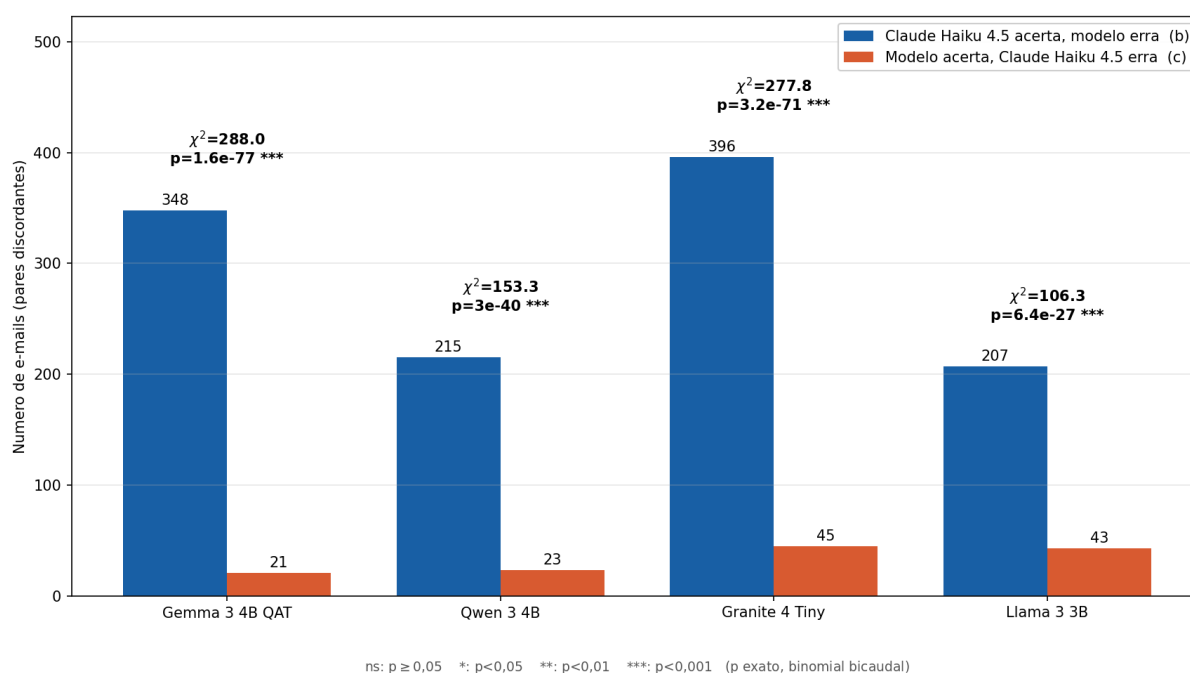


Figura 21 – Pares discordantes do teste de McNemar entre o Claude Haiku 4.5 e cada modelo local (dataset de 2000 elementos).

Fonte: Elaborado pelo autor.

A desproporção é maior nos dois modelos de pior desempenho. No Gemma 3 QAT 4B, o Claude acertou sozinho 348 e-mails contra 21 do modelo local ($\chi^2 = 288,0$); no Granite 4.0 Tiny, 396 contra 45 ($\chi^2 = 277,8$). O Qwen3-4B reduz a distância, com 215 contra 23 ($\chi^2 = 153,3$), e o Llama 3.2 3B registra a menor desproporção do grupo, 207 contra 43 ($\chi^2 = 106,3$). A razão entre acertos exclusivos do Claude e do modelo local cai de cerca de 17 para 1 no Gemma para menos de 5 para 1 no Llama, o que indica que o Llama é o modelo local que mais vezes acerta onde o Claude erra. Ainda assim, todas as diferenças permanecem significativas a $p < 0,001$.

O teste de McNemar indica que as diferenças existem, mas não quantifica sua magnitude. Para isso, realizou-se um *bootstrap* pareado das diferenças entre cada modelo local e o Claude, com 10000 reamostragens e intervalo de confiança de 95% (Figura 22). Foram contrastadas três métricas: o *F1-Score*, o AUC e o coeficiente de correlação de Matthews (MCC), este último mais sensível ao desequilíbrio entre acertos e erros nas duas classes. Todos os intervalos ficaram inteiramente abaixo de zero, sem

que nenhum cruzasse a linha de diferença nula, o que confirma a vantagem do Claude em todas as métricas e em todos os modelos.

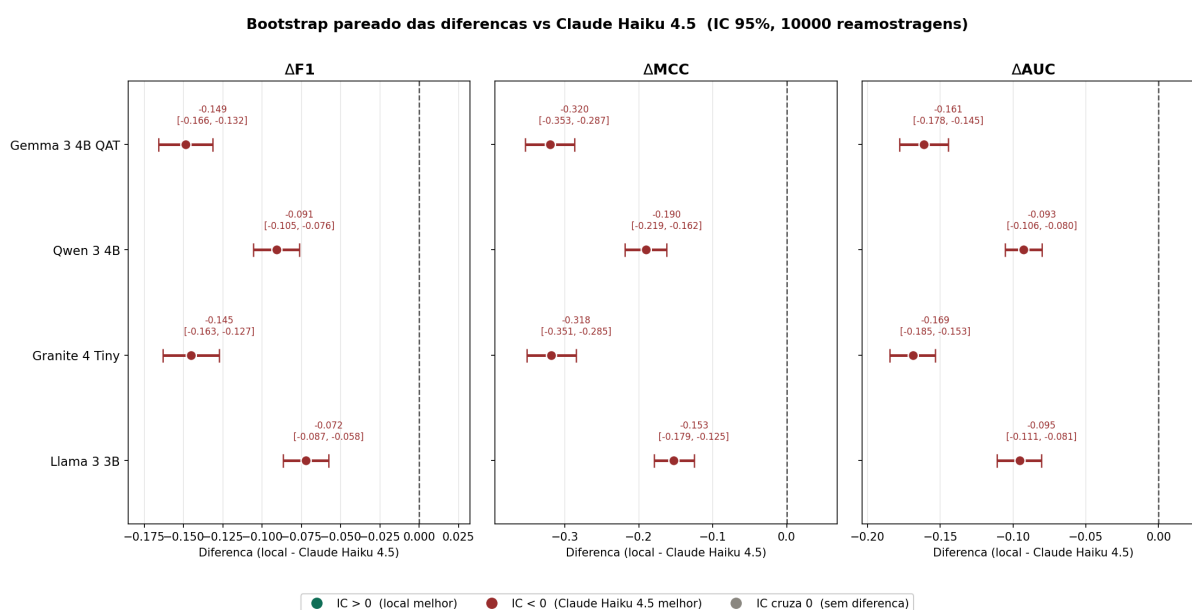


Figura 22 – *Bootstrap* pareado das diferenças (modelo local menos Claude Haiku 4.5) em *F1-Score*, MCC e AUC, com IC de 95%.

Fonte: Elaborado pelo autor.

A ordem dos modelos locais repete a já observada na análise individual. No *F1-Score*, o Llama 3.2 3B é o mais próximo da referência, com diferença de $-0,072$ $[-0,087; -0,058]$, seguido do Qwen3-4B, com $-0,091$ $[-0,105; -0,076]$; o Gemma ($-0,149$) e o Granite ($-0,145$) registram a maior diferença. No MCC, o contraste é mais acentuado: a defasagem chega a $-0,320$ no Gemma e a $-0,318$ no Granite, contra $-0,153$ no Llama e $-0,190$ no Qwen. No AUC, o ordenamento se mantém, com o Llama ($-0,095$) e o Qwen ($-0,093$) à frente do Gemma ($-0,161$) e do Granite ($-0,169$).

Os dois testes convergem. Ainda que o ponto de corte ótimo eleve o desempenho dos modelos locais, nenhum deles alcança o Claude Haiku 4.5 na detecção de *phishing* deste *dataset*, e a diferença é estatisticamente significativa e consistente entre as métricas. A distância, porém, não é uniforme: o Llama 3.2 3B e o Qwen3-4B aproximam-se mais da referência que o Gemma e o Granite, o que confirma o ordenamento observado na Seção 6.2.

6.4 MÉTRICAS OPERACIONAIS

Na Tabela 10 reúnem-se a latência por inferência (média e percentil 95) e o custo de cada condição. Para o Claude, o custo é monetário, cobrado por chamada de API;

para os modelos locais, corresponde ao consumo de *hardware*, medido em memória de vídeo (VRAM) e de sistema (RAM).

Tabela 10 – Métricas operacionais por modelo.

Modelo	Lat. média (s)	Lat. p95 (s)	Recursos
Claude Haiku 4.5	2,159	3,843	≈ 1,41 (US\$/1000 e-mails)
Gemma 3 QAT 4B	8,941	10,395	5,4 GB (VRAM) + 5 GB (RAM)
Qwen3-4B	11,134	18,121	5,4 GB (VRAM) + 6 GB (RAM)
Granite 4.0 Tiny	4,933	6,072	5,4 GB (VRAM) + 8 GB (RAM)
Llama 3.2 3B	7,538	8,335	4,8 GB (VRAM) + 0,8 GB (RAM)

Fonte: Elaborado pelo autor.

A menor latência média foi registrada pelo Claude Haiku 4.5 (2,159 s), ainda que se trate do único modelo acessado remotamente. Entre os locais, o Granite 4.0 Tiny apresentou a resposta mais rápida (4,933 s), comportamento coerente com a arquitetura *Mixture-of-Experts*, que ativa cerca de 1 dos 7 bilhões de parâmetros por inferência. O Qwen3-4B, em contrapartida, foi o mais lento (11,134 s) e o mais variável, pois seu percentil 95 (18,121 s) chega a quase o dobro da média. Esse comportamento é explicável, ao menos em parte, pela quantização do artefato adotado: único do conjunto em 8 bits, sua demanda de memória excede os 6 GB de VRAM da GPU empregada (Quadro 6), o que força a alocação de parte da carga na RAM e penaliza a latência. A comparação com o Claude não é direta, já que sua latência incorpora a rede e a execução em *datacenter*, ao passo que os locais operam no *hardware* descrito no Capítulo 5. Ainda assim, mesmo o modelo local mais lento responde em menos de vinte segundos no percentil 95, e os demais em poucos segundos, prazos compatíveis com a triagem de e-mails.

No plano de custo, a chamada ao Claude equivale a cerca de US\$ 1,41 por mil e-mails. Embora o valor seja baixo, a cobrança por inferência configura uma superfície de ataque, pois um adversário pode enviar grandes volumes de e-mails para inflar a despesa da organização. Os modelos locais, por sua vez, não incorrem em custo por inferência, mas apenas em demanda de *hardware*. O Gemma 3 QAT 4B, o Qwen3-4B e o Granite 4.0 Tiny ocupam cerca de 5,4 GB de VRAM cada, enquanto o Llama 3.2 3B mostrou-se o mais econômico, com 4,8 GB de VRAM e 0,8 GB de RAM, consumo que cabe em uma GPU de gama intermediária. Cabe observar que os consumos medidos superam as recomendações do Quadro 6, caso mais visível no Llama, com recomendação de 2 a 3 GB: a recomendação refere-se apenas aos pesos do modelo, ao

passo que a medição abrange também o cache de atenção (*KV cache*), dimensionado pela janela de contexto, e os *buffers* do ambiente de execução. A coluna de RAM, por sua vez, reflete a parcela da carga que não coube na VRAM, somada aos *buffers* mantidos pelo ambiente de execução na memória do sistema, e por isso varia conforme o tamanho total do artefato de cada modelo. O caso mais visível é o do Granite 4.0 Tiny, com 8 GB de RAM: embora a arquitetura *Mixture-of-Experts* ative cerca de 1 bilhão de parâmetros por inferência, os pesos dos 7 bilhões precisam permanecer integralmente carregados, de modo que a economia da arquitetura recai sobre a computação, e não sobre a memória ocupada.

Constatou-se, portanto, que o Llama 3.2 3B combinou o menor consumo de *hardware* e o maior *F1-Score* entre os locais. O custo do modelo local recai sobre o *hardware*, sob controle de quem o executa, ao passo que o da API é monetário e expõe dados pessoais a um terceiro. Essa exposição e suas implicações sob a LGPD e o GDPR são retomadas na Seção 6.5.

6.5 DISCUSSÃO

A análise dos resultados permite responder à questão que orienta este trabalho. Constatou-se que nenhum dos modelos locais igualou o Claude Haiku 4.5 neste *dataset*: o teste de McNemar apontou diferença significativa nas quatro comparações ($p < 0,001$) e o *bootstrap* pareado manteve todos os intervalos abaixo de zero em *F1-Score*, MCC e AUC (Seção 6.3.1). Observou-se, contudo, que essa defasagem não é uniforme entre os modelos locais. O Llama 3.2 3B situou-se a 0,072 de *F1-Score* da referência e o Qwen3-4B a 0,091, ao passo que o Gemma 3 QAT 4B e o Granite 4.0 Tiny apresentaram defasagem próxima de 0,15. Dessa forma, a resposta não é categórica: os dois modelos locais de melhor desempenho aproximam-se o suficiente para que a escolha frente à API passe a depender de critérios adicionais, enquanto os dois de pior desempenho mantêm distância expressiva.

Além da frequência do erro, a sua natureza também deve ser considerada. Na detecção de *phishing*, o falso negativo constitui a falha crítica de segurança, pois deixa passar a mensagem maliciosa, enquanto o falso positivo onera a operação ao encaminhar e-mails legítimos para revisão. Quanto a esse aspecto, os modelos locais não apresentaram deficiência na retenção das mensagens maliciosas. No ponto de corte ótimo, o Llama registrou apenas 31 falsos negativos e o Granite, 48, ambos abaixo dos 52 do próprio Claude, cujo limiar ótimo (76,5) privilegia a precisão e, nesse ponto de operação, deixa escapar mais *phishing* que esses dois locais. A deficiência dos modelos locais concentrou-se no volume de alarmes falsos, 376 no Granite e 279 no Gemma, que em um fluxo real possivelmente sobrecarregaria a triagem e reduziria a confiança do analista (TARIQ et al., 2025). O Llama apresentou o melhor equilíbrio entre

as duas taxas, com 31 falsos negativos e 206 falsos positivos, aproximando-se de um comportamento operacionalmente adequado. Esse contraste evidencia que o *F1-Score* isolado oculta uma diferença qualitativa, visto que modelos com *F1* semelhante podem errar de maneiras com custos operacionais distintos.

A comparação também deve incorporar o custo e a privacidade. Constatou-se que o Claude apresentou o melhor desempenho, mas opera sob custo por inferência, cerca de US\$ 1,41 por mil e-mails, valor que constitui igualmente uma superfície de ataque, pois um adversário pode inflar a despesa com grande volume de mensagens. Além do custo, o modelo comercial requer o envio do conteúdo integral de cada e-mail a um terceiro, o que expõe dados potencialmente pessoais ou sigilosos mesmo quando a mensagem é legítima. A execução local elimina esse vetor (Seção 2.3.4): os modelos avaliados ocupam cerca de 5,4 GB de VRAM, o Llama apenas 4,8 GB, compatíveis com uma GPU intermediária e mantidos sob infraestrutura controlada. Sob a LGPD e o GDPR, que restringem o tratamento de dados pessoais por agentes externos e a transferência internacional, a permanência dos dados no perímetro da organização configura uma exigência, e não uma mera conveniência. Dessa forma, a defasagem de 0,072 do Llama deve ser ponderada frente aos ganhos correspondentes: custo previsível, soberania dos dados e redução da exposição regulatória. Para organizações que priorizam a privacidade, tal diferença de desempenho tende a ser aceitável, e cabe qualificar o ponto de operação em que a vantagem de cada modelo se manifesta. No corte ótimo adotado nesta comparação, o limiar elevado do Claude (76,5) privilegia a precisão e deixa escapar 52 *phishing* (FNR 5,20%), acima dos 31 do Llama (FNR 3,11%): nesse ponto, é o modelo local que minimiza a falha crítica de segurança, ao passo que o Claude minimiza os falsos positivos. A primazia do Claude reside na discriminação global (AUC 0,991), que lhe permite, sob um limiar voltado à revocação, atingir a maior taxa de detecção do estudo, 99,30% no limiar fixo de 50, ainda com o menor volume de falsos positivos entre as condições. Dessa forma, o Claude permanece preferível para quem demanda a máxima detecção e não trata dados sensíveis, desde que calibrado para esse fim; já no regime de *F1-Score* máximo, a menor taxa de falsos negativos coube ao Llama. Os resultados, portanto, não determinam uma escolha única, mas fornecem a base empírica antes ausente para fundamentá-la.

Os achados também se alinham à literatura correlata. O desempenho do Llama reforça, em escala de 3 bilhões de parâmetros, o que Zhang et al. (2025) observaram em modelos maiores, com a linhagem Llama em posição de destaque em *benchmark* de detecção de *phishing*, e mostra-se coerente com os resultados promissores de Chataut, Gyawali e Usman (2024) quanto ao emprego de LLMs como classificadores. A contribuição deste trabalho consiste em preencher a lacuna identificada na Seção 3.4, visto que os estudos anteriores comparam predominantemente modelos comerciais entre si, sem confrontá-los com alternativas locais sob um protocolo uniforme e sem situar a privacidade como critério central de escolha. Ao avaliar os modelos leves

locais em paralelo com um modelo comercial, sob as mesmas condições de *prompt* e decodificação, este estudo demonstra a viabilidade da opção local e quantifica o seu custo em desempenho.

Os resultados devem ser interpretados dentro dos limites do estudo. No plano da validade interna, as variáveis foram controladas e a decodificação fixada em temperatura 0 e top-p 1, o que reduz a variação atribuível ao acaso; o ponto de corte ótimo, porém, foi estimado sobre a mesma amostra em que é avaliado, de modo que indica o potencial de cada modelo, não uma garantia transferível a dados novos, ainda que a validação por partição (Tabela 9) tenha indicado sobreajuste desprezível dentro da própria amostra. No plano da validade externa, o *dataset* é composto por e-mails em inglês oriundos de fontes específicas (Seção 5.2), e a generalização para outros idiomas e variantes de ataque, como *spear phishing* e BEC, não está assegurada, limitação ilustrada pela falha do Granite diante de um e-mail redigido em árabe (Seção 6.2.4), que o modelo não conseguiu sequer classificar. Pesa também o descompasso temporal entre as fontes: os e-mails legítimos do Enron datam do início dos anos 2000, enquanto o *phishing* do Nazario reúne campanhas de 2024 e 2025, de modo que sinais de época e de estilo podem ter contribuído para a separação das classes. Por atingir as cinco condições por igual, esse viés não compromete a comparação relativa, que é o objeto deste trabalho, mas sugere que as métricas absolutas superestimam o desempenho esperado em um fluxo real de mensagens contemporâneas. Quanto à validade de construção, o *F1-Score* foi adotado como métrica-síntese por equilibrar precisão e revocação, mas, conforme demonstrado neste capítulo, não captura isoladamente o custo assimétrico dos dois tipos de erro, razão pela qual é acompanhado das matrizes de confusão e das curvas ROC. Por fim, a reprodutibilidade estrita é limitada pelo Claude, modelo de natureza fechada e sujeito a atualizações não documentadas do provedor. Essas e as demais limitações de escopo, antecipadas na introdução, são retomadas no Capítulo 7.

7 CONSIDERAÇÕES FINAIS

Neste trabalho, realizou-se uma análise comparativa entre quatro modelos de linguagem leves executados localmente (Gemma 3 QAT 4B, Qwen3-4B, Granite 4.0 Tiny e Llama 3.2 3B) e um modelo comercial de grande porte (Claude Haiku 4.5) na detecção de e-mails de *phishing*. A pesquisa partiu do dilema de implantação exposto no Capítulo 1: os modelos comerciais acessados via API externa oferecem alta capacidade de detecção, mas exigem o envio do conteúdo dos e-mails a um terceiro, exposição que a LGPD e o GDPR tratam como risco legal, enquanto os modelos locais mantêm os dados sob o controle da organização, com capacidade menor. Disso resultou a questão de pesquisa: modelos de linguagem leves, executados localmente, são capazes de oferecer desempenho competitivo frente a um modelo comercial de referência na classificação binária de e-mails de *phishing*?

7.1 RETOMADA DOS OBJETIVOS

O objetivo geral, de comparar modelos leves locais com um modelo comercial de grande porte nessa tarefa, foi realizado pelo encadeamento dos objetivos específicos. Os dois primeiros, de natureza teórica, foram atendidos na primeira parte do trabalho: a revisão sistemática da literatura, conduzida no Capítulo 3, evidenciou a lacuna explorada nesta pesquisa, a ausência de comparações entre modelos comerciais e modelos leves locais sob um protocolo uniforme; e o *phishing* foi caracterizado no Capítulo 2, com seus mecanismos de engenharia social e suas características detectáveis no texto da mensagem.

Os demais objetivos, de natureza experimental, foram atendidos nos Capítulos 4 a 6. Construiu-se um *dataset* balanceado de 2000 e-mails a partir do *Nazario Phishing Corpus* e do *Enron Corpus*; selecionaram-se cinco modelos com embasamento na literatura, um comercial e quatro locais; e desenvolveu-se a aplicação de avaliação que padronizou o *prompt zero-shot*, a decodificação determinística e a saída em JSON, registrando as respostas e a latência de cada amostra. Por fim, as respostas foram avaliadas com as métricas definidas e os modelos foram comparados no Capítulo 6, com teste de significância e discussão integrada de desempenho, custo e privacidade. Dessa forma, os objetivos específicos foram atendidos e, em conjunto, realizaram o objetivo geral.

7.2 SÍNTESE DOS PRINCIPAIS RESULTADOS

O Claude Haiku 4.5 confirmou o papel de referência, com a curva ROC mais próxima do ideal (AUC de 0,991), *F1-Score* de 96,3% no ponto de corte ótimo, o menor volume de falsos positivos e a menor latência média do estudo. Entre os locais, o Llama 3.2 3B obteve o melhor resultado (*F1-Score* de 89%) e foi o único cuja classificação direta coincidiu com o desempenho do corte ótimo, sem depender de um limiar ajustado. O Qwen3-4B ficou logo atrás (87,2%, alcançado apenas com limiar elevado), enquanto o Gemma 3 QAT 4B e o Granite 4.0 Tiny ficaram mais distantes (em torno de 81%), limitados pelo volume de alarmes falsos: 279 e 376 falsos positivos, contra 21 do Claude.

O teste de McNemar indicou diferença significativa nas quatro comparações contra o Claude ($p < 0,001$), e o *bootstrap* pareado manteve todos os intervalos de confiança abaixo de zero em *F1-Score*, MCC e AUC. A defasagem, porém, não é uniforme: vai de $-0,072$ de *F1-Score* no Llama e $-0,091$ no Qwen até cerca de $-0,15$ no Gemma e no Granite. A natureza dos erros qualifica essa leitura: os modelos locais retiveram bem as mensagens maliciosas (o Llama deixou escapar apenas 31 e-mails), e sua deficiência concentrou-se nos falsos positivos, que oneram a triagem, mas não constituem a falha crítica de segurança.

No plano operacional, o custo do Claude é monetário, cerca de US\$ 1,41 por mil e-mails, e acompanha a exposição do conteúdo a um terceiro. Os modelos locais demandam apenas *hardware*: de 4,8 a 5,4 GB de VRAM, consumo compatível com uma GPU intermediária, com respostas em poucos segundos. O Llama 3.2 3B combinou o melhor desempenho entre os locais e o menor consumo.

A resposta à questão de pesquisa é, portanto, condicional. Os modelos locais não igualam o modelo comercial, e a diferença é estatisticamente significativa; os melhores entre eles, contudo, aproximam-se o suficiente para constituir uma alternativa viável quando a privacidade dos dados é prioritária. O Claude mantém a maior capacidade de discriminação do estudo (AUC de 0,991) e permanece preferível para organizações que demandam a máxima taxa de detecção e não tratam dados sensíveis. Esse predomínio, porém, não se estende a todos os pontos de operação. No corte ótimo adotado na comparação, foi o Llama, e não o Claude, quem reteve mais *phishing*, com 31 falsos negativos contra 52, pois o limiar que maximiza o *F1* do Claude privilegia a precisão. Para as organizações que precisam manter os dados no próprio perímetro, a defasagem de $0,072$ de *F1-Score* do Llama é o preço, agora quantificado, da soberania dos dados, e o resultado do Llama no eixo crítico de segurança torna essa contrapartida ainda mais favorável.

7.3 CONTRIBUIÇÕES

A principal contribuição deste trabalho é a comparação empírica controlada entre modelos leves locais e um modelo comercial de grande porte na detecção de *phishing* por e-mail, sob um protocolo uniforme que isola o efeito do modelo. Esse desenho preenche a lacuna identificada na Seção 3.4, quantifica o custo em desempenho da opção local e situa a privacidade como critério central da escolha de um classificador, e não como consideração secundária.

Adicionalmente, a avaliação sob três estratégias de decisão (rótulo direto, limiar fixo e corte ótimo) mostrou que a calibração da pontuação é uma dimensão própria de desempenho, distinta do poder de discriminação: modelos com AUC semelhante exigiram limiares muito diferentes para atingir seu melhor *F1-Score*. Por fim, a documentação do *prompt*, do protocolo experimental e dos identificadores do *dataset* viabiliza a replicação dos experimentos, lacuna de reprodutibilidade apontada na revisão da literatura.

7.4 LIMITAÇÕES

O *dataset* é composto por e-mails em inglês de fontes específicas, de modo que a generalização para outros idiomas e para variantes como *spear phishing* e *Business Email Compromise* (BEC) não está assegurada. As classes provêm ainda de períodos distintos, pois os e-mails legítimos do *Enron Corpus* antecedem em cerca de duas décadas o *phishing* coletado; o viés atinge os cinco modelos por igual e preserva a comparação relativa, mas recomenda cautela na leitura das métricas absolutas.

Quanto ao protocolo, os modelos foram avaliados sem *fine-tuning* e sob um único *prompt zero-shot*, de forma que os resultados não representam o teto de capacidade de cada um. O corte ótimo foi estimado na própria amostra; a validação por partição indicou sobreajuste desprezível, mas o valor exato dos limiares não tem garantia de transferência a dados novos. Além disso, a reprodutibilidade estrita é limitada pela natureza fechada do Claude, as latências refletem uma única configuração de *hardware* e a resiliência a ataques voltados a LLMs, como o *prompt injection*, não foi avaliada.

7.5 SUGESTÕES PARA TRABALHOS FUTUROS

Recomendam-se quatos direções de continuidade. A primeira é a evolução do *dataset*, com outros idiomas, variantes como *spear phishing* e BEC e classes contemporâneas entre si, o que eliminaria o descompasso temporal entre as fontes. A segunda é a exploração da configuração dos modelos: estratégias de *prompting* como *few-shot* e *chain-of-thought*, o ajuste fino dos modelos locais e a validação dos limiares

em partições independentes podem estreitar a distância até a referência comercial. A terceira trata da expansão dos modelos, para cobrir outros modelos além dos aqui estudados. A quarta aproxima a avaliação da operação real, com a integração do classificador a um fluxo de triagem, para medir o efeito dos alarmes falsos sobre os analistas, e com o estudo de defesas contra *prompt injection*, vetor deixado fora deste recorte.

A segurança da informação enfrenta hoje ataques potencializados pela mesma tecnologia que pode defendê-la. Este trabalho reuniu evidências de que essa contraparte defensiva não está restrita aos grandes provedores comerciais: modelos abertos e leves, executados em *hardware* acessível, mostram-se uma alternativa concreta, ainda que não igualem a referência comercial, e abrem um caminho de implantação que preserva a privacidade e a soberania dos dados, requisitos que a LGPD e o GDPR tornam exigências, e não preferências.

REFERÊNCIAS

ADMASS, W. S.; MUNAYE, Y. Y.; DIRO, A. A. Cyber security: State of the art, challenges and future directions. *Cyber Security and Applications*, Elsevier / KeAi Communications Co., Ltd., v. 2, p. 100031, 2024. ISSN 2772-9184. Open access under CC BY-NC-ND 4.0 license. Disponível em: <<https://doi.org/10.1016/j.csa.2023.100031>>.

ALAM, N. A. *Phishing Email Dataset: Phish No More – The Enron, Ling, CEAS, Nazario, Nigerian & SpamAssassin Datasets*. [S.l.]: Kaggle, 2024. <<https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset>>. Accessed: 2026-06-01.

Anthropic. *The AI for Problem Solvers | Claude by Anthropic*. 2025. Página de produto oficial. Disponível em: <<https://claude.com/product/overview>>. Acesso em: 21 mai. 2025.

AXELSSON, S. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, ACM, New York, NY, USA, v. 3, n. 3, p. 186–205, August 2000. ISSN 1094-9224. An earlier version appeared in: Proceedings of the Sixth ACM Conference on Computer and Communications Security (November 1–9), ACM Press, 1999, pp. 1–7. Disponível em: <<https://dl.acm.org/doi/abs/10.1145/357830.357849>>.

BAKI, S.; VERMA, R. Sixteen years of phishing user studies: What have we learned? *arXiv preprint arXiv:2109.04661*, September 2021. Submitted to IEEE for possible publication. Disponível em: <<https://arxiv.org/abs/2109.04661>>.

BAR-HILLEL, M. The base-rate fallacy in probability judgments. *Acta Psychologica*, Elsevier, v. 44, n. 3, p. 211–233, 1980.

BERTINO, E. Data security and privacy: Concepts, approaches, and research directions. In: IEEE. *Proceedings of the 2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)*. Atlanta, GA, USA: IEEE, 2016. p. 400–407.

BOMMASANI, R.; HUDSON, D. A.; ADELI, E.; ALTMAN, R.; ARORA, S.; ARX, S. von; BERNSTEIN, M. S.; BOHG, J.; BOSSELUT, A.; BRUNSKILL, E. et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021. Referência clássica sobre a opacidade (caixa preta) e os desafios da adoção de modelos fundacionais.

BRASIL. *Lei n. 13.709, de 14 de agosto de 2018*. Brasília, DF, 2018. Lei Geral de Proteção de Dados Pessoais (LGPD).

CARLINI, N.; IPPOLITO, D.; JAGIELSKI, M.; LEE, K.; TRAMER, F.; ZHANG, C. Quantifying memorization across neural language models. In: *The Eleventh International Conference on Learning Representations*. [S.l.: s.n.], 2022.

CARLINI, N.; TRAMÈR, F.; WALLACE, E.; JAGIELSKI, M.; HERBERT-VOSS, A.; LEE, K.; ROBERTS, A.; BROWN, T.; SONG, D.; ERLINGSSON, Ú.; OPREA, A.; RAFFEL, C. Extracting training data from large language models. In: *30th USENIX Security Symposium (USENIX Security 21)*. [S.l.]: USENIX Association, 2021. p. 2633–2650.

- CHATAUT, R.; GYAWALI, P. K.; USMAN, Y. Can ai keep you safe? a study of large language models for phishing detection. In: IEEE. *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*. [S.l.], 2024. p. 0548–0554.
- CHICCO, D.; JURMAN, G. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, BioMed Central, v. 21, n. 1, p. 6, 2020.
- DENNING, D. E.; DENNING, P. J. Data security. *ACM Computing Surveys (CSUR)*, Association for Computing Machinery, New York, NY, USA, v. 11, n. 3, p. 227–249, September 1979. ISSN 1557-7341. Open Access supported by Purdue University.
- DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural computation*, MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , v. 10, n. 7, p. 1895–1923, 1998.
- Docker Inc. *Docker Model Runner*. 2026. <<https://docs.docker.com/ai/model-runner/>>. Acesso em: 4 jun. 2026.
- DROR, R.; BAUMER, G.; SHLOMOV, S.; REICHART, R. The hitchhiker's guide to testing statistical significance in natural language processing. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. [S.l.: s.n.], 2018. p. 1383–1392.
- DUAN, H.; DZIEDZIC, A.; YAGHINI, M.; PAPERNOT, N.; BOENISCH, F. On the privacy risk of in-context learning. *arXiv preprint arXiv:2411.10512*, 2024.
- EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. New York: Chapman & Hall/CRC, 1993. (Monographs on Statistics and Applied Probability, 57). ISBN 978-0-412-04231-7.
- EI, L. M.; YONGSIRIWIT, K. Phishing email classification using few-shot learning with a lightweight LLM (Llama 3.2). In: *2025 9th International Conference on Information Technology (InCIT)*. Pathum Thani, Thailand: IEEE, 2025. p. 75–80. ISBN 978-1-6654-7748-2.
- Elsevier. *Scopus Preview: Empowering research with the world's leading abstract and citation database*. 2026. Acesso em: 3 jun. 2026. Disponível em: <<https://www.scopus.com/pages/preview>>.
- European Parliament; Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. 2016. OJ L 119, 4.5.2016, p. 1–88. Disponível em: <<https://data.europa.eu/eli/reg/2016/679/oj>>. Acesso em: 13 abr. 2023.
- EZE, C. S.; SHAMIR, L. Analysis and prevention of AI-based phishing email attacks. *Electronics*, MDPI, v. 13, n. 10, p. 1839, 2024.
- FAWCETT, T. An introduction to roc analysis. *Pattern recognition letters*, Elsevier, v. 27, n. 8, p. 861–874, 2006.

- GOLCHIN, S.; SURDEANU, M. Time travel in LLMs: Tracing data contamination in large language models. In: *The Twelfth International Conference on Learning Representations*. [S.l.: s.n.], 2024.
- GRATTAFIORI, A. et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- GREE, A. D. *O que é o golpe do príncipe nigeriano?* 2026. <<https://www.avast.com/pt-br/c-nigerian-prince-scam>>. Avast Academy. Revisado por Oliver Buxton. Acesso em: 17 maio 2026.
- GRESHAKE, K.; ABDELNABI, S.; MISHRA, S.; ENDRES, C.; HOLZ, T.; FRITZ, M. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. [S.l.: s.n.], 2023. p. 79–90.
- GRIFFITHS, C. *The Latest 2025 Phishing Statistics*. 2025. <<https://aag-it.com/the-latest-phishing-statistics/>>. AAG IT Support. Acessado em: 27 mai. 2026.
- GULYAMOV, S.; GULYAMOV, S.; RODIONOV, A.; KHURSANOV, R.; MEKHMUNOV, K.; BABAEV, D.; RAKHIMJONOV, A. Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms. *Information*, MDPI, v. 17, n. 1, p. 54, 2026. Received: 25 October 2025; Revised: 23 December 2025; Accepted: 31 December 2025; Published: 7 January 2026. Disponível em: <<https://doi.org/10.3390/info17010054>>.
- HAN, D.; Unsloth Team. *granite-4.0-h-tiny-GGUF: Quantized GGUF Format of IBM Granite-4.0-H-Tiny by Unsloth*. [S.l.]: Hugging Face, 2025. <<https://huggingface.co/unsloth/granite-4.0-h-tiny-GGUF>>. Modelo base: Granite-4.0-H-Tiny (IBM), modelo instruct de 7B parâmetros com arquitetura híbrida Mamba. Licença: Apache 2.0. Arquivo acessado em: 31 de maio de 2026.
- HEIDING, F.; SCHNEIER, B.; VISHWANATH, A.; BERNSTEIN, J.; PARK, P. S. Devising and detecting phishing emails using large language models. *IEEE Access*, v. 13, p. 42131–42146, 2024.
- HU, Z.; ROSTAMI, M.; THOMASON, J. Expert personas improve llm alignment but damage accuracy: Bootstrapping intent-based persona routing with prism. *arXiv preprint arXiv:2603.18507*, 2026.
- IBM Security; Ponemon Institute. *Cost of a Data Breach Report 2025*. Armonk, NY, USA, 2025. Pesquisa conduzida pelo Ponemon Institute, analisada e patrocinada pela IBM. Disponível em: <<https://www.ibm.com/reports/data-breach>>.
- Inventive HQ Team. *What is URL Defanging and Why It*. 2025. Acessado em junho de 2026. Disponível em: <<https://inventivehq.com/blog/what-is-url-defanging-cybersecurity>>.
- JAIN, V.; PANDEY, B.; ALDASHEVA, L.; SHUKLA, P.; PANDEY, P.; JAIN, D. A comprehensive usage of ai in prevention of social engineering attack. In: *2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT)*. [S.l.: s.n.], 2025. p. 668–673.

- JAYATILAKA, A.; ARACHCHILAGE, N. A. G.; BABAR, M. A. Why people still fall for phishing emails: An empirical investigation into how users make email response decisions. In: *Proceedings of the Symposium on Usable Security and Privacy (USEC)*. [s.n.], 2024. Disponível em: <<https://arxiv.org/abs/2401.13199>>.
- JI, L.; YAO, L.; XU, W.; MIN, J.; YUAN, L. The impact of prompt language and representation on llm reasoning: A multilingual empirical study. *IEEE Access*, IEEE, v. 14, p. 2249–2267, 2025.
- JI, Z.; LEE, N.; FRIESKE, R.; YU, T.; SU, D.; XU, Y.; ISHII, E.; BANG, Y.; MADOTTO, A.; FUNG, P. Survey of hallucination in natural language generation. *ACM Computing Surveys*, ACM New York, NY, v. 55, n. 12, p. 1–38, 2023.
- JOURNAL, S. *Phishing Email Detection*. [S.l.]: Kaggle, 2023. <<https://www.kaggle.com/datasets/subhajournal/phishingemails>>. Accessed: 2026-06-01.
- KITCHENHAM, B. *Procedures for Performing Systematic Reviews*. [S.l.], 2004.
- KLIMT, B.; YANG, Y. The Enron corpus: A new dataset for email classification research. In: *Machine Learning: ECML 2004*. Berlin, Heidelberg: Springer, 2004. p. 217–226.
- MAGAR, I.; SCHWARTZ, R. Data contamination: From memorization to exploitation. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. [S.l.: s.n.], 2022. p. 157–165.
- MESTER, G. Rankings scientists, journals and countries using h-index. *Interdisciplinary Description of Complex Systems*, v. 14, n. 1, p. 1–9, jan. 2016. Disponível em: <<https://indecs.eu/index.php?s=x&y=2016&p=1-9>>. Acesso em: 17 mai. 2026.
- Meta AI. *Llama 3.2: Revolutionizing edge AI and vision with open, customizable models*. 2024. <<https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>>. Anúncio oficial da família Llama 3.2, incluindo as variantes leves de 1B e 3B.
- NASUTION, A. H.; MONIKA, W.; ONAN, A.; MURAKAMI, Y. Benchmarking 21 open-source large language models for phishing link detection with prompt engineering. *Information*, MDPI, v. 16, n. 5, p. 366, 2025.
- NAZARIO, J. *Phishing Corpus*. 2005. <<https://monkey.org/~jose/phishing/>>. Accessed: 2026-06-06.
- NGUYEN, T. et al. An evaluation of llms inference on popular single-board computers. *arXiv preprint arXiv:2511.07425*, 2025.
- ONWUEGBUCHE, F. C.; TITUNG, R.; RANTANEN, E. M.; JURCUT, A. D.; ALM, C. O.; PASQUALE, L. Securing the weakest link: Exploring affective states exploited in phishing emails with large language models. *IEEE Access*, v. 13, p. 173460–173486, 2025.
- OPENAI. *GPT-2: 1.5B Release*. 2019. Acedido em 11 de junho de 2026. Disponível em: <<https://openai.com/index/gpt-2-1-5b-release/>>.
- OpenAI. *Introducing GPTs*. 2023. Disponível em: <<https://openai.com/index/introducing-gpts/>>.

PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A.; MICHEL, V.; THIRION, B.; GRISEL, O.; BLONDEL, M.; PRETTENHOFER, P.; WEISS, R.; DUBOURG, V.; VANDERPLAS, J.; PASSOS, A.; COUNAPEAU, D.; BRUCHER, M.; PERROT, M.; DUCHESNAY, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

POWERS, D. M. W. Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061*, p. 37–63, October 2020. Disponível em: <<https://arxiv.org/abs/2010.16061>>.

PRODANOV, C. C.; FREITAS, E. C. D. *Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico-2ª Edição*. [S.l.]: Editora Feevale, 2013.

Python Software Foundation. *Python 3.14.5 Documentation*. 2026. <<https://docs.python.org/3/>>. Acesso em: 4 jun. 2026.

Qwen Team; Alibaba Cloud. *Qwen Studio*. 2025. Plataforma web. Acessado em: 22 maio 2026. Disponível em: <<https://qwen.ai/home>>.

ROCHER, L.; HENDRICKX, J. M.; MONTJOYE, Y.-A. de. Estimating the success of re-identifications in incomplete datasets using generative models. *Nature Communications*, v. 10, n. 1, p. 3069, 2019.

SAINZ, O.; CAMPOS, J.; GARCÍA-FERRERO, I.; ETXANIZ, J.; LACALLE, O. L. de; AGIRRE, E. Nlp evaluation in trouble: On the need to measure llm data contamination for each benchmark. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. [S.l.: s.n.], 2023. p. 10776–10787.

SAKA, T.; JAIN, R.; VANIEA, K.; KÖKCIYAN, N. Phishing codebook: A structured framework for the characterization of phishing emails. *arXiv preprint arXiv:2408.08967*, 2024.

SCHEIDT, P. *phishing-llm-comparison: Benchmark que compara LLMs na tarefa de classificação de e-mails de phishing*. [S.l.]: GitHub, 2026. <<https://github.com/Scheidt/phishing-llm-comparison>>. Acesso em: 4 jun. 2026.

SCHNEIER, B. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. 2. ed. New York: John Wiley & Sons, 1996. ISBN 0471128457.

SHANMUGARASA, Y.; DING, M.; ARACHCHIGE, C. M.; RAKOTOARIVELO, T. SoK: The privacy paradox of large language models: Advancements, privacy risks, and mitigation. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: Association for Computing Machinery, 2025. (ASIA CCS '25), p. 425–441.

SHARAFALDIN, I.; Habibi Lashkari, A.; GHORBANI, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *INSTICC. Proceedings of the 4th International Conference on Information Systems Security and Privacy - ICISPP*. [S.l.]: SciTePress, 2018. p. 108–116. ISBN 978-989-758-282-0. ISSN 2184-4356.

SHENG, Y.; ZHENG, L.; YUAN, B.; LI, Z.; RYABININ, M.; CHEN, B.; LIANG, P.; RÉ, C.; STOICA, I.; ZHANG, C. High-throughput generative inference of large language

models with a single GPU. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. [S.l.]: JMLR.org, 2023. Destaca a barreira de hardware para inferência de LLMs por usuários comuns.

SILVANO, W. F.; MARCELINO, R. Iota tangle: A cryptocurrency to communicate internet-of-things data. *Future Generation Computer Systems*, v. 112, p. 307–319, 2020. ISSN 0167-739X. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0167739X19329048>>.

SIVANESWARAN, D.; HEWAGE, C. T.; HERATH, H.; RATHORE, R. S.; SINGH, V. K.; JIANG, W. A systematic literature review of large language models in phishing attack generation and detection. *Array*, Elsevier, v. 30, p. 100775, 2026.

SOOD, R. *Phishing Email Dataset (Nazario 5 and TREC07)*. [S.l.]: Kaggle, 2024. <<https://www.kaggle.com/datasets/rohansood98/phishing-email-dataset-nazario-5-and-trec07>>. Acesso em: 6 jun. 2026.

SOULE, K.; BERGMANN, D. *IBM Granite 4.0 Tiny Preview: uma prévia da próxima geração de modelos Granite*. IBM, 2025. Licença Apache 2.0. Modelo disponível em Hugging Face: <<https://huggingface.co/ibm-granite/granite-4.0-tiny-preview>>. Disponível em: <<https://www.ibm.com/br-pt/new/announcements/ibm-granite-4-0-tiny-preview-sneak-peek>>.

SOUSA, L. C. P. d. *Anonimização como pilar de segurança em interações com modelos de linguagem: potenciais e limitações*. 109 p. Monografia (Trabalho de Conclusão de Curso) — Universidade Federal de Santa Catarina, Florianópolis, 2025. Graduação em Ciência da Computação, Centro Tecnológico.

SUN, L.; HUANG, Y.; WANG, H.; WU, S.; ZHANG, Q.; GAO, C.; HUANG, Y.; LYU, W.; ZHANG, Y.; LI, X. et al. Trustllm: Trustworthiness in large language models. *arXiv preprint arXiv:2401.05561*, 2024. Mapeia de forma atualizada os problemas de confiança decorrentes do controle de desenvolvedores sobre LLMs.

TARIQ, S.; CHHETRI, M. B.; NEPAL, S.; PARIS, C. Alert fatigue in security operations centres: Research challenges and opportunities. *ACM Computing Surveys*, Association for Computing Machinery, New York, NY, USA, v. 57, n. 9, p. 1–38, April 2025. ISSN 1557-7341. Published online: 12 March 2025; Accepted: 04 March 2025.

TRAD, F.; CHEHAB, A. Prompt engineering or fine-tuning? A case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction*, v. 6, n. 1, p. 367–384, 2024.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. [S.l.]: Curran Associates, Inc., 2017. Artigo fundamental da arquitetura Transformer.

WAN, Z.; WANG, X.; LIU, C.; ALAM, S.; ZHENG, Y.; LIU, J.; QU, Z.; YAN, S.; ZHU, Y.; ZHANG, Q. et al. Efficient large language models: A survey. *arXiv preprint arXiv:2312.03863*, 2023.

WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; XIA, F.; CHI, E.; LE, Q. V.; ZHOU, D. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, v. 35, p. 24824–24837, 2022.

- WEIDINGER, L.; UESATO, J.; RAUH, M.; GRIFFIN, C.; HUANG, P.-S.; MELLOR, J.; GLAESE, A.; CHEN, M.; BALLE, B.; KASIRZADEH, A. et al. Taxonomy of risks posed by language models. *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, 2022. Mapeia riscos organizacionais e danos causados por falhas de confiabilidade e segurança.
- WHITE, J.; FU, Q.; HAYS, S.; SANDBORN, M.; OLEA, C.; GILBERT, H.; ELNASHAR, A.; SPENCER-SMITH, J.; SCHMIDT, D. C. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, 2023.
- WU, Y.-c.; FENG, J.-w. Development and application of artificial neural network. *Wireless Personal Communications*, Springer, v. 102, n. 2, p. 1645–1656, 2018.
- XIAO, T.; ZHU, J. Foundations of large language models. *arXiv preprint*, 2025. Preprint técnico sobre fundamentos de LLMs.
- YANG, A.; LI, A.; YANG, B.; ZHANG, B.; HUI, B.; ZHENG, B.; YU, B.; GAO, C.; HUANG, C.; LV, C. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- YAO, Y.; DUAN, J.; XU, K.; CAI, Y.; SUN, Z.; ZHANG, Y. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, Elsevier, p. 100211, 2024.
- YVINEC, E.; CULLITON, P. *Gemma 3 QAT Models: Bringing State-of-the-Art AI to Consumer GPUs*. 2025. Google Developers Blog. Acessado em: 22 maio 2026. Disponível em: <<https://developers.googleblog.com/en/gemma-3-quantized-aware-trained-state-of-the-art-ai-to-consumer-gpus/>>.
- ZHANG, J.; WU, P.; LONDON, J.; TENNEY, D. Benchmarking and evaluating large language models in phishing detection for small and midsize enterprises: A comprehensive analysis. *IEEE Access*, IEEE, v. 13, p. 28335–28352, February 2025. ISSN 2169-3536. Received: January 24, 2025; Accepted: February 5, 2025; Published: February 10, 2025.
- ZHAO, W. X.; ZHOU, K.; LI, J.; TANG, T.; WANG, X.; HOU, Y.; MIN, Y.; ZHANG, B.; ZHANG, J.; DONG, Z. et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, v. 1, n. 2, p. 1–124, 2023.

Apêndices

APÊNDICE A – PROTOCOLO DE REVISÃO SISTEMÁTICA

Quadro 9 – Triagem dos estudos recuperados na base Scopus.

Título	Ano	h5-index	Decisão	Filtro	Motivo
Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection with Large Language Models	2024	40	Excluído	2 - Escopo Divergente	Trata de comparação com Fine-tuning
An explainable transformer-based model for phishing email detection: A large language model approach	2026	99	Excluído	2 - Escopo Divergente	Foca em explainable AI
From Chatbots to Phishbots?: Phishing Scam Generation in Commercial Large Language Models	2024	110	Excluído	2 - Escopo Divergente	Geração de Phishing
KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection	2024	116	Excluído	2 - Escopo Divergente	Criação de um sistema próprio
Robust cybersecurity through discrete and large language models for effective phishing attack detection	2025	33	Excluído	3 - Sem avaliação empírica comparativa	Aplica BERT em um sistema próprio, sem comparar com outros modelos

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
ChatPhishDetector: Detecting Phishing Sites Using Large Language Models	2024	288	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Next-Generation Phishing: How LLM Agents Empower Cyber Attackers	2024	55	Excluído	2 - Escopo Divergente	Geração de Phishing
A System for Real-Time Classification of Phishing Emails Using Large Language Models (LLMs) with Natural Language Processing (NLP)	2025	25	Excluído	3 - Sem avaliação empírica comparativa	Criação de um sistema próprio
From ML to LLM: Evaluating the Robustness of Phishing Web Page Detection Models against Adversarial Attacks	2025	23	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Multimodal Large Language Models for Phishing Webpage Detection and Identification	2024	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Lateral Phishing With Large Language Models: A Large Organization Comparative Study	2025	288	Excluído	2 - Escopo Divergente	Não tem a ver com o tema

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
To Ensemble or Not: Assessing Majority Voting Strategies for Phishing Detection with Large Language Models	2025	12	Excluído	1 - H5	H5 inferior a 20
LMs go Phishing: Adapting Pre-trained Language Models to Detect Phishing Emails	2022	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
SecureNet: A Comparative Study of DeBERTa and Large Language Models for Phishing Detection	2024	14	Excluído	1 - H5	H5 inferior a 20
Utilizing Large Language Models with Human Feedback Integration for Generating Dedicated Warning for Phishing Emails	2024	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
The Impact of Emerging Phishing Threats: Assessing Quishing and LLM-generated Phishing Emails against Organizations	2025	41	Excluído	2 - Escopo Divergente	Trata de Quishing, e do impacto de e-mails, não da detecção deste segundo
Evaluating the Efficacy of Large Language Models in Identifying Phishing Attempts	2024	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
SpearBot: Leveraging large language models in a generative-critique framework for spear-phishing email generation	2025	143	Excluído	2 - Escopo Divergente	Modalidade de phishing diferente, foco em gerar e-mails
Benchmarking and Evaluating Large Language Models in Phishing Detection for Small and Midsize Enterprises: A Comprehensive Analysis	2025	288	Incluído	—	Atende aos critérios de inclusão
Can AI Keep You Safe? A Study of Large Language Models for Phishing Detection	2024	43	Incluído	—	Atende aos critérios de inclusão
Phishing detection in IoT: an integrated CNN-LSTM framework with explainable AI and LLM-enhanced analysis	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Generating Personalized Phishing Emails for Social Engineering Training Based on Neural Language Models	2023	40	Excluído	2 - Escopo Divergente	Geração de Phishing
LLM-Driven SAT Impact on Phishing Defense: A Cross-Sectional Analysis	2024	27	Excluído	2 - Escopo Divergente	Não tem a ver com o tema

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
ChatSpamDetector: Leveraging Large Language Models for Effective Phishing Email Detection	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Stacking Large Language Models is All You Need: A Case Study on Phishing Url Detection	2025	21	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
The Dual-Edged Sword of Large Language Models in Phishing	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Leveraging Human Knowledge in Large Language Model for Obfuscation-Resisted Phishing URL Detection	2024	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Cacography based Ransomware Email Phishing Attack Prevention using Language Pack Tuned Transformer Language Model	2025	234	Excluído	2 - Escopo Divergente	Não tem a ver com o tema
Generating Phishing Attacks and Novel Detection Algorithms in the Era of Large Language Models	2024	55	Excluído	2 - Escopo Divergente	Geração de Phishing
A Survey on Large Language Models in Phishing Detection	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
Securing the Weakest Link: Exploring Affective States Exploited in Phishing Emails With Large Language Models	2025	288	Excluído	3 - Sem avaliação empírica comparativa	Foca na detecção de estados afetivos (medo, urgência, ganância, curiosidade, confiança, compaixão) dentro de e-mails já sabidamente de phishing.
Fraud-R1 : A Multi-Round Benchmark for Assessing the Robustness of LLM Against Augmented Fraud and Phishing Inducements	2025	236	Excluído	3 - Sem avaliação empírica comparativa	Não trata de e-mails, e trata de ataques além de phishing
Phishing Email Classification Using Few-Shot Learning with a Lightweight LLM (Llama 3.2)	2025	32	Excluído	3 - Sem avaliação empírica comparativa	Cria um sistema próprio, sem avaliar LLMs comparativamente
PhishParrot: LLM-Driven Adaptive Crawling to Unveil Cloaked Phishing Sites	2025	58	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Evaluating the Effectiveness of Large Language Models (LLMs) Versus Machine Learning (ML) in Identifying and Detecting Phishing Email Attempts	2025	56	Excluído	3 - Sem avaliação empírica comparativa	Avalia LLMs contra modelos de Machine Learning, não entre si

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
Can You Walk Me Through It? Explainable SMS Phishing Detection using LLM-based Agents	2025	35	Excluído	2 - Escopo Divergente	Foca em explainable AI e Smishing
Demo: LLM-Based Browser Extension for Phishing Detection	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Synthesizing Deception: Countering Large Language Model-Generated Phishing Campaigns through Adaptive Semantic Anomaly Detection	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Enhanced Voice Phishing Detection Using an LLM-Based Framework for Data Augmentation and Classification	2025	288	Excluído	2 - Escopo Divergente	Foca em Phishing por voz
HyperSeq-LLM: Evolving hypergraph sequence learning with LLMs for phishing scam detection on Ethereum	2026	183	Excluído	2 - Escopo Divergente	Não tem a ver com o tema
Improving online anti-phishing training using cognitive large language models	2026	160	Excluído	2 - Escopo Divergente	Foca em melhorar o treinamento de usuários contra phishing

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
Supporting the Design of Phishing Education, Training and Awareness interventions: an LLM-based approach	2024	27	Excluído	2 - Escopo Divergente	Foca em melhorar o treinamento de usuários contra phishing
David versus Goliath: Can Machine Learning Detect LLM-Generated Text? A Case Study in the Detection of Phishing Emails	2024	27	Excluído	3 - Sem avaliação empírica comparativa	Testa Machine Learning Clássico e foca na detecção de texto gerado por LLMs
Smart-Shield: A Hybrid ML-AI Framework for Advanced Phishing Detection using Gradient Boosting and LLM Analysis	2025	15	Excluído	1 - H5	H5 inferior a 20
A method for detecting financial phishing in instant messengers using an ensemble of dialogical intelligent assistants based on large language models; [Metod vyivlennia finansovoho fishynhu v instant-mesendzherakh za dopomohoiu ansamblu dialohovykh intelektualnykh pomichnykiv na bazi velykykh movnykh modelei]	2026	12	Excluído	1 - H5	H5 inferior a 20

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
Generative Phishing URL Detection Based on Large Language Model	2025	10	Excluído	1 - H5	H5 inferior a 20
User-Centric Phishing Detection: A RAG and LLM-Based Approach	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Leveraging Large Language Models for Detection and Intelligent Analysis of Phishing Threats	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Demo: PhishSense: A LLM-Enhanced Multimodal Framework for Phishing Website Detection	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Multi-Agent Phishing Detection And Deletion via Small VLM and LLM Reasoning	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
When semantic plausibility becomes a liability: LLM-based phishing detection from an adversarial asymmetry perspective	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Toward accurate and cost-effective LLM agents via information flow optimization: Insights from a phishing detection case study	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
LLM-Driven Data Augmentation with Contrastive Learning to Address Mobile Phishing Attacks: A Persuasion-Theory Approach	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
A GNN-LLM Hybrid Framework for Phishing URL Detection Using Textual Features	2026	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Email Phishing Detection Using BERT Large Language Model	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar
Enhancing Business-Specific Phishing Chat Detection via Few-Shot Learning LLM Augmentation	2025	288	Excluído	2 - Escopo Divergente	Não foca em phishing por e-mail
Enhancing Phishing Detection with AI: A Novel Dataset and Comprehensive Analysis Using Machine Learning and Large Language Models	2024	25	Excluído	3 - Sem avaliação empírica comparativa	Criação de dataset
Evaluating large language models' ability to automate spear phishing	2026	183	Excluído	2 - Escopo Divergente	Foca em spear-phishing

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
Psychology of phishing emails: Quantifying persuasion principles and simulating detection with large language models	2026	183	Excluído	3 - Sem avaliação empírica comparativa	Avaliação Psico-Linguística de Phishing com LLM
LLM-structured expert feedback for knowledge calibration of deep learning model to detect phishing URL	2026	144	Excluído	2 - Escopo Divergente	Treino de modelos
You Got Phished! Analyzing How to Provide Useful Feedback in Anti-Phishing Training with LLM Teacher Models	2025	141	Excluído	2 - Escopo Divergente	Foca em melhorar o treinamento de usuários contra phishing
LLM-Based Multimodal Feature Extraction and Hierarchical Fusion for Phishing Email Detection	2026	141	Excluído	2 - Escopo Divergente	Criação de Sistema Próprio Multimodal e Detecção de Sites
Towards Explainable and Educational Phishing Detection: A Zero-Shot LLM Approach	2025	55	Excluído	2 - Escopo Divergente	Foca em melhorar o treinamento de usuários contra phishing e explainable Ai
PhishDebate: An LLM-Based Multi-Agent Framework for Phishing Website Detection	2025	55	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
Phish-Master: Leveraging Large Language Models for Advanced Phishing Email Generation and Detection	2025	188	Excluído	3 - Sem avaliação empírica comparativa	Criação de dataset
Identifying Phishing Attacks Using URL-Centric Approaches with Character-Conscious Transformer-Based Language Model	2024	40	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
A Machine Learning Approach to Detecting Phishing Attacks Using LLM-Based Embeddings	2026	40	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Phishing Email Detection using Header Features Leveraging Large Language Models (LLMs)	2026	40	Excluído	3 - Sem avaliação empírica comparativa	Paywalled
A systematic literature review of large language models in phishing attack generation and detection	2026	39	Excluído	3 - Sem avaliação empírica comparativa	Revisão da literatura
LLM-Powered Intent-Based Categorization of Phishing Emails	2025	20	Excluído	1 - H5	Ainda não publicado, estando em estado de pre-impressão

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
Small Language Models for Phishing Website Detection: Cost, Performance, and Privacy Trade-Offs	2026	33	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Benchmarking 21 Open-Source Large Language Models for Phishing Link Detection with Prompt Engineering	2025	94	Excluído	2 - Escopo Divergente	Detecção de Sites de Phishing
Empirical Evaluation of Machine Learning Algorithms and T5 Language Model for Advanced Phishing Email Detection	2025	40	Excluído	3 - Sem avaliação empírica comparativa	Paywalled
Turkish Email Phishing Detection Using Fine Tuned Large Language Models	2025	25	Excluído	2 - Escopo Divergente	Trata de comparação com Fine-tuning
Leveraging Large Language Models for Enhanced URL Phishing Detection	2026	22	Excluído	2 - Escopo Divergente	Detecção de URL de Phishing
Phishing email uniform resource locator detection based on large language model	2025	21	Excluído	2 - Escopo Divergente	Foca na detecção por meio de URL
Devising and Detecting Phishing Emails Using Large Language Models	2024	288	Incluído	—	Atende aos critérios de inclusão

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
Data Augmentation for Messenger Phishing Detection Using Large Language Models	2025	20	Excluído	2 - Escopo Divergente	Phishing por mensagens
CLASP: Cost-Optimized LLM-based Agentic System for Phishing Detection	2025	19	Excluído	1 - H5	H5 inferior a 20
Hybrid Phishing Detection System: Integrating URL Feature Analysis with Local Large Language Models for Enhanced Security	2025	17	Excluído	1 - H5	H5 inferior a 20
The Emotional Lens of Cybersecurity: A Study on Sentiment Analysis Enhanced Phishing Detection via Large Language Models	2025	15	Excluído	1 - H5	H5 inferior a 20
Phishing Website Detection Method Based on Multimodal Large Language Model	2025	14	Excluído	1 - H5	H5 inferior a 20
Hybrid Spear-Phishing Email Detection with LLM and Machine Learning	2025	12	Excluído	1 - H5	H5 inferior a 20

continua na próxima página

continuação do Quadro 9

Título	Ano	h5-index	Decisão	Filtro	Motivo
DRILL: Dual-Reasoning Large Language Models for Phishing Email Detection with Limited Data	2025	12	Excluído	1 - H5	H5 inferior a 20
Phishing Email Detection Using Large Language Models: A Comparative Analysis of Standalone and Retrieval Augmented Generation Models	2025	11	Excluído	1 - H5	H5 inferior a 20
LLM Agent-Based Simulations for Phishing Email Detection in Various Multi-Agent Sizes	2026	6	Excluído	1 - H5	H5 inferior a 20
Application of Large Language Models for Detecting Fraud and Phishing Messages Indonesian Language	2025	6	Excluído	1 - H5	H5 inferior a 20
Developing an Anti-Phishing Large Language Model: A Focus Group Study on Human, Technological, and Legal Factors	2024	3	Excluído	1 - H5	H5 inferior a 20
A Comparative Analysis of BERT and LLM Models for Advanced Phishing Detection	2025	n/d	Excluído	1 - H5	H5 indisponível no Google Scholar

continua na próxima página

continuação do Quadro 9

Título	Ano	h5- index	Decisão	Filtro	Motivo
How effective are large language models in detecting phishing emails?	2024	16	Excluído	1 - H5	H5 inferior a 20
A Culturally Aware LLM Framework for Analyzing Social Engineering Tactics in Korean Phishing Messages	2026	141	Excluído	2 - Escopo Divergente	Específico para Coreano

Fonte: Elaborado pelo autor, a partir de dados extraídos da base Scopus (2026).

APÊNDICE B – CONFIGURAÇÕES DO AMBIENTE EXPERIMENTAL

O *digest* SHA256 identifica de forma única o artefato de cada modelo local baixado, fixando a versão exata empregada nos experimentos e tornando a configuração reproduzível. O Quadro 10 reúne esses identificadores. O modelo comercial Claude Haiku 4.5, acessado via *API*, não expõe esse identificador.

Modelo (<i>tag</i>)	<i>Digest</i> SHA256
<code>gemma3-qat:4B-Q4_K_M</code>	<code>b3752411b3bb15e4606609f9de8d525ee17c262efbec03d003d557edb1ae9b85</code>
<code>qwen3:4B-UD-Q8_K_XL</code>	<code>cc292606149733c2d0e598d5b4bc166d86b1b4ce4ef02aed4beea283ea9a8aa2</code>
<code>granite-4.0-h-tiny:7B-Q4_K_M</code>	<code>c7b6e5774260ca7c903bc609a5b9b01ac11b93b3baadc7062a1a026df83db8c7</code>
<code>llama3.2:3B-Q4_0</code>	<code>da80a841836d36cd6e93239d7d1d3c6352defc90275e238d8c99ff131f0ec96c</code>

Fonte: Elaborado pelo autor.

Quadro 10: *Digests* SHA256 dos artefatos dos modelos locais.